NGW

NGW Documentation

Complete Suite — All 15 Documents

NO GUESSWORK LIGHTING

v1.0 · March 2026 · Confidential

NGW

Table of Contents

DOCUMENT 01

Product Overview

What NGW is, what it does, and who it's built for.

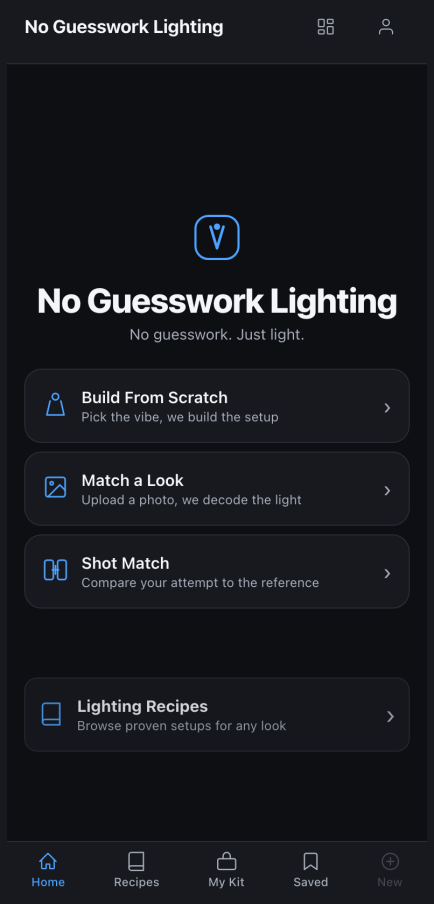
NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

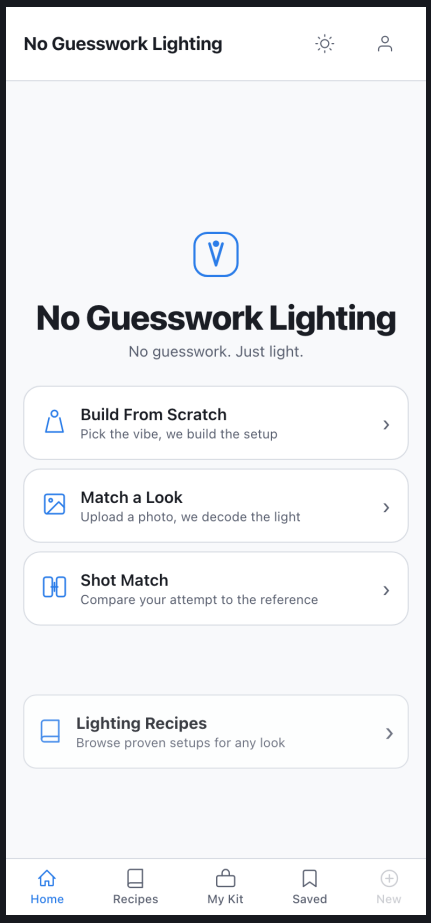
What is No Guesswork Lighting?

No Guesswork Lighting (NGW) is a deterministic lighting recommendation engine. Upload any reference photo — portrait, editorial, product — and NGW instantly analyzes its lighting, matches it against 30+ canonical patterns, and delivers a precise setup guide tailored to your gear.

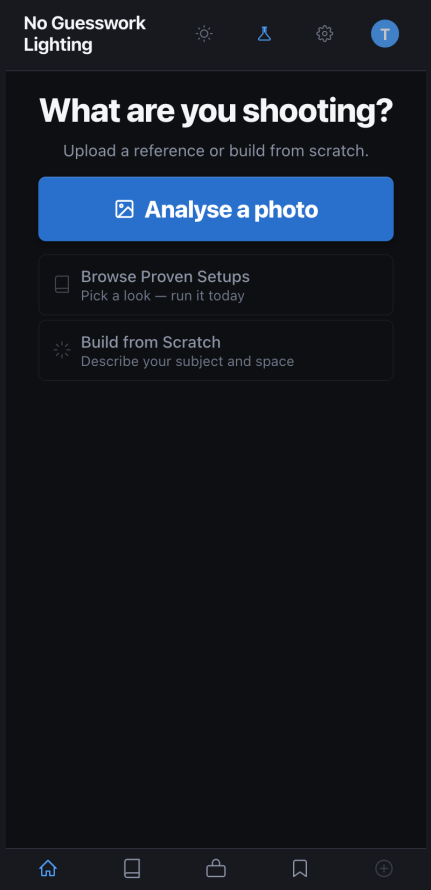
There is no guessing. No forum searching. No trial-and-error. You get exact positions, distances, angles, modifier choices, and power settings — all derived from the image itself.



NGW — Welcome screen, dark theme



NGW — Welcome screen, light theme



NGW Home — three entry points: Analyze, Shoot Mode, My Kit

Core Capabilities

- Reference Photo Analysis — Upload any reference image; NGW identifies the lighting pattern, modifiers, and geometry.
- Pattern Matching — Compares against 30+ patterns: clamshell, loop, Rembrandt, butterfly, Caravaggio, and more.
- Gear-Aware Recommendations — Filters suggestions through your actual equipment across 5 flexibility tiers.
- Shoot Mode — Converts a recommendation into step-by-step on-set instructions for photographers and assistants.
- Recipes — 13 pre-built master lighting setups as learning references and starting points.
- NGW Lab — Developer tools for curating gold-set test images and reviewing engine improvement candidates.
- Signal System — Tracks real user outcomes to power continuous, safety-gated learning.

Who It's For

Audience	Primary Use Case	Key Benefit
Photographers	Replicate reference lighting	Save hours of setup time on location
Assistants	Execute lighting instructions	Clear, role-specific step guidance
Studios	Maintain consistent look & feel	Repeatable gear configurations
Educators	Teach lighting principles	Named patterns + visual examples

Retouchers	Understand source lighting	Pattern detection from final images

Technical Highlights

- Vision-Powered — Multi-layer analysis pipeline combining image processing and VLM interpretation.
- Deterministic Output — Same reference image always produces the same recommendation baseline.
- Consensus Solver — Multiple analysis passes are merged through a weighted consensus algorithm.
- 5-Tier Gear Matching — From exact-match gear to ambient-only fallback, always a useful output.
- Signal Hygiene — Live, seeded, internal, and expert signals are tracked and filtered separately.
- Auto-Candidate Generation — Failure clusters automatically surface as reviewable improvement proposals.

NOTE

- NGW recommendations are deterministic by design. The system does not randomly vary output. Variation comes only from changes to your kit, the reference image, or engine updates.

DOCUMENT 02

User Guide

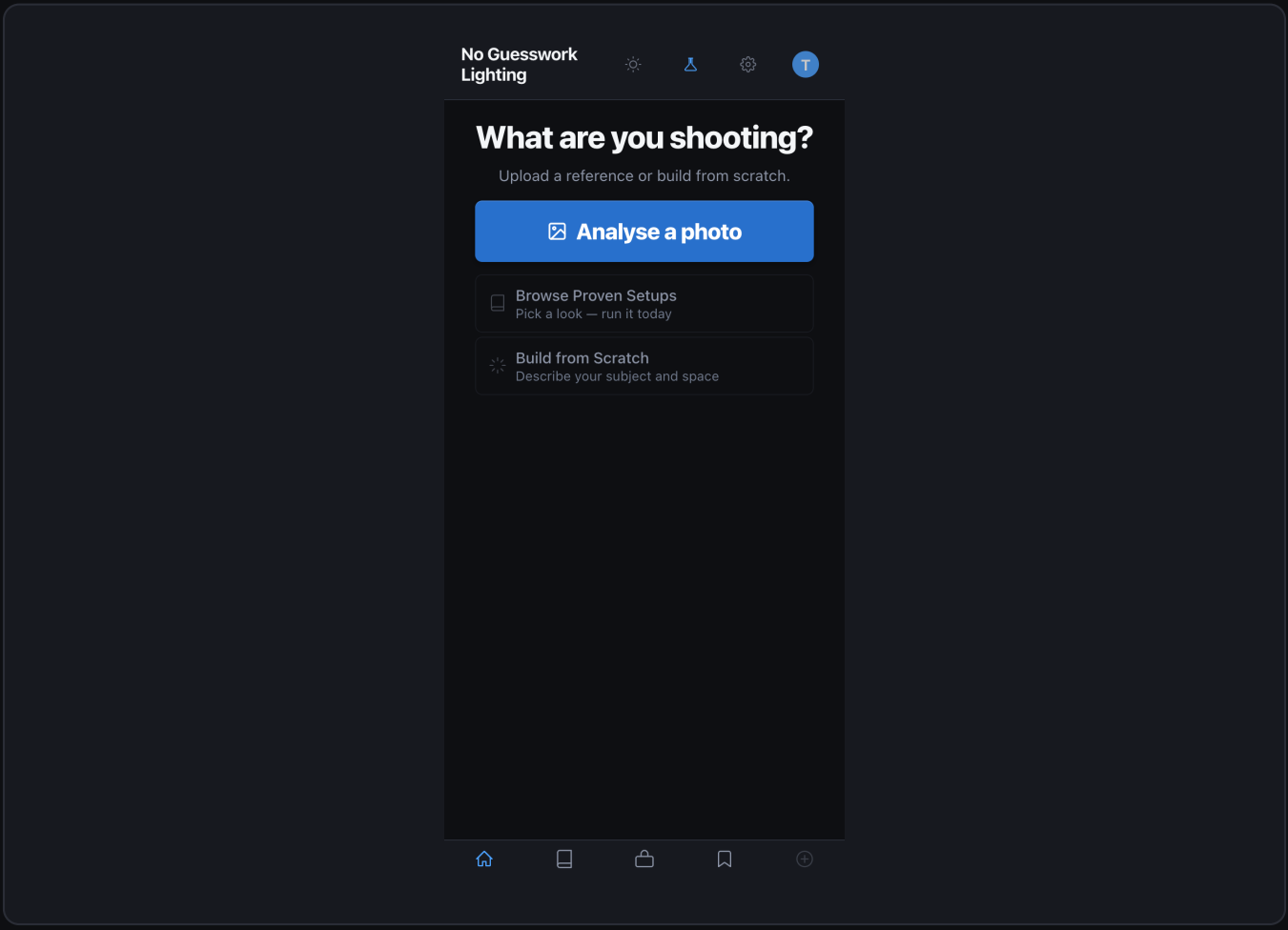
Screen-by-screen guide to every feature in NGW.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Application Overview

NGW is organized around three primary workflows: Analyze, Shoot Mode, and My Kit. Access each from the Home screen. Settings and the NGW Lab are available via the navigation bar at all times.



Home Screen — Analyze, Shoot Mode, and My Kit entry points

Home Screen

Entry Points

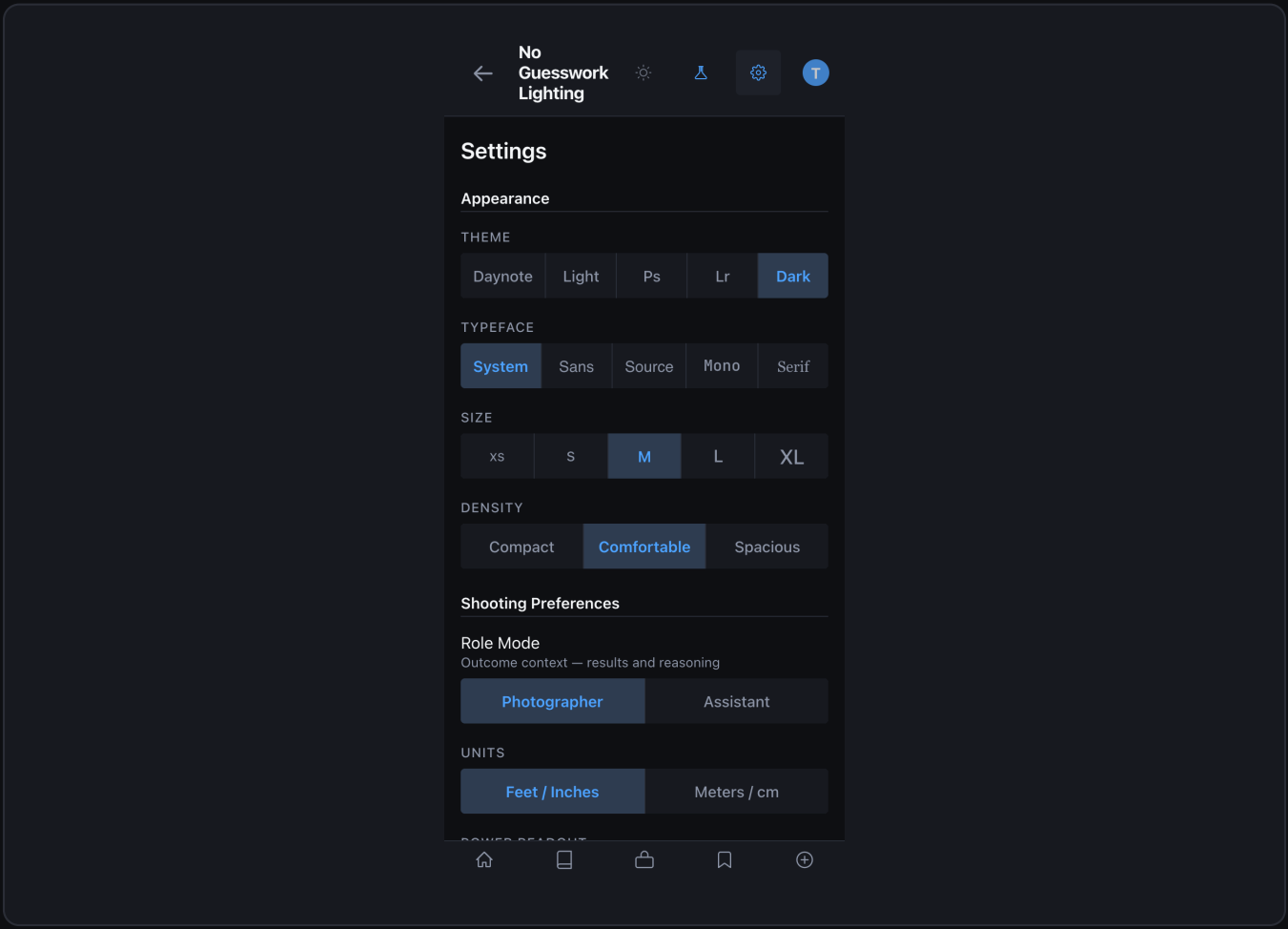
- Analyze — Upload a reference photo and receive an immediate lighting recommendation.
- Shoot Mode — Convert a recommendation into a step-by-step on-set workflow.
- My Kit — Manage the equipment list that constrains every recommendation.

Navigation Bar

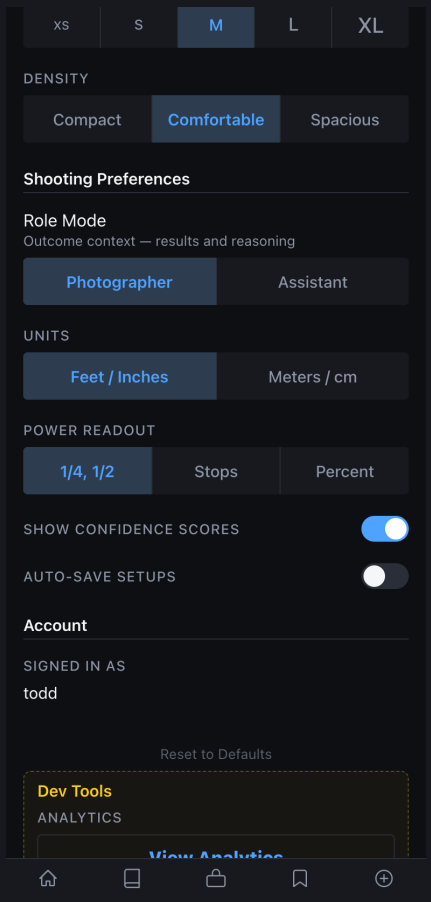
- Home — Return to the main entry screen.
- Recipes — Browse 13 master lighting recipes as learning references.
- Settings — Configure units, notifications, and account preferences.
- NGW Lab — Developer tools (restricted — see Lab Guide).

Settings

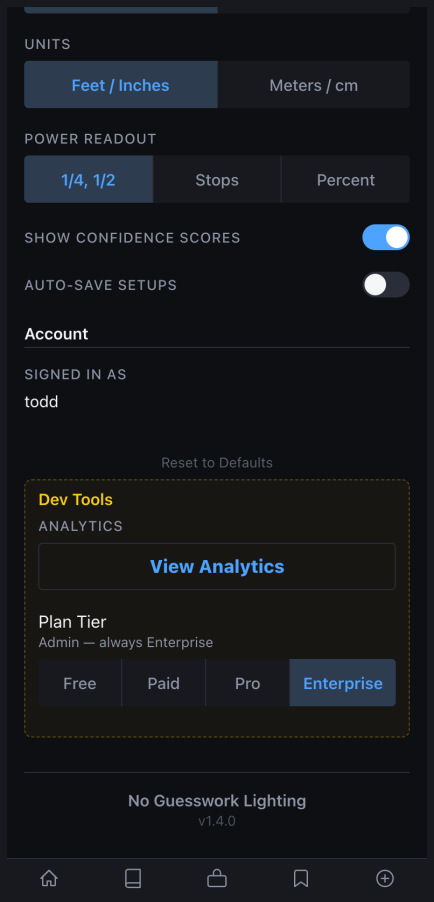
Settings controls system-wide preferences and account details. Changes take effect immediately — no restart required.



Settings Screen — units, notifications, account, and lab access



Settings — Lab access toggle (restricted to allowlisted emails)



Settings — Developer tools panel with debug options

Configurable Options

- Distance Units — Switch between feet and meters for all positional guidance.
- Ceiling Height Default — Set your studio's ceiling height; Shoot Mode warns when lights exceed it.
- Room Dimensions — Pre-set your shooting space for proximity and wall-bounce guidance.
- Notification Preferences — Control when NGW sends analysis-complete and outcome reminders.
- Developer Tools — Enable the NGW Lab (restricted to approved team members).

How To: Change Your Distance Units

NGW supports both feet and metric units. All positional guidance in Shoot Mode and recommendations updates instantly when you change the setting.

1. **Open Settings**
   Tap the gear icon in the navigation bar from any screen.

2. **Find Distance Units**
   Scroll to the "Units & Measurements" section.

3. **Select your unit**
   Toggle between Feet and Meters. The label updates live.

4. **Return to Analyze**
   All distances in the current recommendation re-render immediately.

How To: Set Up Your Studio Space

Entering your room dimensions lets Shoot Mode flag when a light position is too close to a wall, or when a boom arm would exceed your ceiling height.

1. **Open Settings**
   Tap the gear icon in the navigation bar.

2. **Enter Ceiling Height**
   Type your ceiling height in feet or meters. Include any drop from beams.

3. **Enter Room Dimensions**
   Width and depth in your preferred units. Approximate is fine.

4. **Save**
   Tap outside the field or hit Done on the keyboard. Settings persist across sessions.

WHY THIS MATTERS

• Without room dimensions, Shoot Mode cannot warn you about wall-proximity bounce, ceiling clearance, or spill risk from nearby reflective surfaces. Takes 30 seconds and significantly improves instruction accuracy.

Running Your First Analysis

1. **Open the Analyze screen**
   Tap "Analyze" from the Home screen.

2. **Upload a reference image**
   Choose a photo with clear subject lighting. Portraits work best; avoid heavy post-processi…ng.

3. **Set your scene context**
   Specify subject type (portrait / product), environment (studio / outdoor), and any constra…ints.

4. **Review the recommendation**
   NGW returns the detected pattern, confidence score, modifier list, and gear setup.

5. **Launch Shoot Mode**
   Tap "Start Shoot Mode" to convert the recommendation into on-set steps.

PRO TIP

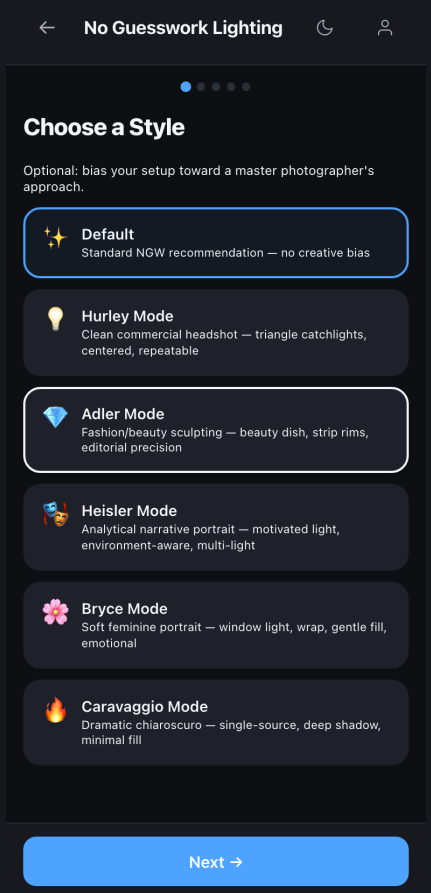
• For best results, use a reference photo with a single subject against a neutral background. Heavy color grading or composite backgrounds reduce detection confidence.

Reading a Recommendation

Field	What It Means	Example
Pattern	Named lighting configuration detected	Loop — Key 45° off-axis, slight elevation
Confidence	Engine certainty (0–100)	87%
Key Light	Primary source: type, position, modifier	Godox AD600 · 45°R · 6 ft · Octa 90cm
Fill Light	Secondary source or reflector	V-flat reflector · camera-left
Modifiers	Shaping tools detected	Diffusion sock · grid
Power	Estimated flash stop relative to key	Key: f/8 · Fill: –2 stops
Gear Tier	Flexibility applied to your kit	Tier 2 — Close substitute

How To: Use a Master Mode

Master Mode lets you bias recommendations toward the aesthetic of a specific photographer — adjusting contrast ratios, modifier softness, and tonal guidance to match a signature look.



Wizard — Master Mode selection screen

1 **Start an analysis**
Upload a reference image from the Analyze screen.

2 **Open Master Mode**
After the initial result appears, tap the "Master Mode" option.

3 **Select a photographer**
Choose from Avedon, Penn, Lindbergh, LaChapelle, and others.

4 **Review adjusted output**
The recommendation re-renders with the selected master's aesthetic biases applied.

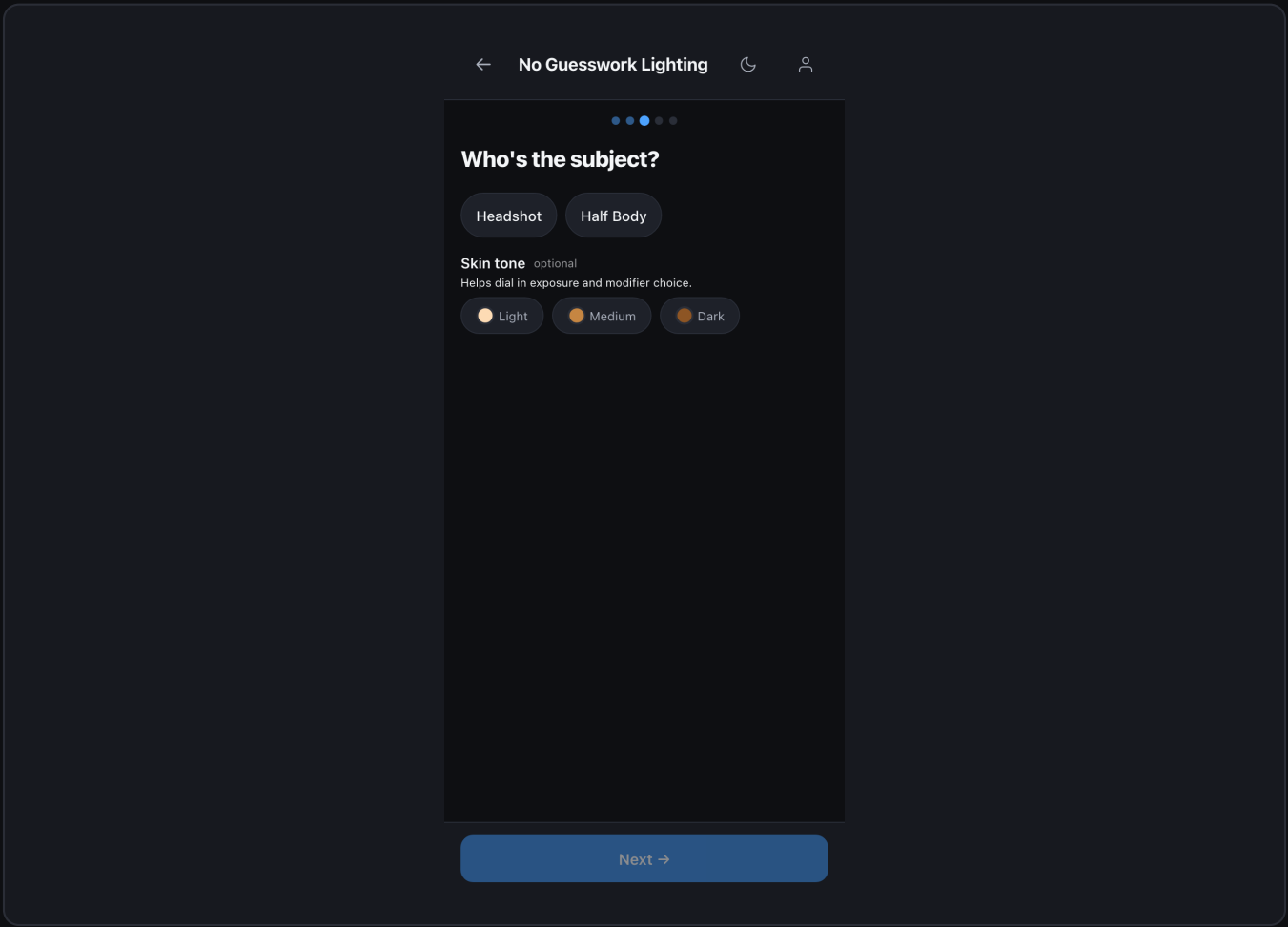
5 **Toggle off**
Tap "None" to return to the unbiased recommendation at any time.

WHAT CHANGES

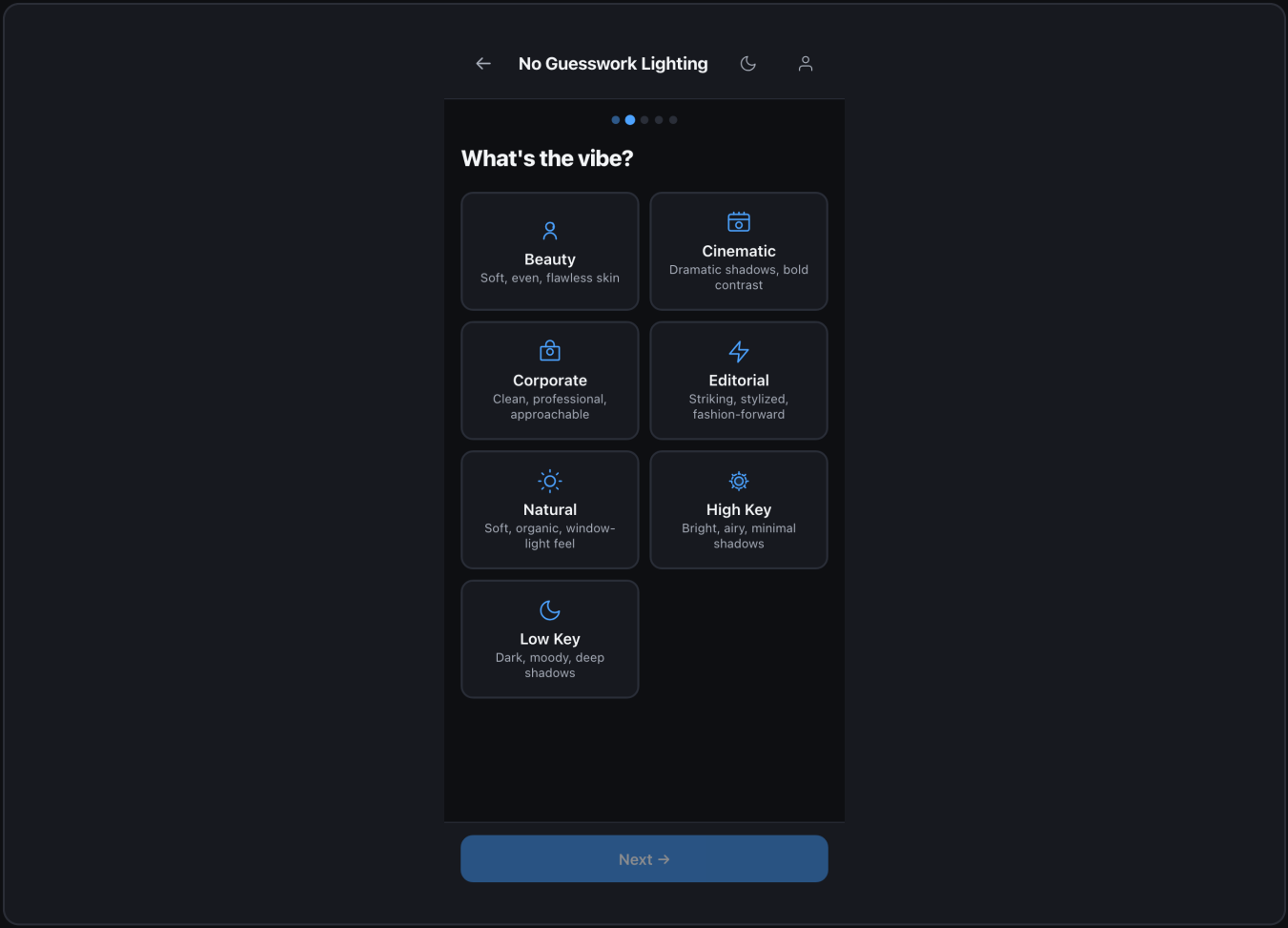
- Contrast ratio target shifts to match the master's signature fill ratio.
- Modifier softness guidance adjusts — Penn favors harder sources; Avedon softer.
- Copy in the recommendation reflects the visual philosophy of the selected master.

How To: Set Subject and Mood

NGW adjusts recommendations based on subject type and mood context. Setting these before analysis improves pattern confidence and modifier selection.



Wizard — Subject type selection



Wizard — Mood and tone selection

Subject Types

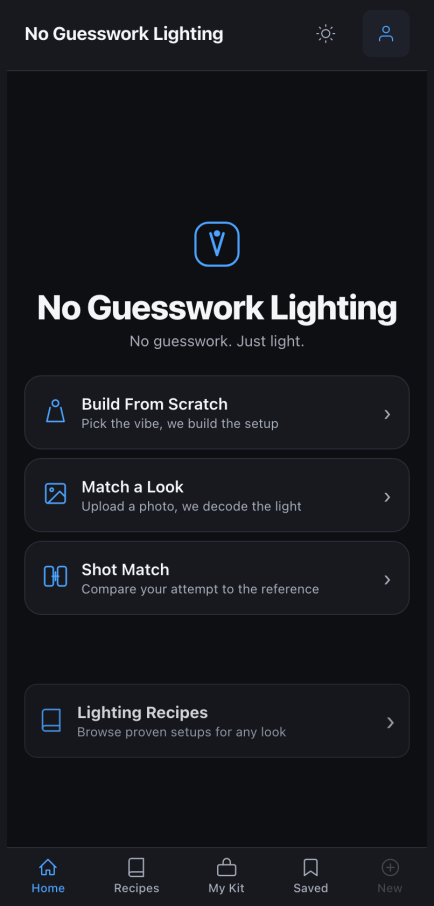
- Portrait — Individual subject, face-priority analysis.
- Editorial — Fashion or lifestyle; full-body and environment considered.
- Product — Still life; specular and reflection signature prioritized.
- Group — Multiple subjects; pattern limited to wrap-friendly setups.

Mood Presets

- Natural — Prioritizes believable, soft, directional light.
- Dramatic — High contrast, low fill ratio, deep shadows.
- Commercial — Clean, on-axis, even, highly controlled.
- Editorial — Edgy, cross-light, mixed source temperature acceptable.

How To: Enable NGW Lab Access

The NGW Lab is restricted to approved team members. Access requires both a Studio plan and an email on the allowlist configured in the backend environment.



Home screen with NGW Lab tab visible

1 **Confirm eligibility**

Your email must be in LAB_ALLOWED_EMAILS on the backend. Contact your team admin.

2 **Open Settings**
Tap the gear icon in the navigation bar.

3 **Enable Developer Tools**
Scroll to the bottom. Toggle "Developer Tools" on.

4 **Enter your lab email**
Type the email registered in the allowlist.

5 **Lab tab appears**
The NGW Lab tab appears in the navigation bar immediately.

TROUBLESHOOTING LAB ACCESS

• If the Lab tab does not appear after enabling Developer Tools, your email is not in the allowlist. Ask your backend admin to add it to LAB_ALLOWED_EMAILS in the server environment variables and restart the backend.

DOCUMENT 03

Shoot Mode Guide

Your on-set playbook — step-by-step lighting execution.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

What is Shoot Mode?

Shoot Mode transforms an NGW recommendation into an ordered, role-aware list of on-set instructions. Instead of reading a gear list and figuring out placement yourself, Shoot Mode tells you exactly where to stand each light, at what height, what angle, and what power — and then walks you through your first test exposure.

Three Role Views

- Photographer — Full sequence: camera setup light placement test exposure adjustments.
- Assistant — Light-placement steps only. No camera settings.
- Second Shooter — Camera setup and test exposure only. No light placement.

Pre-Shoot Setup

Before launching Shoot Mode, confirm these inputs are correct. Errors here propagate through every step.

REQUIRED BEFORE LAUNCH

- My Kit is up to date — add any new gear; remove anything unavailable today.
- Ceiling height is set in Settings — Shoot Mode warns if a light position exceeds it.
- Room dimensions are entered — enables wall-proximity and bounce guidance.
- Reference analysis is complete — Shoot Mode requires an active recommendation.

Optional Inputs

- Master Mode — Select a photographer reference (e.g., Avedon, Penn, Lindbergh) to bias tone suggestions.
- Session Label — Name the session for future reference and outcome tracking.
- Estimated Duration — Helps NGW weight urgency in step ordering.

The Shoot Mode Workflow

1

Camera Setup

NGW gives aperture, ISO, shutter speed starting points matched to the pattern.

2

Place Key Light

Position the primary source at exact angle, height, and distance. Wall-proximity notes inc… luded.

3

Place Fill Light

Add fill or reflector. Shoot Mode specifies camera-side placement and fill ratio.

4

Add Accent Lights

Background, hair, rim, and kicker lights added in order of priority.

5

Set Power Levels

Starting power suggestions for each source, referenced to f/8 at key.

6

Take a Test Shot

Checklist of what to look for: catch-lights, shadow direction, fill ratio on the face.

7

Evaluate & Adjust

NGW compares your test shot result against the pattern and suggests corrections.

Step Detail — Light Placement

Each light-placement step includes all measurements in your preferred unit system. Distance is given in feet, meters, arm-lengths, and walking steps simultaneously for fieldwork without a tape measure.

Field	Example Value	Usage
Angle	45° camera-right	Stand in front of subject; count 45° off-center
Height	7.5 ft (arms + 1 step up)	Top of stand; flag if above ceiling
Distance	6 ft / 1.8 m / ~4 steps	Subject to light source front element
Wall proximity	3.2 ft from left wall	Bounce and spill warning zone
Modifier	Octa 90cm + diffusion sock	Attach before powering on
Power hint	Start at 1/4 power (~f/8)	Full-stop guide only — meter to confirm

Test Shot Evaluation

After your test shot, NGW can analyze the image and compare it to the target pattern. Upload the test shot via the evaluate button in Shoot Mode.

What NGW Checks

- Catch-light position — matches expected angle for the pattern.
- Shadow direction — loop vs. Rembrandt vs. butterfly shadow fall.
- Fill ratio — side of face in shadow vs. fill, compared to target.
- Background separation — key vs. background ratio estimate.
- Specular highlights — modifier signature matches (harsh vs. soft).

FIELD NOTE

- If your ceiling is below 9 ft, Shoot Mode will warn on any step that requires a boom arm above ceiling height. It will automatically suggest floor-level or camera-mounted alternatives.

DOCUMENT 04

Recipes Guide

Thirteen master lighting setups, pre-built and ready to shoot.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

What Are Recipes?

Recipes are 13 pre-built lighting setups curated by master photographers. Each recipe packages a named lighting pattern with specific modifier choices, gear requirements, and a philosophy statement explaining why the setup works.

Recipes are not user-editable templates. They are reference setups — learning tools that show how canonical lighting patterns translate to real gear decisions. They also serve as priors for the recommendation engine.

KEY DISTINCTION

• Recipes are read-only reference material. Use them to learn patterns, not to replace custom analysis. For your specific reference image, always run Analyze first.

The 13 Master Recipes

Recipe Name	Pattern	Difficulty	Gear Flex
Beauty Clamshell	Clamshell	Medium	2/5
Dramatic Rembrandt	Rembrandt	Hard	3/5
Clean Corporate Headshot	Loop	Easy	4/5
Editorial Hard Light	Hard Direct	Medium	3/5
Natural Window Light Look	Window / Soft Ambient	Easy	5/5
High Key Product	Flat High Key	Medium	3/5

Low Key Moody	Split / Rembrandt	Hard	3/5
Butterfly / Paramount	Butterfly	Medium	3/5
Caravaggio Chiaroscuro	Single Hard Source	Expert	2/5
Vermeer Window	Directional Soft	Medium	4/5
Avedon High-Key	Flat Shadowless	Hard	2/5
Penn Precision Beauty	Modified Clamshell	Expert	2/5
Heisler Adaptive Rembrandt	Rembrandt + Fill	Hard	3/5

Recipe Detail — Beauty Clamshell

The Beauty Clamshell is the canonical beauty portrait setup: a large soft source directly above camera axis, a reflector or second source directly below, creating symmetric shadow fill and a distinctive double catch-light.

Field	Value
Pattern	Clamshell
Key Light	Large octa or beauty dish above camera axis — 12–18 inches above lens level
Fill	White reflector or strip box below lens — fills under chin and cheekbone shadows
Modifiers	Beauty dish + diffusion sock OR 100cm octa; fill V-flat at 45°
Power Ratio	Key: base exposure · Fill: –1 to –1.5 stops
Gear Flex	2/5 — requires specific modifier; reflector substitute loses symmetry

Use Case	Beauty, cosmetics, clean portraiture
Why It Works	Eliminates under-eye shadows; maximizes catch-light size; flatters most faces

How to Use Recipes

1. **Browse Recipes**
   Navigate to Recipes in the nav bar. Filter by pattern, difficulty, or gear flexibility.

2. **Study the Setup**
   Read the pattern, modifiers, and "Why It Works" for lighting education.

3. **Run a Matching Analysis**
   Upload a reference photo inspired by the recipe; NGW confirms how closely it matches.

4. **Shoot Mode from Recipe**
   Launch Shoot Mode with a recipe as your target setup instead of an analyzed photo.

DOCUMENT 05

Build From Scratch Guide

Create a custom setup from zero — no reference photo required.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

When to Build From Scratch

Use Build From Scratch when you have no reference image but know your target. You specify the pattern, subject type, environment, and gear constraints directly and NGW generates a full setup recommendation as if it had analyzed a reference.

Common Use Cases

- Teaching a named pattern without a reference photo.
- Reproducing a setup you remember but don't have documented.
- Testing how NGW handles your kit against a specific pattern.
- Pre-planning a shoot based on a style brief, not a specific image.

Step-by-Step Build Process

1. **Choose a Pattern**
   Select from 30+ canonical patterns: loop, Rembrandt, clamshell, butterfly, split, hard, hi… gh-key, low-key, and more.

2. **Set Subject Type**
   Specify portrait, headshot, product, editorial, or architectural. Affects modifier and dis… tance suggestions.

3. **Set Environment**
   Studio with controlled ceiling height, outdoor ambient, or hybrid. Affects power and fill … recommendations.

4 **Confirm Your Kit**
NGW reads from My Kit automatically. Override specific items for this session only.

5 **Set Constraints**
Ceiling height, room width, power availability (battery vs. AC). All optional but improve … output.

6 **Generate Setup**
NGW produces a full recommendation: lights, positions, modifiers, power, and checklist.

7 **Launch Shoot Mode**
Convert the generated setup directly into on-set steps.

Pattern Reference

Pattern	Shadow Direction	Key Position	Ideal Subject
Loop	Short loop under nose	45° off-axis, high	Most portraits
Rembrandt	Triangle cheek shadow	45°+ off-axis	Dramatic / editorial
Clamshell	Near-none (symmetric)	On-axis, above	Beauty / cosmetics
Butterfly	Butterfly nose shadow	On-axis, very high	Glamour / beauty
Split	Half-face shadow	90° side	High drama

Broad	Away from camera	45°, broad side	Slimming portraits
Short	Toward camera	45°, short side	Full-face emphasis
Hard Direct	Full frontal harsh	On-axis, flat	Editorial / fashion

IMPORTANT

• Build From Scratch outputs are based on canonical pattern definitions, not image analysis. Confidence will read as "Blueprint" rather than a percentage. Always run a test shot to validate against your actual environment.

DOCUMENT 06

My Kit Guide

Manage your equipment so every recommendation fits what you own

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Why My Kit Matters

My Kit is the equipment registry that constrains every NGW recommendation. If a recommendation includes gear you don't own, the engine automatically substitutes from your kit through a 5-tier matching system. The more accurate your kit, the more useful every recommendation.

RULE OF THUMB

• Update My Kit before every shoot. If you're renting gear for a session, add it to your kit temporarily and remove it after. NGW always recommends from current kit state.

Kit Categories

Category	Examples	Notes
Flash	Godox AD600, Profoto B10, Speedlight	Include mount and power range
Continuous	LED panel, SL-60W, HMI	Include lumen output if known
Modifiers	Octa 90cm, beauty dish, strip box, umbrella	Include size and brand
Accessories	V-flat, reflector, gobo, grid, snoot	Mark if available at location
Camera	Body + lens focal length range	Affects angle-of-view guidance
Room	Ceiling height, room width, power access	Per-location; update per shoot

The 5 Gear Flexibility Tiers

When exact gear is unavailable, NGW cascades through 5 tiers, always choosing the highest-fidelity option available in your kit.

Tier 1 — Exact Match

The exact gear specified in the recommendation is in your kit. No substitution. Highest output confidence.

Tier 2 — Close Modifier Substitute

Compatible modifier with similar shaping characteristics (e.g., octa 120 instead of 90). Minor adjustments noted in the step guidance.

Tier 3 — Alternative Light Source

Different light type with equivalent output (e.g., continuous LED if no flash available). Power and exposure notes are adjusted.

Tier 4 — Improvised / DIY

Uses available surfaces as reflectors, flags, or diffusion (foam core, white wall, curtain). Marked as improvised in the output; confidence drops by ~20%.

Tier 5 — Ambient Only

No controllable lighting in kit or at location. NGW recommends ambient-light strategies: window placement, time of day, reflector positioning. Always a fallback, never a first choice.

GEAR FLEXIBILITY SCORE

• Each Recipe shows a Gear Flexibility score (1–5). Score 5 means "any gear works." Score 1 means "specific modifier required." Use this to pre-filter recipes that will work with your current kit.

DOCUMENT 07

Quick Start

On your feet in five steps.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

Get from zero to a complete on-set lighting plan in under 3 minutes.

1 Add Your Gear

Open My Kit add every light, modifier, and accessory you have today.

2 Confirm Your Space

In Settings enter ceiling height and room dimensions. Takes 30 seconds.

3 Upload a Reference

Tap Analyze upload a photo with the lighting you want to replicate.

4 Review the Result

Read the detected pattern, confidence score, and gear setup. Adjust as needed.

5 Execute in Shoot Mode

Tap "Start Shoot Mode" follow steps in order. Check off each as you go.

<3 min	30+	5 tiers	100%
From launch to setup plan	Lighting patterns in library	Gear flexibility matching	Deterministic output

PRO TIP

• Bookmark the Recipes section to study patterns before your shoot. Knowing your target pattern by name makes analysis results immediately actionable.

• Use the "Evaluate Test Shot" feature in Shoot Mode to get instant feedback on how closely your first frame matches the target.

DOCUMENT 08

On-Set Quick Reference

Printable field card — patterns, positions, and checks.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

Pattern Quick Reference

Pattern	Key Height	Key Angle	Fill	Best For
Loop	Above eye level	30–45°	–1.5 stops	All-purpose portraits
Rembrandt	High, angled	45°+	Minimal	Drama, editorial
Clamshell	On-axis above	0°	Below-axis	Beauty, fashion
Butterfly	Very high	0°	Reflector	Glamour, beauty
Split	Eye level	90°	None/minimal	High contrast
Broad	Above eye	45° broad	–2 stops	Slimming
Short	Above eye	45° short	–1 stop	Full-face width
Hard Direct	On-axis, flat	0°	White card	Fashion, editorial

Pre-Shoot Checklist

Kit & Space

- My Kit updated for today's gear
- Ceiling height in Settings
- Room dimensions in Settings
- Battery / power checked

Lights

- All modifiers mounted
- All stands at rough height
- Power at starting values
- Triggers paired

Camera

- Reference aperture set
- ISO at base (100–400)
- Shutter at sync speed
- Tethering connected (if used)

First Test Shot

- Shadow direction — correct?
- Catch-light placement — correct?
- Fill ratio — on target?
- Background separation — correct?

Power Ratio Reference

Ratio	Fill f-stop vs Key	Effect	Pattern Examples
1:1	0 stops	Flat, even fill	High-key, product
1:2	–1 stop	Subtle fill, soft shadow	Corporate headshot
1:4	–2 stops	Defined shadow, natural look	Portrait, loop
1:8	–3 stops	Strong shadow, dramatic fill	Rembrandt, low-key
No fill	–4 stops+	Hard shadow, minimum fill	Split, editorial

DOCUMENT 09

System Architecture

How NGW works — pipeline, schema, auth, and data flow.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Architecture Overview

NGW is a mobile-first web application backed by a Python/FastAPI service. The frontend is a React 18 SPA built with Tailwind CSS. The backend orchestrates a five-stage analysis pipeline that combines image processing, VLM inference, and a weighted consensus solver. All model calls flow through Vercel AI Gateway for cost tracking, failover, and OIDC-based authentication.

Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18	Mobile-first SPA
Frontend	Tailwind CSS	3	Utility styling
Frontend	Vite	5	Build + HMR
Backend	Python	3.11+	Runtime
Backend	FastAPI	0.110+	REST API framework
Backend	Uvicorn	0.28+	ASGI server
Database	SQLite	3.x	Dev / embedded prod
Database	PostgreSQL	15+	Prod (optional)
AI	Vercel AI Gateway	current	VLM routing + auth
AI	Vision LLM	routed	Image interpretation
Deploy	Vercel	current	Frontend + serverless API

Auth	OIDC / Bearer Token	JWT	API + Lab access control

Database Schema

Five primary tables. Schema lives in db/database.py and is created on first run via init_db(). Migrations are run with python3 scripts/run_migrations.py.

db/database.py — session_signals (core outcomes table)

```
CREATE TABLE session_signals (
  id            TEXT PRIMARY KEY,   -- UUID v4
  session_id    TEXT NOT NULL,
  user_id       TEXT,
  pattern       TEXT NOT NULL,      -- "loop", "rembrandt", "clamshell" …
  outcome       TEXT NOT NULL,
    -- CHECK IN (nailed_it, close, failed, unknown)
  signal_source TEXT NOT NULL DEFAULT 'live',
    -- CHECK IN (live, seeded, internal, expert_review)
  include_in_learning    BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_metrics     BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_conversion  BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_cohorts     BOOLEAN NOT NULL DEFAULT TRUE,
  reliability_score      REAL,
  region_scores          TEXT,   -- JSON {face,torso,bg,specular,shadow}
  degradation_flags      TEXT,   -- JSON {bw,low_res,high_contrast,...}
  gear_tier              INTEGER CHECK (gear_tier BETWEEN 1 AND 5),
  environment            TEXT,   -- studio | outdoor | hybrid
  created_at             DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

db/database.py — gold_sets and candidates tables

```
CREATE TABLE gold_sets (
  id              TEXT PRIMARY KEY,
  image_path      TEXT NOT NULL,
  pattern         TEXT NOT NULL,    -- curator-verified ground truth
  setup_details   TEXT,             -- JSON: gear, positions, power
  subject_type    TEXT,
  environment     TEXT,
  outcome_labels  TEXT,             -- JSON: expected engine output
  status          TEXT DEFAULT 'draft', -- draft|approved|archived
  curator_email   TEXT NOT NULL,
  created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE candidates (
  id                        TEXT PRIMARY KEY,
  title                     TEXT NOT NULL,
  description               TEXT NOT NULL,
  candidate_type            TEXT NOT NULL,
  proposed_change           TEXT,  -- JSON
  status TEXT DEFAULT 'proposed',
    -- proposed|under_review|approved|rejected|applied
  estimated_success_lift    REAL,
  estimated_regression_risk REAL,
  source                    TEXT DEFAULT 'auto', -- auto|manual
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Analysis Pipeline — 5 Stages

Every reference image runs through five sequential stages. Each stage is logged independently with timing and confidence. A stage may short-circuit at the early-exit threshold (confidence ≥ 0.94).

Stage	Module	Input	Output
1. Ingestion	api/routes/shoot_match.py	Raw image file	Normalized 1024px JPEG
2. Region Extraction	engine/region_extractor.py	Normalized image	Bounding boxes + region scores
3. VLM Interpretation	engine/vlm_adapter.py	Annotated regions (3 passes)	Pattern vocabulary dicts
4. Consensus Solving	engine/solver.py	Three VLM output dicts	Resolved pattern + confidence
5. Blueprint Generation	engine/blueprint_engine.py	Pattern + user kit	Gear setup + ordered steps

VLM Integration

The VLM adapter runs three passes per image with different prompt templates: pattern-focused, modifier-focused, and geometry-focused. This reduces single-pass bias. All three response dicts go to the consensus solver.

engine/vlm_adapter.py — call structure (simplified)

```
# All model calls route through Vercel AI Gateway automatically.
# Set model via AI_MODEL_STRING env var; default: gateway-routed vision LLM.
PASS_PROMPTS = [
    PATTERN_SYSTEM_PROMPT,   # "What lighting pattern is this?"
    MODIFIER_SYSTEM_PROMPT,  # "What modifiers and shaping tools are visible?"
    GEOMETRY_SYSTEM_PROMPT,  # "Describe light positions and distances."
]

async def analyze(image_bytes, kit, model=AI_MODEL_STRING):
    results = []
    for prompt in PASS_PROMPTS:
        r = await gateway.analyze_image(
            image=image_bytes, system=prompt,
            model=model, max_tokens=1024, temperature=0.2
        )
        results.append(parse_response(r))
    return results  # List[Dict] — one per pass
```

engine/solver.py — weighted consensus (simplified)

```
# Pass weights: pass 0 = 1.0, pass 1 = 1.05, pass 2 = 1.10
# Later passes get slight boost (more context in prompt chain).
CONSENSUS_ATTRS = [
    "pattern_name", "key_angle", "key_height",
    "fill_type",    "modifier",  "fill_ratio",
]

def resolve(pass_results: List[Dict]) -> ResolvedPattern:
    out = {}
    for attr in CONSENSUS_ATTRS:
        votes = {v[attr]: 0.0 for v in pass_results if attr in v}
        for i, v in enumerate(pass_results):
            if attr in v: votes[v[attr]] += PASS_WEIGHTS[i]
        winner = max(votes, key=votes.get)
        confidence = votes[winner] / sum(votes.values())
        out[attr] = (winner, round(confidence, 3))
    return ResolvedPattern(**out)
```

Blueprint Generation & Kit Matching

The Blueprint Engine maps a resolved pattern to the user's kit through a 5-tier cascade. It stops at the highest-fidelity tier that yields a viable setup, then constructs the ordered Shoot Mode step list.

engine/blueprint_engine.py — tier cascade

```
# Tiers: 1=exact  2=close_modifier  3=alt_source  4=diy  5=ambient
for tier in range(1, 6):
    gear = kit_matcher.match(blueprint, user_kit, tier=tier)
    if gear.is_viable():
        steps = step_builder.build(
            gear=gear, role=role,
            ceiling_m=ceiling_m, room=room_dims
        )
        return Blueprint(
            pattern=resolved.pattern_name,
            confidence=resolved.confidence,
            gear=gear,   steps=steps,
            gear_tier=tier,
        )
return Blueprint.ambient_fallback(user_kit)
```

API Authentication

Method	Header / Env Var	Source	TTL
OIDC Token	Authorization: Bearer $VERCEL_OIDC_TOKEN	vercel env pull	~24 h
API Key	Authorization: Bearer $AI_GATEWAY_API_KEY	Vercel Dashboard	Manual
Lab Email Gate	x-lab-email: you@org.com	LAB_ALLOWED_EMAILS	Per request
Public Routes	(none)	N/A	N/A

OIDC VS API KEY

• On Vercel: prefer OIDC — tokens auto-refresh, no secret rotation. For local dev run: vercel link && vercel env pull .env.local

• Never commit VERCEL_OIDC_TOKEN or AI_GATEWAY_API_KEY to source control.

Full API Surface Reference

Endpoint	Method	Auth	Description
/health	GET	Public	Service liveness check
/health/db	GET	Public	Database schema + table count
/api/shoot-match	POST	Bearer	Core analysis: image recommendation
/api/upload-reference	POST	Bearer	Upload reference image (10MB max)
/api/master-modes	GET	Bearer	List photographer reference modes
/api/shoot-mode/start	POST	Bearer	Start Shoot Mode from recommendation
/api/shoot-mode/evaluate-test-shot	POST	Bearer	Compare test shot to target pattern
/api/user-data/kit	GET/POST	Bearer	Load / save user equipment kit

/api/user-data/setup	GET/ POST	Beare r	Load / save named user setups
/api/reference-library	CRU D	Beare r	Reference image library management
/api/reference-library/ingest	POST	Beare r	Ingest reference with metadata
/api/lab/gold-set	CRU D	Lab	Gold-set test image management
/api/lab/gold-set/evaluate	POST	Lab	Run engine on approved gold sets
/api/lab/candidates	CRU D	Lab	Engine improvement candidates
/api/lab/signals	POST	Lab	Ingest signals with source tags
/api/lab/signals/summary	GET	Lab	Signal hygiene counts by source
/api/lab/analyze	POST	Lab	Full-fidelity analysis + debug overlay

DOCUMENT 10

NGW Lab Guide

Developer tools — API reference, CLI, gold sets, candidates, and wo

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

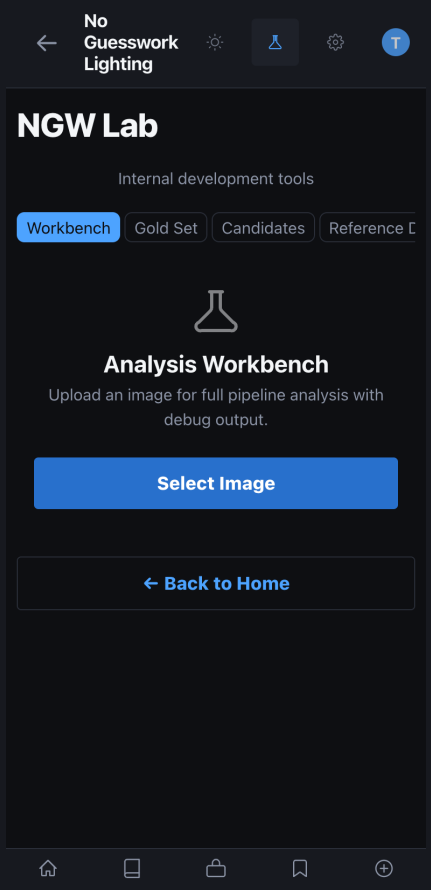
NGW

What is the NGW Lab?

The Lab is a restricted developer environment for curating ground-truth test images, reviewing engine improvement candidates, running full-fidelity analyses with debug overlays, and managing the learning signal pipeline. Access is controlled by an email allowlist (LAB_ALLOWED_EMAILS env var).

ACCESS CONTROL

• All Lab endpoints check the x-lab-email request header against LAB_ALLOWED_EMAILS. Unauthorized access returns 403 and is logged. Enable Lab UI in Settings Developer Tools.



NGW Lab Workbench — empty state showing tabs: Workbench, Gold Sets, Candidates, Signals

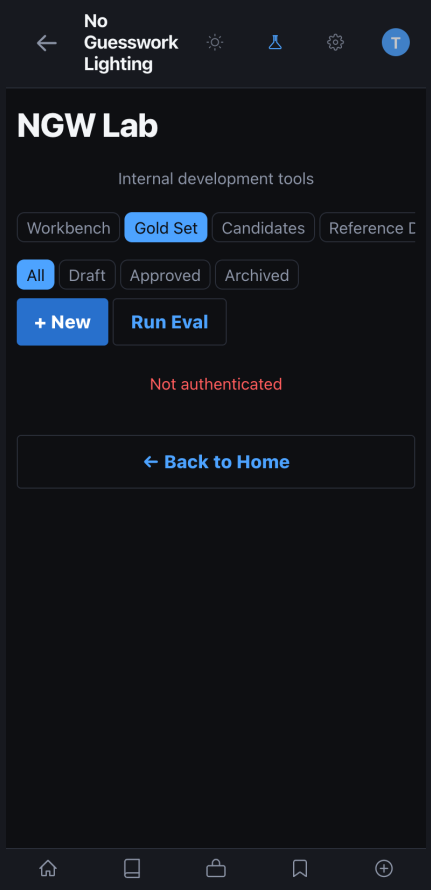
Lab API Reference — Gold Sets

POST /api/lab/gold-set — create a new gold set

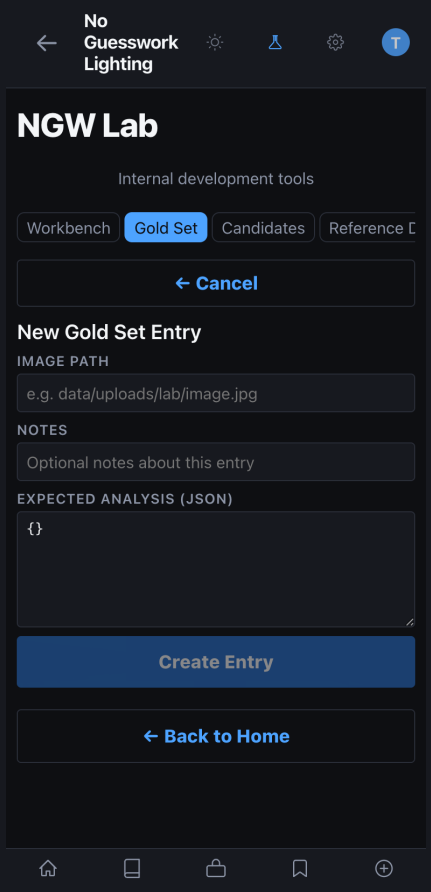
```
# Request body
{
  "image_path":    "static/uploads/portrait_001.jpg",
  "pattern":       "loop",      # curator-verified ground truth
  "subject_type":  "portrait",
  "environment":   "studio",
  "setup_details": {
    "key_light":  "Godox AD600 + octa 90cm",
    "key_angle":  45,
    "key_height": 7.5,
    "key_dist_ft": 6,
    "fill":       "V-flat reflector camera-left"
  },
  "outcome_labels": {
    "expected_pattern":    "loop",
    "expected_confidence": 0.85
  },
  "curator_email": "you@org.com"
}
# Response: {"id":"gs_uuid","status":"draft"}
```

POST /api/lab/gold-set/evaluate — run engine on approved sets

```
# Request body (optional: limit to specific IDs)
{
  "gold_set_ids": ["gs_001","gs_002"],  # omit for all approved
  "include_debug_overlay": true
}

# Response
{
  "evaluated": 12,
  "passed": 10,
  "failed": 2,
  "scores": {
    "gs_001": {"detected":"loop","expected":"loop","match":true,"conf":0.88},
    "gs_002": {"detected":"broad","expected":"loop","match":false,"conf":0.72}
  }
}
```



Gold Set listing — all test images with pattern labels, status, and evaluation scores



New Gold Set form — image upload, pattern selection, and setup metadata entry

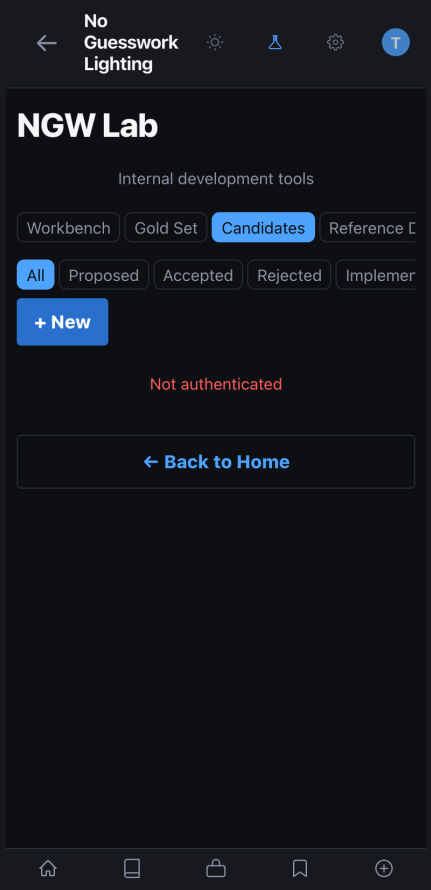
Lab API Reference — Candidates

POST /api/lab/candidates — create a candidate manually

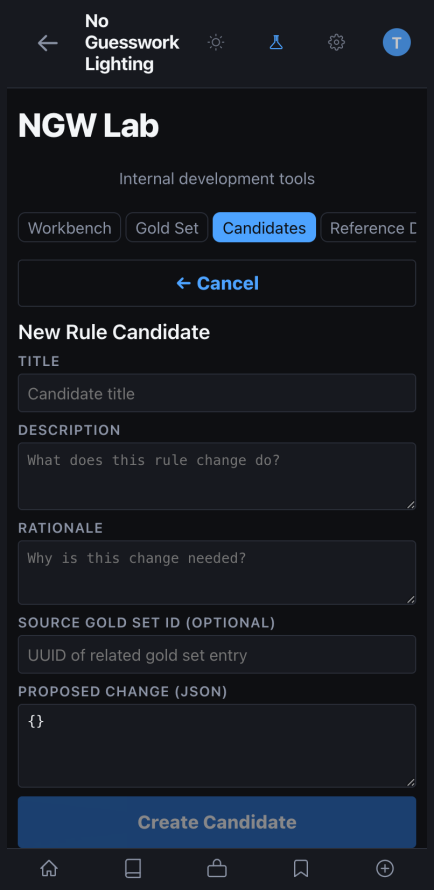
```
{
  "title":       "Recalibrate loop confidence threshold",
  "description": "Loop pattern confidence is 20% above measured CVR.
    Threshold should be lowered to reduce over-detection.",
  "candidate_type": "confidence_recalibration",
  "proposed_change": {
    "type":    "confidence_recalibration",
    "pattern": "loop",
    "action":  "lower_threshold",
    "delta":   -0.08
  },
  "estimated_success_lift":    0.12,
  "estimated_regression_risk": 0.05,
  "source": "manual"
}
# Response: {"id":"cand_uuid","status":"proposed"}
```

PUT /api/lab/candidates/{id} — update status (approve/reject)

```
{
  "status": "approved",
  "reviewer_email": "curator@org.com",
  "review_notes": "Risk acceptable. CVR data supports the change.",
  "second_reviewer_email": null
  # Required if estimated_regression_risk > 0.20
}
```



Candidates listing — proposed and reviewed engine changes with risk and lift estimates



New Candidate form — type, description, proposed change JSON, and risk estimation

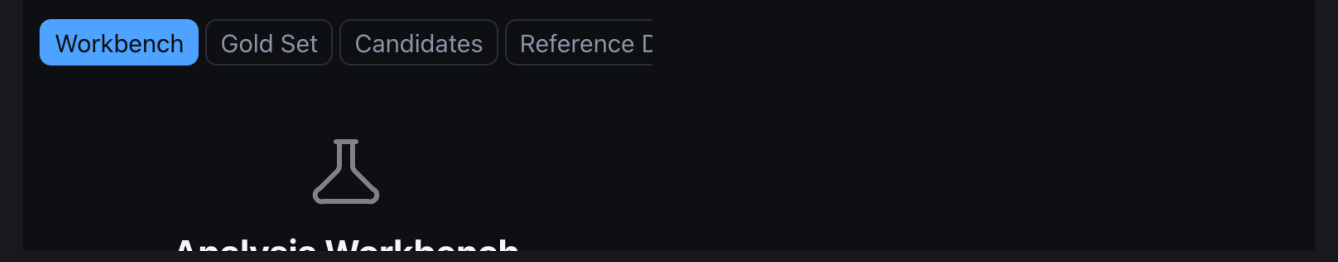
Lab API Reference — Analysis & Signals

POST /api/lab/analyze — full-fidelity debug analysis

```
# Request body
{
  "image_path":          "static/uploads/ref_001.jpg",
  "include_debug_overlay": true,
  "return_pass_details":   true,
  "kit":                   null  # null = use default test kit
}

# Response includes:
# - all three VLM pass results (raw)
# - consensus resolver scores per attribute
# - region extraction bounding boxes
# - blueprint output + gear tier
# - debug_overlay_url (annotated image)
```

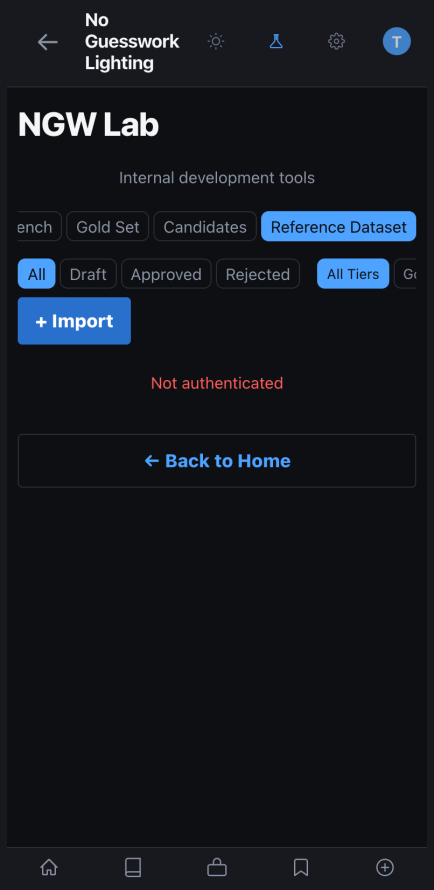
GET /api/lab/signals/summary — hygiene counts

```
# Response
{
  "live":              1482,
  "seeded":             500,
  "internal":            23,
  "expert_review":        8,
  "total":             2013,
  "learning_eligible": 1482,
  "metrics_eligible":  1482,
  "flag_mismatches":      0
}
```

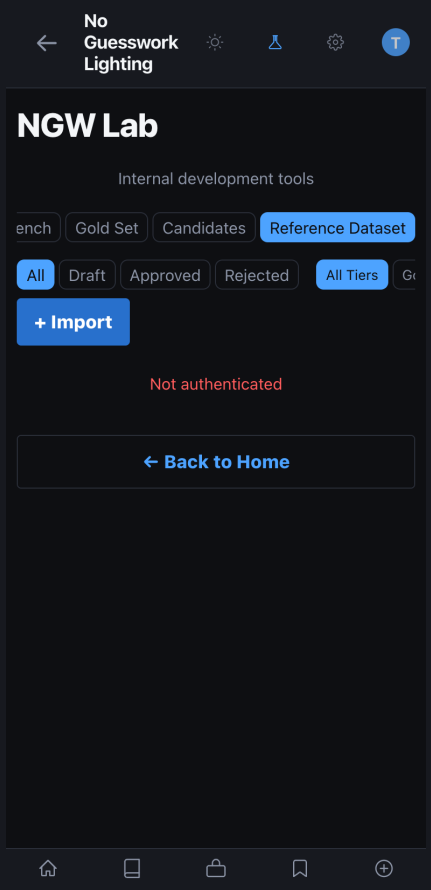


Lab — Signal Hygiene card showing live / seeded / internal / learning-eligible counts

Reference Dataset



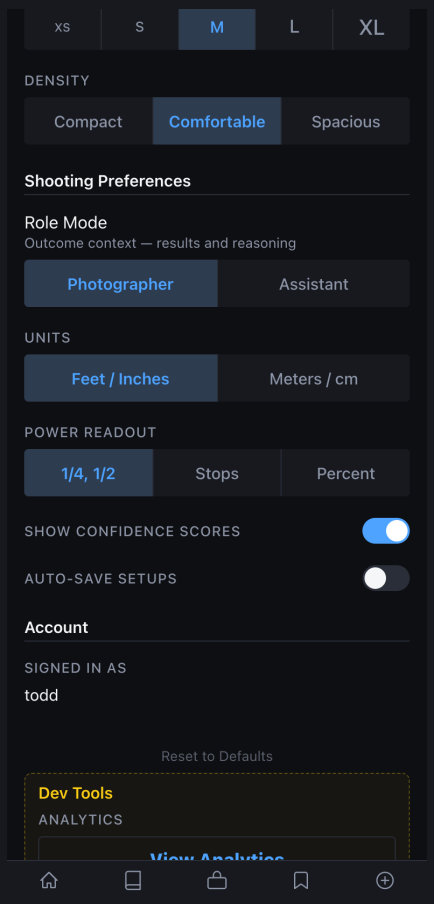
Reference Dataset — full grid view with filter bar



Reference Dataset detail — image with pattern label and approval status

POST /api/reference-library/ingest — ingest with metadata

```
{
  "image_path":   "static/uploads/ref_new.jpg",
  "pattern":      "rembrandt",
  "modifiers":    ["beauty dish","grid"],
  "subject_type": "portrait",
  "environment":  "studio",
  "quality_score": 0.92,
  "submitted_by": "curator@org.com"
}
# Starts as status="pending_review".
# Second curator approves → status="approved" → enters training set.
```



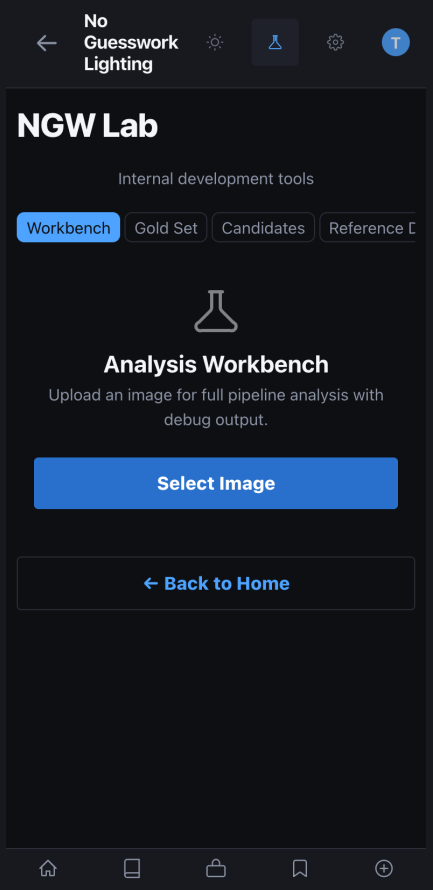
Lab Settings — evaluation thresholds, access list, and confidence gates

CLI Tools

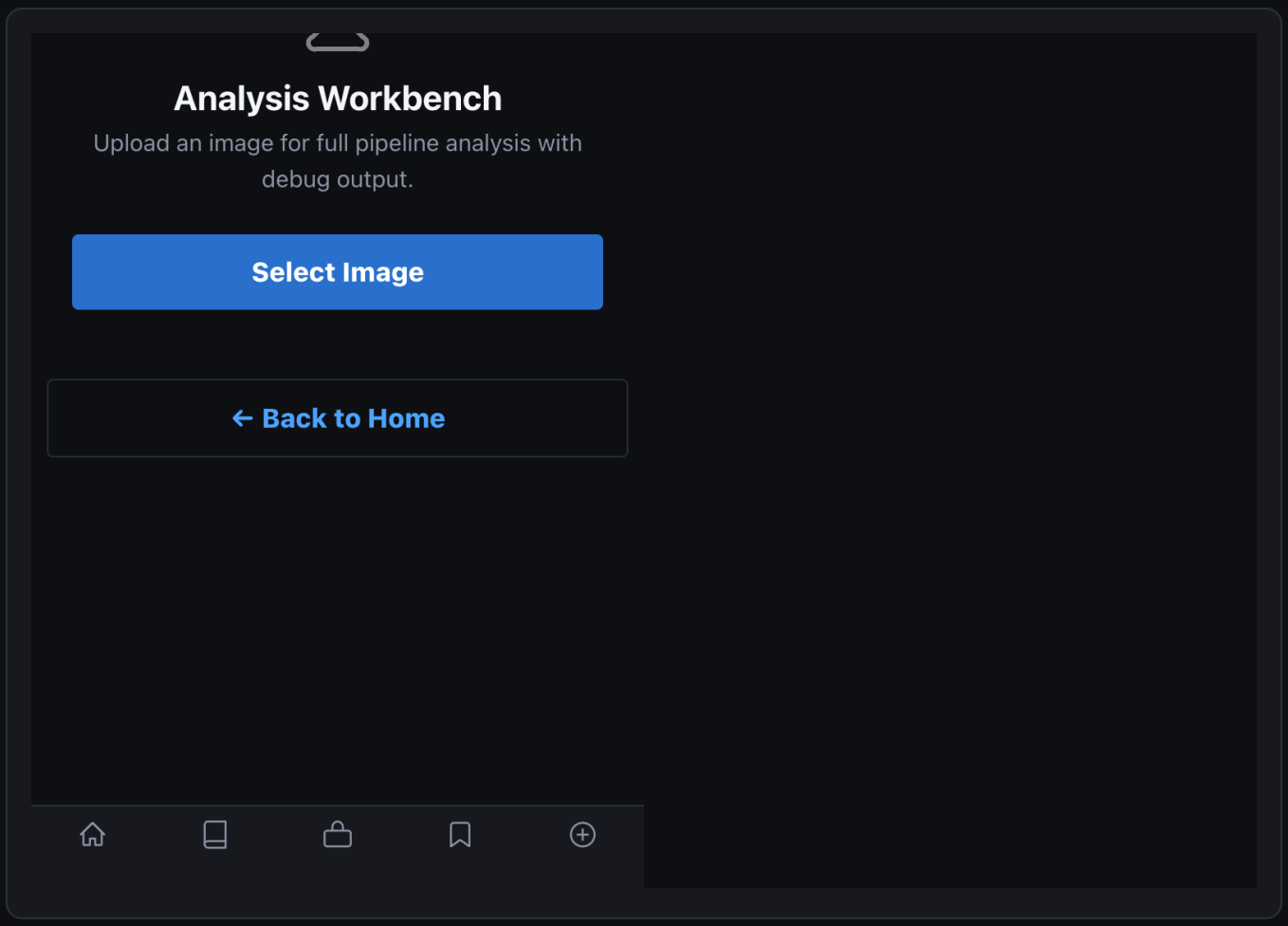
Script	Purpose	Key Flags
seed_starter_dataset.py	Load seeded signals + gold sets	--reset to clear first
verify_dataset.py	Check dataset integrity and counts	--verbose for row details
run_benchmarks.py	Pattern matching accuracy benchmarks	--gold-sets-only, --pattern=X
nightly_benchmark.py	Full benchmark suite (CI use)	--output=json
validate_system_yamls.py	Lint pattern YAML configurations	--strict
validate_catalog_and_packs.py	Validate gear catalog completeness	—
ci_benchmark.sh	CI/CD benchmark wrapper with exit codes	EXIT 1 on regression
capture_screenshots.py	Capture app screenshots for docs	--output-dir=docs/screenshots
generate_ngw_docs.py	Build this PDF documentation suite	—

bash — common Lab CLI operations

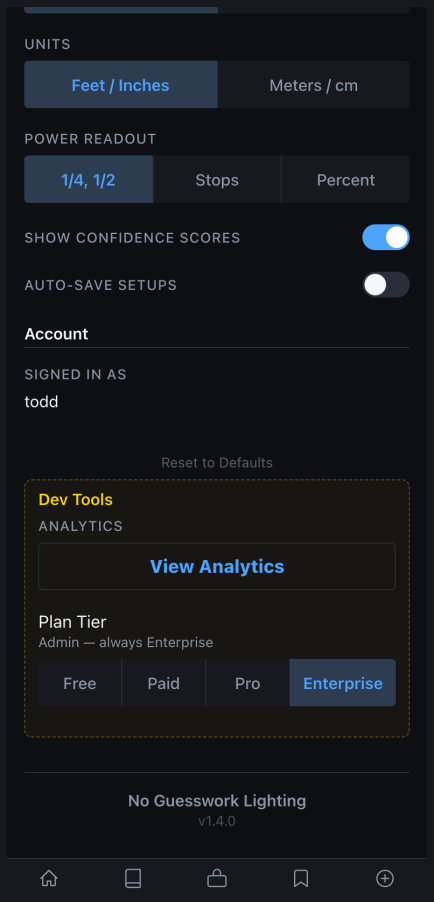
```
# Seed starter data (run once after init_db)
$ python3 scripts/seed_starter_dataset.py

# Verify dataset integrity
$ python3 scripts/verify_dataset.py --verbose

# Run all benchmarks
$ python3 scripts/run_benchmarks.py

# Run gold-set-only evaluation
$ python3 scripts/run_benchmarks.py --gold-sets-only

# Validate YAML configs (run before any blueprint changes)
$ python3 scripts/validate_system_yamls.py --strict
```



Workbench ready state — analysis loaded with debug overlay and region annotations



Workbench detail — per-region confidence scores and VLM pass breakdown



Dev Tools panel — database status, migration runner, and signal count monitor

How To: Add a Gold Set Image

Gold Set images are the ground-truth test set that benchmarks run against. Adding well-labeled images directly improves evaluation coverage.

1. **Open NGW Lab**
   Tap the Lab tab in the navigation bar (requires Lab access enabled).

2. **Go to Gold Sets**
   Tap "Gold Sets" in the Lab navigation tabs.

3. **Tap + Add New**
   Opens the gold set creation form.

4. **Upload the image**
   Choose a clean, unedited reference photo with clear lighting.

5. **Select pattern**
   Choose the ground-truth pattern from the dropdown (e.g. "loop").

6. **Set metadata**
   Fill in subject type, environment, modifier list, and quality notes.

7 Submit for review

Status starts as "pending_review". A second curator approves it.

8 After approval

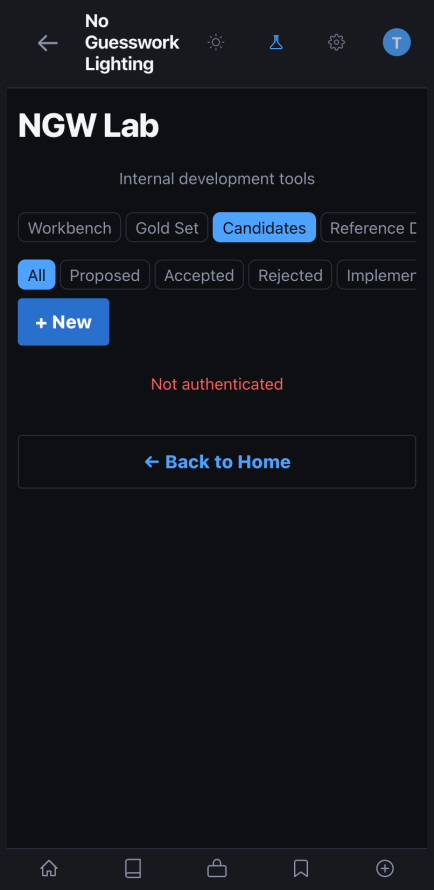
Status changes to "approved" — the image enters the benchmark set.

QUALITY STANDARDS

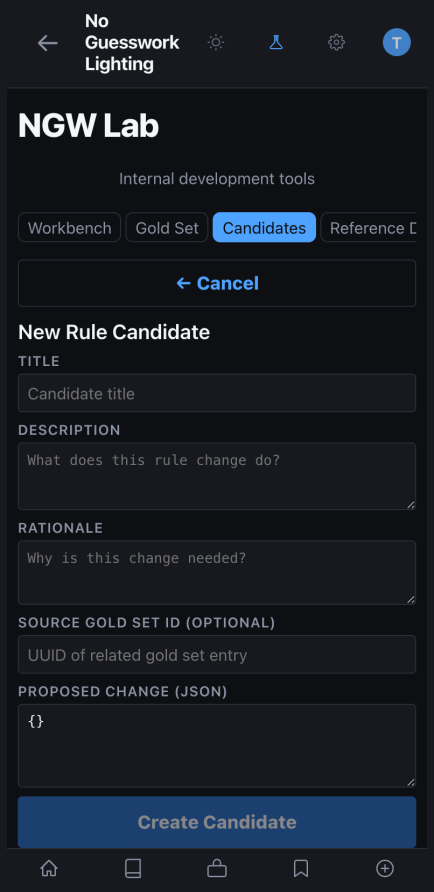
- Use only clean originals — no heavy color grading, compositing, or retouching.
- Minimum 1024 px wide. Portrait orientation preferred for face-priority patterns.
- Label with the ground-truth pattern, not what you think NGW will detect.
- Add notes for any unusual conditions (high ceiling, very large modifier, etc.).

How To: Review and Approve a Candidate

Candidates are improvement proposals — manually created or auto-generated. Review each one carefully before approving. High-regression-risk candidates require a second curator.



Candidate queue — proposed, approved, and rejected status columns



New candidate form — type selection, description, and risk assessment fields

1 Open Candidates tab

Tap "Candidates" in the Lab navigation. Filter by status="proposed".

2 Read the description

Understand what change is proposed and why the failure cluster triggered it.

3 Check regression risk

If regression_risk > 0.20, a second curator approval is required.

4 **Review proposed_change**
Read the full JSON. Understand which blueprint or config key changes.

5 **Approve or Reject**
Tap Approve to advance to "approved", or Reject to archive with a note.

6 **Validate after apply**
Gold set benchmarks re-run automatically. Check scores in CLI output.

NEVER SKIP VALIDATION

• Every approved candidate triggers an automatic gold set re-evaluation. If any gold set drops below 75%, the deployment pipeline fails and the candidate reverts to "proposed". This gate cannot be bypassed in production.

DOCUMENT 11

Signal System Guide

Schema, ingestion API, hygiene flags, queries, and reliability weights.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

What is the Signal System?

Every shoot outcome a user records — "nailed it", "close", "failed", or "unknown" — is a signal. The Signal System classifies, tags, and filters these outcomes so that only the right signals reach each downstream consumer. The four inclusion flags are the key mechanism: they let you separate seeded bootstrap data from live user data without deleting any rows.

Database Schema

session_signals — full CREATE TABLE

```
CREATE TABLE session_signals (
  id            TEXT PRIMARY KEY,     -- UUID v4, client-generated
  session_id    TEXT NOT NULL,         -- links to the shoot session
  user_id       TEXT,                  -- nullable for anonymous
  pattern       TEXT NOT NULL,
    -- detected or target pattern name
  outcome       TEXT NOT NULL
    CHECK (outcome IN
      ('nailed_it', 'close', 'failed', 'unknown')),
  signal_source TEXT NOT NULL DEFAULT 'live'
    CHECK (signal_source IN
      ('live','seeded','internal','expert_review')),
  include_in_learning    BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_metrics     BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_conversion  BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_cohorts     BOOLEAN NOT NULL DEFAULT TRUE,
  reliability_score      REAL CHECK (reliability_score BETWEEN 0 AND 1),
  region_scores TEXT,  -- JSON: {"face":0.91,"torso":0.78,...}
  degradation_flags TEXT, -- JSON: {"bw":false,"low_res":false,...}
  gear_tier   INTEGER CHECK (gear_tier BETWEEN 1 AND 5),
  environment TEXT,    -- studio | outdoor | hybrid
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Signal Sources — Default Flag Values

source	in_learning	in_metrics	in_conversion	in_cohorts	Who Creates It
live	TRUE	TRUE	TRUE	TRUE	Real end users via app
seeded	FALSE	FALSE	FALSE	FALSE	Bootstrap / seed scripts
internal	FALSE	FALSE	FALSE	FALSE	Developer / QA testing
expert_review	FALSE	FALSE	FALSE	FALSE	Curator-verified outcomes

OVERRIDE

• The default flags can be overridden per-row at insert time or updated in bulk later. For example, expert_review signals can have include_in_learning = TRUE set manually to use them as high-quality learning data while still excluding them from public metrics.

Signal Ingestion API

POST /api/lab/signals accepts a batch of signal objects. Each object must include at minimum pattern, outcome, and signal_source. All other fields are optional.

POST /api/lab/signals — request body (JSON)

```
{
  "signals": [
    {
      "id":            "sig_uuid_here",
      "session_id":    "sess_abc123",
      "user_id":       "usr_xyz",
      "pattern":       "loop",
      "outcome":       "nailed_it",
      "signal_source": "live",
      "gear_tier":     1,
      "environment":   "studio"
    }
  ]
}
```

POST /api/lab/signals — response (JSON)

```
{
  "inserted": 1,
  "skipped":  0,
  "errors":   []
}
```

bash — ingest signals via curl

```
$ curl -X POST http://localhost:8000/api/lab/signals \
  -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
  -H "x-lab-email: you@org.com" \
  -H "Content-Type: application/json" \
  -d @signals_batch.json
```

Inclusion Flags — How Each Is Used

Flag	Consumer	Query Filter Applied

include_in_learning	Learning system (auto_candidate)	WHERE include_in_learning = TRUE
include_in_metrics	Dashboard analytics queries	WHERE include_in_metrics = TRUE
include_in_conversion	CVR calculations	WHERE include_in_conversion = TRUE
include_in_cohorts	User segmentation queries	WHERE include_in_cohorts = TRUE

SQL Query Reference

sql — learning-eligible signals by pattern (last 30 days)

```sql
SELECT
  pattern,
  outcome,
  COUNT(*)                          AS n,
  ROUND(AVG(reliability_score), 3)  AS avg_reliability,
  ROUND(AVG(gear_tier), 2)          AS avg_gear_tier
FROM session_signals
WHERE include_in_learning = TRUE
  AND signal_source       = 'live'
  AND created_at          >= DATE('now', '-30 days')
GROUP BY pattern, outcome
ORDER BY pattern, n DESC;
```

sql — metrics dashboard query (CVR per pattern)

```sql
SELECT
  pattern,
  COUNT(*) FILTER (WHERE outcome = 'nailed_it') AS nailed,
  COUNT(*) FILTER (WHERE outcome = 'close')     AS close,
  COUNT(*) FILTER (WHERE outcome = 'failed')    AS failed,
  COUNT(*)                                       AS total,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE outcome IN ('nailed_it','close'))
    / NULLIF(COUNT(*), 0), 1
  ) AS cvr_pct
FROM session_signals
WHERE include_in_metrics = TRUE
GROUP BY pattern
ORDER BY cvr_pct DESC;
```

sql — hygiene audit: check for mis-tagged seeded rows

```sql
-- Find seeded rows that were incorrectly flagged as live
SELECT id, session_id, created_at
FROM session_signals
WHERE signal_source = 'seeded'
  AND (include_in_metrics = TRUE
    OR include_in_learning = TRUE);

-- Fix: reset all seeded flags to FALSE
UPDATE session_signals
SET include_in_learning   = FALSE,
    include_in_metrics    = FALSE,
    include_in_conversion = FALSE,
    include_in_cohorts    = FALSE
WHERE signal_source = 'seeded';
```

Signal Reliability Weights

Each signal is scored across eight image regions. The raw score is multiplied by degradation factors that reduce weight when analysis conditions are poor.

Region	Weight	What It Detects
face	0.35	Primary lighting indicator — catch-lights, shadows, fill ratio
torso	0.15	Secondary — useful for full-length and 3/4 portraits
background	0.12	Fall-off curve, separation from subject
specular_surfaces	0.14	Modifier signature — size and hardness of specular highlights
shadow_regions	0.12	Shadow direction, density, and edge hardness
highlight_regions	0.12	Specular placement confirms key position

Degradation Factor	Multiplier	Trigger Condition
Black & white / monochrome	×0.75	Chroma saturation < 0.05
Heavy color grade	×0.80	Hue shift > 30° or LUT detected
No face detected	×0.70	Face region score == 0
Low resolution	×0.70	Image width < 512 px
Extreme contrast	×0.85	Histogram clipping > 5%
Environmental clutter	×0.90	Background region score < 0.3
Multiple shadow directions	×0.80	Solver finds > 1 shadow vector

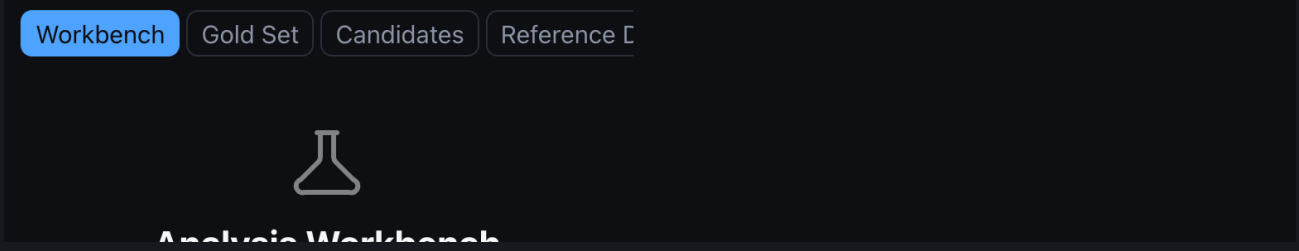
Signal Hygiene Summary API

GET /api/lab/signals/summary — response


```
{
  "live":              1482,
  "seeded":            500,
  "internal":          23,
  "expert_review":     8,
  "total":             2013,
  "learning_eligible": 1482,
  "metrics_eligible":  1482,
  "conversion_eligible": 1482,
  "cohorts_eligible":  1482,
  "flag_mismatches":   0
}
```


FLAG_MISMATCHES

• A non-zero flag_mismatches count means rows have signal_source="seeded" or "internal" but one or more include_* flags is TRUE. Run the hygiene audit SQL query above to identify and fix affected rows immediately.



Lab Workbench — Signal Hygiene summary card (live / seeded / internal / learning eligible)

DOCUMENT 12

Learning System Guide

Pattern knowledge base, signal quality weighting, 3-gate CI evaluatio

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Overview

The NGW learning system is a closed-loop pipeline that turns production user outcomes into validated engine improvements. It combines a curated pattern knowledge base, quality-weighted signal aggregation, a 3-gate CI evaluator, and revenue projection to ensure every proposed change is safe, signal-sufficient, and worth shipping.

Nothing is auto-implemented without passing all three gates. Low-risk changes can auto-deploy once gates clear; medium risk requires human review; high risk always requires a human gate regardless of gate scores.

27	3-Gate CI	3 Risk tiers	+$2,063
Patterns in knowledge base	Signal · Benchmark · Pattern	low / medium / high	Annual uplift, S3 vs S1

Architecture — Six Modules

Module	Location	Role
ingestion	engine/learning/ingestion.py	Analytics failure clusters (6 failure modes)
auto_candidate	engine/learning/auto_candidate.py	Failure cluster typed candidate proposals
sandbox_eval	engine/learning/sandbox_eval.py	Gold Set evaluation — safe / risky / blocked
monitoring	engine/learning/monitoring.py	Post-release metric tracking at 7/14/30d windows

knowledge	engine/learning/knowledge.py	Pattern KB · signal weighting · MIN_SIGNALS gate
ci_gate	engine/learning/ci_gate.py	3-gate CI evaluator with risk-tier disposition
revenue	engine/learning/revenue.py	CVR delta revenue projection · 30-day simulation

Pattern Knowledge Base

engine/learning/knowledge.py holds a PATTERN_KNOWLEDGE_BASE dict with 27 entries. Each entry records the pattern's risk level, minimum signals for change, and known failure symptoms with fix steps. The risk level drives gate thresholds.

Risk Level	Min Weighted Signals	Patterns (examples)	Auto-Deploy?
low	25	broad, short, high_key, low_key	Yes, if gates pass
medium	75	split, butterfly, three_point, rim	Human review required
high	200	loop, rembrandt, clamshell, unknown	Human gate always

engine/learning/knowledge.py — PatternEntry

```
@dataclass
class PatternEntry:
    pattern_id:            str
    display_name:          str
    family:                str   # portrait|editorial|commercial|tabletop
    risk_level:            str   # low | medium | high
    min_signals_for_change: int   # MIN_SIGNALS[risk_level]
    symptoms:              List[SymptomEntry]
    tags:                  List[str]

# Access helpers
entry  = get_pattern_entry("clamshell")   # → PatternEntry
high   = list_high_risk_patterns()         # → ["loop","rembrandt","clamshell",....
thresh = get_min_signals("rembrandt")      # → 200
```

Pattern Comparison System

When analysis confidence falls below 0.72 and alternative patterns are present, the UI switches into comparison mode. The comparison system surfaces the two most likely patterns side-by-side so the photographer can confirm or reject the engine's top pick — adding a high-quality ambiguity signal back to the learning loop.

Source: ui/src/knowledge/graph.js builds a bidirectional confusionWith graph. ui/src/knowledge/index.js getPatternComparisons() returns structured comparison pairs including key distinguishing cues (e.g., nose shadow side, catch-light position, shadow falloff) for the detected and candidate patterns.

Component	Location	Role

buildGraph()	knowledge/graph.js	Computes patternToPatterns + patternToSymptoms maps once
getPatternComparisons()	knowledge/index.js	Returns ComparisonPair[] for a pattern and its confusionWith list
ResultPatternComparePrompt	screens/ResultsScreen.jsx	Rendered when showComparison === true (confidence < 0.72)
confusionWith	knowledge/patterns.js each entry	Array of pattern IDs most often confused with this one

ui/src/knowledge/index.js — comparison trigger logic

```
// In buildResultsData():
const showComparison = confidence < 0.72 && alternatives.length > 0;
const comparisons    = showComparison
  ? getPatternComparisons(patternId)
  : [];

// Each ComparisonPair includes:
// { patternA, patternB, distinguishers: [
//   { cue, aValue, bValue, weight },   // e.g. nose shadow position
//   ...] }
```

WHY COMPARISON SIGNALS ARE HIGH-VALUE

• A photographer who manually selects "Loop — not Rembrandt" after seeing ambiguous results

• provides a human-verified ambiguity correction. These signals carry source="live" + outcome

• "failed" weight in the knowledge system. Accumulating 15–20 such corrections reliably shifts

• the classifier thresholds for the confused pattern pair.

Signal Quality Weighting

Raw session signals are weighted before counting toward a gate threshold. A pro photographer's expert-reviewed correction carries 5× the weight of a beginner's live session. This prevents a flood of low-quality signals from triggering unsafe changes.

weight = tier × source × outcome × confidence_boost (if confidence ≥ 0.75)

Factor	Value	Multiplier
Tier	pro	2.0×
	advanced	1.5×
	intermediate	1.0×
	beginner	0.5×
	unknown	0.7×
Source	expert_review	2.5×
	internal	1.2×
	live	1.0×
	seeded	0.4×
Outcome	nailed_it / failed	1.0×
	close	0.7×
	unknown	0.3×
Conf boost	≥ 0.75	+1.8× (capped at 5.0 total)

engine/learning/knowledge.py — compute_signal_weight()

```
w = compute_signal_weight(
    skill_tier="pro",
    confidence_score=0.9,
    source="expert_review",
    outcome="nailed_it",
)  # → 5.0  (capped)

w = compute_signal_weight("intermediate", 0.8, "live", "failed")
# → 1.8  (confidence boost applied)

w = compute_signal_weight("beginner", 0.3, "live", "unknown")
# → 0.15
```

3-Gate CI Evaluator

Before any candidate can be promoted, it must pass three gates in sequence. Failing any gate blocks promotion entirely — the gates cannot be overridden except by the override_blocked flag with an explicit written reason.

Gate	What it checks	Failure condition
1 Signal Sufficiency	Weighted signals ≥ MIN_SIGNALS[risk_level]	Returns disposition=insufficient
2 Benchmark Delta	Overall score drop ≤ 2%	Returns disposition=blocked
3 Pattern Regression	No single pattern drops > 5%	Returns disposition=blocked

If all gates pass, the risk level determines the final disposition:

Risk Level	Disposition	What happens next

low	auto_deploy	Change deployed automatically; monitoring window opens
medium	human_review	Curator notified; must approve before deploy
high	human_gate	Committee review required regardless of gate scores
any	blocked	Gate 2 or 3 failed; fix regressions before re-evaluating
any	insufficient	Gate 1 failed; accumulate more signals and retry

engine/learning/ci_gate.py — evaluate_candidate_dict()

```
# Pure-function test (no DB)
result = evaluate_candidate_dict(
    candidate       = {"id": "c1", "pattern_id": "loop", "risk_level": "low"},
    insight         = aggregate_signals_for_pattern("loop", signals),
    benchmark_delta = 0.01,   # +1% overall score
)
# result.disposition → "auto_deploy"
# result.overall_verdict → "pass"

# DB-backed evaluation (loads signals + runs benchmark)
result = evaluate_candidate_gate("cand-uuid-here")
print(summarise_gate_result(result))
# → "■ Auto-deploy: 3/3 gates passed (low risk)"
```

Failure Detection & Candidate Pipeline

Failure Mode	Candidate Type	Trigger Condition
conversion_gap	blueprint_correction	CVR < 1.0% with n ≥ 5 sessions
confidence_mismatch	confidence_recalibration	CVR > 2σ below fleet mean
step_deviation	shoot_mode_step_fix	Global match rate < 30%

pattern_drift	dataset_promotion	Daily volume decline > 40%
trust_gap	trust_safety	Matched sessions convert worse than unmatched
env_conversion_gap	blueprint_correction	Environment-segmented CVR gap

CANDIDATE SAFETY RULES

- Candidates always begin in status="proposed". No candidate is ever auto-accepted.
- sandbox_eval verdict "blocked" prevents promotion. Override requires override_blocked=true + written reason.
- Confidence_recalibration is the only candidate type with an auto-apply endpoint.
- All other types require manual engine edits and a benchmark re-run.

Benchmark Enforcement System

The benchmark enforcement system ensures no candidate can degrade core accuracy. Gate 2 of the CI evaluator runs sandbox_eval.py against the Gold Set before any candidate can advance. Gate 3 adds a per-pattern regression check. Together they create a two-layer safety net: fleet-level accuracy (Gate 2) and individual pattern regression (Gate 3).

Gate	Enforcement layer	Pass threshold	Source module
Gate 2	Benchmark delta	Overall acc Δ ≥ −2%	sandbox_eval.py vs Gold Set

Gate 3	Pattern regression	Per-pattern acc Δ ≥ −5%	per-pattern subset of Gold Set
Blocked	Both gates failed	N/A — manual override req	ci_gate.py disposition=blocked

Gold Set promotion: when a session reaches confidence ≥ 0.85 with outcome "nailed_it" it becomes a gold set candidate (GET /lab/signals/gold-set-suggestions). Human-reviewed gold set images set the accuracy baseline that Gates 2 and 3 measure against. The larger and more diverse the gold set, the tighter the enforcement.

engine/learning/ci_gate.py — gate dispositions

```
# Disposition map after all gates:
# "auto_deploy"    → all 3 passed + risk_level="low"
# "human_review"   → all 3 passed + risk_level="medium"
# "human_gate"     → all 3 passed + risk_level="high"
# "blocked"        → Gate 2 or Gate 3 failed
# "insufficient"   → Gate 1 (signal count) failed

# Run it programmatically:
result = evaluate_candidate_dict({
    "pattern_id":          "rembrandt",
    "weighted_signals":    212,
    "benchmark_delta":     -0.01,
    "pattern_acc_delta":   0.02,
    "risk_level":          "high",
})
# → CIGateResult(disposition="human_gate", gates_passed=[1,2,3])
```

ADDING BENCHMARK CASES

- POST /lab/benchmarks/cases — add individual cases manually.
- POST /lab/benchmarks/from-gold-set — promote gold set suggestions in bulk.
- Run python3 scripts/run_benchmarks.py after each engine edit.
- Zero FAIL results required before any Gate 2 benchmark delta is computed.

Revenue Projection & 30-Day Simulation

engine/learning/revenue.py quantifies the revenue impact of a CVR improvement. It answers three questions: (1) How much revenue does fixing this pattern unlock? (2) Which day does the signal gate clear? (3) What's the difference between treating all patterns identically vs. risk-tiering the deployment schedule?

engine/learning/revenue.py — compute_revenue_impact()

```
scenario = compute_revenue_impact(
    pattern_id       = "rembrandt",
    sessions_per_day = 40,
    before_cvr       = 0.042,   # 4.2% current
    after_cvr        = 0.065,   # 6.5% target
    arpu             = 9.00,    # $9 per conversion
)
# scenario.cvr_lift                  → +2.3pp
# scenario.additional_conversions_30d → 27.6
# scenario.delta_revenue_30d          → $248.40
# scenario.annualised_delta           → $2,980.80
```

project_30_day_metrics() simulates day-by-day signal accumulation for a list of patterns under a given scenario. For each pattern it computes:

Field	Meaning
gate_unlock_day	Day signals cross MIN_SIGNALS[risk_level] threshold
deploy_day	gate_unlock_day + risk deploy lag (low=0d, med=+3d, high=+7d)
cumulative_revenue_delta	Running Σ of daily revenue delta from deploy_day onward
day_snapshots	List of 30 DaySnapshot objects — one per day, suitable for charts

scripts/simulate_30day.py runs the full simulation across three strategic scenarios with ASCII day-by-day signal bars, gate unlock events, and a final comparison table. The CRITICAL insight: risk-tiered deployment (Scenario 3) outperforms treating all patterns as high-risk by $172/month (+$2,063/year) — while maintaining the same safety gates for high-risk patterns.

bash — run the 30-day simulation

```
# Full 3-scenario comparison (recommended)
python3 scripts/simulate_30day.py

# Single scenario (Controlled Autonomy)
python3 scripts/simulate_30day.py --scenario 3

# JSON output for API / dashboard integration
python3 scripts/simulate_30day.py --json --scenario 3

# Example Scenario 3 output (key events):
# Day  1: Loop gate cleared  (low risk, 25 signals) → deploy same day → +$224
# Day  5: Clamshell gate cleared (med risk, 75 sig) → deploy day 8   → +$296
# Day 15: Rembrandt gate cleared (high risk, 200 sig)→ deploy day 22 → +$99
# ████████████████████████████████████████████████████████████████████
# 30d total: +$620.01   |   Annualised: +$7,440.12
```

Scenario	Loop	Clamshell	Rembrandt	30d Delta	Annualised	vs S1
1 — Conservative	High day 10	High day 10	High day 10	+$448	+$5,377	baseline
2 — Moderate	Low day 1	High day 10	Med day 5	+$600	+$7,204	+$1,827/yr
3 — Controlled Auto	Low day 1	Med day 5	High day 15	+$620	+$7,440	+$2,063/yr

THE RISK-TIER ADVANTAGE

- Scenario 3 dominates because Loop and Clamshell are the highest-volume patterns.
- Deploying Loop on Day 1 (not Day 10) captures 9 extra days of revenue lift.
- Rembrandt is correctly delayed — it is a high-risk ambiguous pattern requiring 200 signals.
- The $172/month difference vs Scenario 1 compounds to $2,063/year at current volume.
- This justifies maintaining the knowledge base risk tiers with discipline.

API integration: POST /lab/learning/revenue/simulate accepts a JSON array of SimulateScenario objects and returns a RevenueProjection array with full day_snapshots suitable for rendering as a 30-day bar chart in the Lab UI.

API Endpoints — Knowledge & Revenue

Method	Path	Purpose
GET	/lab/learning/knowledge	Full pattern knowledge base (27 patterns)
GET	/lab/learning/knowledge/{id}	Single pattern entry with symptoms + fix steps
POST	/lab/learning/knowledge/{id}/signals	Aggregate weighted signals for a pattern
POST	/lab/learning/knowledge/{id}/ci-gate	Run full 3-gate CI evaluation for a candidate
POST	/lab/learning/revenue/impact	Compute CVR delta revenue impact
POST	/lab/learning/revenue/simulate	30-day simulation for N scenario configs

AUTH REQUIREMENTS

- All /lab/learning/* endpoints require JWT with email in NGW_DEV_EMAILS.
- Set NGW_DEV_MODE=1 to bypass auth locally. Never in production.
- Signal aggregate endpoint reads db/signals.py — ensure ngw.db is seeded for local testing.

Monitoring — Post-Release Windows

After a candidate is released, monitoring.py captures metric snapshots at 7, 14, and 30 days. Two alert types trigger automatic notifications:

Alert Type	Trigger Condition	Recommended Action

candidate_regression	success_rate_delta < –5pp OR conf < –0.1	Review candidate; consider rollback
rollback_review	CVR delta < –3pp OR trust delta < –0.15	Immediate rollback review required

bash — monitoring sweep

```
# Sweep all 3 windows (7d, 14d, 30d)
POST /lab/learning/monitoring/sweep-all

# Check alerts
GET  /lab/learning/monitoring

# Detailed report for one release
GET  /lab/learning/monitoring/{attribution_id}
```

DOCUMENT 13

Operations Manual

Installation, CLI, database, deployment, and incident response.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

System Requirements

Requirement	Minimum	Recommended	Notes
Python	3.11	3.12	FastAPI backend
Node.js	20 LTS	22 LTS	Frontend build
RAM	2 GB	4 GB	VLM peaks ~1.5 GB
Storage	5 GB	20 GB	Images + DB
Network	Outbound HTTPS	Low-latency	VLM API calls
Database	SQLite 3.x	PostgreSQL 15+	SQLite = dev only

Installation

bash — clone repository

```
$ git clone https://github.com/your-org/ngw-core.git
$ cd ngw-core
```

bash — Python environment setup

```
$ python3 -m venv .venv
$ source .venv/bin/activate          # macOS / Linux
$ .venv\Scripts\activate             # Windows
$ pip install --upgrade pip
$ pip install -r requirements.txt
```

bash — Frontend setup

```
$ cd ui
$ npm install
$ cd ..
```

First-Time Setup

Copy the environment template and fill in required values before starting the server.

.env.local — minimum required variables

```
# AI Gateway (choose one auth method)
AI_GATEWAY_API_KEY=gw_xxxxxxxxxxxxxxx
# OR use OIDC (preferred on Vercel):
# VERCEL_OIDC_TOKEN=<auto-provisioned by vercel env pull>

# Database
DATABASE_URL=sqlite:///./data/ngw_users.db

# CORS
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173

# Lab access (comma-separated emails)
LAB_ALLOWED_EMAILS=you@org.com,teammate@org.com

# Cron authentication
CRON_SECRET=your-random-secret-here
```

bash — initialize database and seed data

```
# Create tables
$ python3 -c "from db.database import init_db; init_db()"

# Load starter dataset (seeded signals + gold sets)
$ python3 scripts/seed_starter_dataset.py

# Verify dataset integrity
$ python3 scripts/verify_dataset.py
```

Development Server

Run backend and frontend in two separate terminal sessions.

terminal 1 — start backend (auto-reload on save)

```
$ source .venv/bin/activate
$ uvicorn main:app --reload --host 0.0.0.0 --port 8000
# INFO:     Uvicorn running on http://0.0.0.0:8000
# INFO:     Application startup complete.
```

terminal 2 — start frontend (Vite HMR)

```
$ cd ui
$ npm run dev
# VITE v5.x  ready in 300 ms
# ➜  Local:   http://localhost:5173/
# ➜  API:     http://localhost:8000/ (proxied)
```

QUICK VERIFY

- Open http://localhost:5173 — the home screen should load.
- Open http://localhost:8000/health — should return {"status":"ok"}.
- Open http://localhost:8000/docs — FastAPI auto-generated Swagger UI.

Database Operations

bash — run schema migrations

```
$ python3 scripts/run_migrations.py
# Migrations are idempotent — safe to re-run.
# Always backup before migrating in production.
```

bash — backup and restore (SQLite)

```
# Backup
$ sqlite3 data/ngw_users.db ".backup data/backup_$(date +%Y%m%d).db"

# Verify backup
$ sqlite3 data/backup_$(date +%Y%m%d).db ".tables"

# Restore
$ cp data/backup_YYYYMMDD.db data/ngw_users.db
```

bash — inspect schema and table sizes

```
$ sqlite3 data/ngw_users.db
sqlite> .tables
sqlite> .schema session_signals
sqlite> SELECT COUNT(*) FROM session_signals;
sqlite> SELECT signal_source, COUNT(*) FROM session_signals GROUP BY 1;
sqlite> .quit
```

Environment Variables — Complete Reference

Variable	Type	Required	Default	Description
AI_GATEWAY_API_KEY	string	alt.	—	Vercel AI Gateway key (or use OIDC)
VERCEL_OIDC_TOKEN	string	alt.	auto on Vercel	Short-lived JWT; preferred in prod
AI_MODEL_STRING	string	No	gateway default	VLM model identifier for AI Gateway
DATABASE_URL	string	Yes	sqlite:///./data/…	SQLite path or Postgres DSN
ALLOWED_ORIGINS	string	Yes	localhost:3000,5173	Comma-separated CORS origins
UPLOAD_DIR	string	No	static/uploads	Directory for uploaded images
LAB_ALLOWED_EMAILS	string	Yes	—	Comma-separated Lab-access emails
CRON_SECRET	string	Yes	—	Auth header for scheduled jobs
LOG_LEVEL	string	No	INFO	DEBUG for verbose output
MAX_UPLOAD_MB	integer	No	10	Max image upload size in MB
VLM_TIMEOUT_SECONDS	integer	No	30	Per-pass VLM call timeout
VLM_PASS_COUNT	integer	No	3	Analysis passes per image

CONFIDENCE_EARLY_EXIT	float	No	0.94	Skip remaining passes if exceeded

Running Tests & Benchmarks

bash — test suite

```
# Full test suite
$ python3 -m pytest tests/ -q --tb=short

# Specific test files
$ python3 -m pytest tests/test_shoot_match.py -v
$ python3 -m pytest tests/test_engine.py -v
$ python3 -m pytest tests/test_signals.py -v

# Stop on first failure
$ python3 -m pytest tests/ -x --tb=long
```

bash — benchmarks and validation

```
# Pattern matching benchmarks
$ python3 scripts/run_benchmarks.py

# CI benchmark (used in CI/CD pipeline)
$ bash scripts/ci_benchmark.sh

# Validate system YAML configurations
$ python3 scripts/validate_system_yamls.py

# Validate reference catalog and packs
$ python3 scripts/validate_catalog_and_packs.py
```

Production Deployment — Vercel

bash — Vercel setup and deploy

```bash
# Install Vercel CLI
$ npm i -g vercel

# Link to Vercel project (first time only)
$ vercel link

# Pull environment variables (provisions OIDC token)
$ vercel env pull .env.local

# Deploy to preview
$ vercel deploy

# Deploy to production
$ vercel deploy --prod

# Inspect deployment
$ vercel inspect <deployment-url>
```

vercel.json — cron jobs configuration

```json
{
  "crons": [
    {
      "path": "/api/cron/nightly-benchmark",
      "schedule": "0 3 * * *"
    },
    {
      "path": "/api/cron/learning-sweep",
      "schedule": "0 4 * * 1"
    }
  ]
}
```

Health Monitoring

bash — health check commands

```
# Service liveness
$ curl http://localhost:8000/health
# → {"status":"ok","version":"1.0","uptime_s":3621}

# Database status
$ curl http://localhost:8000/health/db
# → {"tables":5,"schema_version":"2025-03","ok":true}

# Signal hygiene summary (Lab auth required)
$ curl -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       -H "x-lab-email: you@org.com" \
       http://localhost:8000/api/lab/signals/summary
# → {"live":1482,"seeded":500,"internal":23,"expert_review":8,
#    "learning_eligible":1482,"metrics_eligible":1482}
```

bash — log tailing and analysis

```
# Tail logs in development (uvicorn output)
$ uvicorn main:app --reload 2>&1 | tee -a logs/dev.log

# Filter for errors
$ grep "ERROR\|Exception\|500" logs/dev.log

# Count VLM timeouts in the last 24 hours
$ grep "VLM_TIMEOUT" logs/dev.log | grep "$(date +%Y-%m-%d)" | wc -l

# Watch for Lab access attempts
$ tail -f logs/dev.log | grep "lab-access"
```

Maintenance Procedures

Weekly

- Review open candidates — approve or reject items older than 7 days.
- Check gold set evaluation scores — flag any below 80%.
- Review signal hygiene summary — confirm live count is growing.
- Check VLM timeout rate — should stay below 2% of requests.

Monthly

- Run full gold set re-evaluation after any engine update.
- Rotate AI Gateway API key or verify OIDC refresh is working.
- Audit LAB_ALLOWED_EMAILS — remove any leavers.
- Archive seeded signals older than 90 days (they skew storage).
- Run python3 scripts/nightly_benchmark.py manually to verify regression baseline.

Incident Response Playbook

API RETURNS 500 ON /API/SHOOT-MATCH

- 1. curl http://localhost:8000/health/db confirm DB is accessible.
- 2. Check LOG_LEVEL=DEBUG output for exception traceback.
- 3. Confirm AI_GATEWAY_API_KEY / OIDC token is valid and not expired.
- 4. curl the /api/lab/analyze endpoint with the same image — compare responses.

VLM CALLS FAILING / TIMING OUT

- 1. Test gateway connectivity: curl https://api.vercel.ai/health
- 2. Check VLM_TIMEOUT_SECONDS — increase to 60 if inference is slow.
- 3. Check AI Gateway quota in the Vercel Dashboard.
- 4. Fallback: set AI_MODEL_STRING to a lighter model temporarily.

GOLD SET SCORES DROPPED AFTER ENGINE UPDATE

- 1. Identify which candidate was recently applied (check candidates table).
- 2. Review the candidate proposed_change JSON for the problematic change.
- 3. Revert: UPDATE candidates SET status='proposed' WHERE id='...';
- 4. Re-run gold set evaluation to confirm scores recover.
- 5. File a new candidate with more conservative proposed_change values.

DOCUMENT 14

Troubleshooting Guide

Diagnose and resolve issues with CLI commands, SQL queries, and I

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

Common Issues — Quick Reference

Symptom	Likely Cause	First Action
Recommendation confidence < 40%	Image quality / heavy grading	Use clean unedited reference
Wrong pattern detected	Mixed lighting or color grade	See Image Quality section below
Shoot Mode ceiling warning	Ceiling height not in Settings	Settings set ceiling height
Kit gear not in suggestions	Gear missing from My Kit	My Kit add missing item
Lab returns 403	Email not in allowlist	Add email to LAB_ALLOWED_EMAILS
Analysis > 30 s	VLM timeout	Check API key; increase timeout
/health returns 500	DB not initialized	python3 -c "from db.database import init_db; init_db()"
Gold set scores dropped	Recent candidate applied	Check candidates table; revert
Seeded data in live metrics	Signal flags not set correctly	Run hygiene audit SQL query
VLM returns garbled JSON	Model output parsing error	Check LOG_LEVEL=DEBUG for raw output
CORS error in browser	Origin not in ALLOWED_ORIGINS	Add origin to env var; restart

API Diagnostic Sequence

Run these checks in order. Each step narrows the failure surface.

bash — step-by-step API diagnostics

```
# 1. Is the backend alive?
$ curl -s http://localhost:8000/health | python3 -m json.tool
# Expected: {"status":"ok","version":"1.0"}

# 2. Is the database accessible?
$ curl -s http://localhost:8000/health/db | python3 -m json.tool
# Expected: {"tables":5,"schema_version":"...","ok":true}

# 3. Can you authenticate?
$ curl -s -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       http://localhost:8000/api/master-modes
# Expected: JSON list of master modes

# 4. Is Lab accessible?
$ curl -s -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       -H "x-lab-email: you@org.com" \
       http://localhost:8000/api/lab/signals/summary
# Expected: JSON with live/seeded/internal counts

# 5. Can VLM be reached? (requires a test image)
$ curl -s -X POST \
       -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       -H "x-lab-email: you@org.com" \
       -d '{"image_path":"static/uploads/test.jpg","include_debug_overlay":true}' …
       http://localhost:8000/api/lab/analyze | python3 -m json.tool
```

Log Analysis

The backend logs to stdout in structured format. Key prefixes to filter for:

bash — log patterns and grep commands

```
# Enable verbose logging
$ LOG_LEVEL=DEBUG uvicorn main:app --reload

# All errors in current run
$ grep -E "ERROR|Exception|Traceback" /tmp/ngw-dev.log

# VLM call latencies (ms)
$ grep "vlm_pass_ms" /tmp/ngw-dev.log | tail -20

# Failed Lab access attempts
$ grep "lab_access_denied" /tmp/ngw-dev.log

# VLM timeout events
$ grep "VLM_TIMEOUT" /tmp/ngw-dev.log | wc -l

# Signals inserted today
$ grep "signal_inserted" /tmp/ngw-dev.log \
  | grep "$(date +%Y-%m-%d)" | wc -l
```

Database Diagnostic Queries

sql — database integrity and health checks

```sql
-- Table row counts
SELECT name,
  (SELECT COUNT(*) FROM sqlite_master WHERE type='table' AND name=m.name)
  AS exists_flag
FROM (VALUES
  ('session_signals'),('gold_sets'),('candidates'),
  ('reference_library'),('user_kits')
) AS m(name);

-- Signal count by source
SELECT signal_source, COUNT(*) AS n
FROM session_signals GROUP BY signal_source;

-- Open candidate queue
SELECT status, COUNT(*) FROM candidates GROUP BY status;

-- Gold set score summary
SELECT status, COUNT(*) FROM gold_sets GROUP BY status;

-- Most recent 5 signals
SELECT id, pattern, outcome, signal_source, created_at
FROM session_signals ORDER BY created_at DESC LIMIT 5;
```

sql — find flag_mismatch rows (seeded marked as live)

```sql
SELECT id, session_id, signal_source, created_at,
       include_in_metrics, include_in_learning
FROM session_signals
WHERE signal_source IN ('seeded', 'internal')
  AND (include_in_metrics    = TRUE
    OR include_in_learning   = TRUE
    OR include_in_conversion = TRUE
    OR include_in_cohorts    = TRUE);
```

Image Quality Diagnostics

Condition	Confidence Impact	Detection	Fix
B&W; / monochrome	–25%	Chroma saturation < 0.05	Use color original
Heavy color grade	–20%	Hue shift or LUT signature	Export ungraded
No face detected	–30%	Face region score = 0	Ensure face in frame
Resolution < 512 px	–30%	Image width check at ingest	Use ≥ 1024 px wide
Extreme contrast	–15%	Histogram clipping > 5%	Recover highlights/shadows
Multiple subjects	–10%	Multi-face region flag	Crop to primary subject
Environmental clutter	–10%	Background complexity score	Plain background preferred

VLM Troubleshooting

bash — test VLM connectivity independently

```
# Test gateway reachability
$ curl -s -o /dev/null -w "%{http_code}" \
       -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       https://api.vercel.ai/v1/models
# Expected: 200

# Test with a simple text prompt (no image)
$ curl -s -X POST https://api.vercel.ai/v1/chat/completions \
       -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       -H "Content-Type: application/json" \
       -d '{"model":"anthropic/claude-haiku-4","messages":[{"role":"user","content…
```

VLM TIMEOUT CHECKLIST

- VLM_TIMEOUT_SECONDS is set (default 30). Increase to 60 for slow inference.
- VLM_PASS_COUNT is set (default 3). Reduce to 1 for faster debug iteration.
- Check AI Gateway quota in Vercel Dashboard AI Gateway Usage.
- Try a lighter model: set AI_MODEL_STRING=anthropic/claude-haiku-4 temporarily.

Error Codes — Complete Reference

Code	Message	Likely Cause	Action
400	Invalid image format	Unsupported file type	Use JPEG/PNG/HEIC ≤ 10 MB
400	Missing required field: pattern	Bad request body	Check API docs at /docs
401	Unauthorized	Missing or expired Bearer token	Refresh token; re-run vercel env pull
403	Lab access denied	Email not in allowlist	Add to LAB_ALLOWED_EMAILS; restart
413	Request entity too large	Image > MAX_UPLOAD_MB	Resize image; or raise limit
422	Unprocessable entity	Schema validation failed	Check field types in body
429	Too many requests	Rate limit hit	Back off; check gateway quota
500	Internal server error	Unhandled exception	Check logs; curl /health/db
503	Service unavailable	Backend not running	Start uvicorn; check process
504	Gateway timeout	VLM call timed out	Increase VLM_TIMEOUT_SECONDS

Common Fix Procedures

Reset a stuck analysis session

sql — clear stuck in-flight sessions

```sql
UPDATE session_signals
SET outcome = 'unknown'
WHERE outcome IS NULL AND created_at < DATE('now','-1 hour');
```

Rebuild SQLite indexes after large import

sql — rebuild indexes

```sql
REINDEX session_signals;
REINDEX gold_sets;
VACUUM;  -- reclaim space after bulk deletes
```

GETTING HELP

- Include: session_id (from URL after analysis), error code, LOG_LEVEL=DEBUG output snippet, and signal_source if the issue is signal-related.
- Open a Lab candidate with type="trust_safety" to track systemic issues for curator review.

DOCUMENT 15

Paywall + Pricing Guide

Plans, features, and upgrade paths.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

Plans Overview

NGW offers three plans designed for different photographer profiles — from occasional use to full professional studio workflows. All plans include the core analysis engine; premium plans add Shoot Mode, advanced matching, and multi-session history.

	Free	Pro	Studio
Price	$0/mo	$14/mo	$39/mo
Analyses / month	5	Unlimited	Unlimited
Shoot Mode	—		
My Kit	Basic	Full	Full + Teams
Pattern Library	10 patterns	30+ patterns	30+ patterns
Recipes	3 recipes	13 recipes	13 recipes
Session History	7 days	90 days	Unlimited
Test Shot Evaluation	—		
NGW Lab Access	—	—	On request
Priority Support	—	Email	Priority

Feature Gating

Features are gated at the component level. When a user without access triggers a premium feature, a Paywall Gate is shown inline — no redirect, no interruption to their current session.

Gated Features

- Shoot Mode — Requires Pro or Studio. PaywallGate shown inline on entry.
- Shoot Mode Test Shot Evaluation — Requires Pro or Studio.
- Full pattern library (30+ patterns) — Free plan sees 10 core patterns only.
- All 13 Recipes — Free plan sees 3. Pro unlocks all.
- Kit sub-groups and team sharing — Studio only.
- NGW Lab — Studio + approved email. Requires Settings Dev Tools activation.

Upgrade Paths

From	To	Key Unlock	Suggested Trigger
Free	Pro	Shoot Mode + unlimited analyses	After 3rd analysis in a month
Free	Pro	All 13 Recipes	After viewing recipe 3 of 3
Pro	Studio	Team kit sharing + Lab access	After first team session
Studio	Studio+	Lab access + priority onboarding	After 3 months Studio

Billing & Access Notes

BILLING

• Subscriptions are managed through the account portal. Upgrades take effect immediately; downgrades take effect at the end of the billing period. No partial-month proration.

• NGW Lab access is not purchasable — it requires an approved team email. Studio plan is required as a prerequisite, but does not automatically grant Lab access.

ENTERPRISE & TEAM PRICING

• For teams of 5+ photographers, contact us for custom pricing. Team plans include shared kit libraries, multi-user Lab access, dedicated onboarding, and SLA support.

DOCUMENT 16

Developer Guide

Contributing, project structure, adding patterns, and the test suite.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

Who This Guide Is For

This guide is for engineers working on the NGW core engine, frontend, or CLI tooling. It assumes familiarity with Python 3.11+, FastAPI, and React 18. It covers: project layout, backend architecture, how to add or modify a lighting pattern, the test and benchmark systems, and the contribution workflow.

Python 3.11	FastAPI	React 18	Pytest
Backend runtime	REST API layer	Frontend SPA	Test framework

Project Structure

ngw-core — top-level directory layout

```
ngw-core/
■■■ engine/                # Core analysis pipeline
■   ■■■ orchestrator.py    # analyze_image() entry point
■   ■■■ vision_pipeline.py # Image processing + MediaPipe
■   ■■■ vlm_adapter.py     # AI Gateway VLM calls
■   ■■■ solver.py          # Weighted consensus solver
■   ■■■ blueprints/        # Pattern YAML definitions
■   ■■■ learning/          # Failure detection + candidates
■   ■■■ services/          # shoot_match_service.py etc.
■■■ api/
■   ■■■ main.py            # FastAPI app + lifespan
■   ■■■ routes/            # shoot_match.py, lab.py, signals.py …
■■■ db/
■   ■■■ database.py        # init_db(), engine, session
■   ■■■ models.py          # SQLAlchemy table models
■■■ ui/
■   ■■■ src/
■   ■   ■■■ components/    # React components
■   ■   ■■■ pages/         # Route-level pages
■   ■   ■■■ hooks/         # Custom React hooks
■   ■■■ vite.config.ts
■■■ tests/                 # pytest test suite
■■■ scripts/               # CLI tools (see Lab Guide)
■■■ docs/                  # Screenshots, PDFs, markdown
■■■ data/                  # SQLite DB (dev only)
```

Backend Architecture

The FastAPI backend is structured around three layers: routes (HTTP handlers), services (orchestration logic), and the engine (analysis pipeline). Routes are thin — they validate input, call services, and format responses. Services own business logic. The engine is stateless and pure.

Layer	File(s)	Responsibility
Routes	api/routes/*.py	HTTP validation, auth checks, response formatting
Services	engine/services/*.py	Orchestrate engine calls, build response models
Engine	engine/orchestrator.py	analyze_image() — calls pipeline solver blueprint
Pipeline	engine/vision_pipeline.py	Image decode, MediaPipe, region extraction
VLM	engine/vlm_adapter.py	AI Gateway calls, multi-pass, parse responses
Solver	engine/solver.py	Weighted consensus across VLM passes
Blueprints	engine/blueprints/*.yaml	Pattern definitions: tiers, thresholds, copy
DB	db/database.py + db/models.py	SQLAlchemy sessions, table models

Adding a New Lighting Pattern

Patterns are defined in YAML files under engine/blueprints/. The engine loads all YAML files at startup. Adding a pattern requires three steps: write the YAML definition, add gold set images, run benchmarks.

engine/blueprints/loop.yaml — example pattern definition

```yaml
pattern_id:    loop
display_name:  Loop
description:   "Key 45° off-axis, slight elevation. Nose-shadow loops down."

detection:
  confidence_threshold: 0.72
  primary_cues:
    - catchlight_position: [45, 315]   # degrees off center
    - shadow_direction:    loop         # named shadow type
    - fill_ratio_range:    [1.5, 4.0]  # key:fill stop range

blueprint:
  tiers:
    - tier: 1   # exact match
      key:   { type: monolight, modifier: octa_90cm, angle: 45, height: 7.5 }
      fill:  { type: v_flat, side: camera_left }
    - tier: 2   # close substitute
      key:   { type: speedlight, modifier: umbrella_60cm, angle: 45, height: 6.5 }
      fill:  { type: reflector, side: camera_left }

shoot_mode:
  step_order: [camera, key, fill, test, evaluate]
  ceiling_min_ft: 8
```

REQUIRED BLUEPRINT FIELDS

- pattern_id — must be unique, lowercase, underscore-separated.
- confidence_threshold — set conservatively (0.65–0.80). Too high = misses; too low = false positives.
- At least one tier-1 blueprint entry. Tier-5 (ambient) is optional but recommended.
- shoot_mode.ceiling_min_ft — prevents illegal placements on low ceilings.

1 Write the YAML

Create engine/blueprints/{pattern_id}.yaml following the schema above.

2 Validate YAML

Run: python3 scripts/validate_system_yamls.py --strict

3 Add Gold Set images

Add 3–5 labeled reference images via the Lab Gold Sets + Add New.

4 Run benchmarks

Run: python3 scripts/run_benchmarks.py. New pattern shows as NOT_TESTED.

5 Tune thresholds

Adjust confidence_threshold until PASS rate ≥ 75% on gold sets.

6 Write tests

Add test cases to tests/test_engine.py covering detection and blueprint generation.

7 Submit PR

Include: YAML, test results screenshot, benchmark output, gold set IDs.

Adding a New API Endpoint

All API endpoints are FastAPI route functions in api/routes/. Follow the existing patterns: validate with Pydantic, call a service function, return a typed response model.

api/routes/example.py — minimal endpoint pattern

```
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from db.database import get_db
from sqlalchemy.orm import Session

router = APIRouter(prefix="/api/example", tags=["example"])

class ExampleRequest(BaseModel):
    name: str
    value: float

class ExampleResponse(BaseModel):
    result: str
    processed: bool = True

@router.post("/", response_model=ExampleResponse)
async def create_example(
    body: ExampleRequest,
    db: Session = Depends(get_db)
) -> ExampleResponse:
    # Call service layer — never put business logic in routes
    result = example_service.process(db, body.name, body.value)
    return ExampleResponse(result=result)
```

api/main.py — register the router

```
# In api/main.py, import and include the new router:
from api.routes.example import router as example_router

app.include_router(example_router)
# Endpoint is now live at /api/example/
# Visible in Swagger UI at http://localhost:8000/docs
```

Convention	Rule
Route prefix	All routes use /api/ prefix. Lab routes use /api/lab/.
Auth	Lab routes require x-lab-email header checked against LAB_ALLOWED_EMAILS.
Error responses	Raise HTTPException with status_code and detail. Never return raw strings.
Pydantic models	Every request body and response is a typed Pydantic model. No bare dicts.
DB sessions	Always use Depends(get_db). Never import db.engine directly in routes.
Idempotency	POST endpoints that create resources must be idempotent on duplicate IDs.

Frontend Development

The frontend is a React 18 SPA built with Vite 5 and Tailwind CSS 3. The UI proxies all /api/ requests to localhost:8000. Component structure follows a pages components hooks hierarchy.

ui/src — component hierarchy

```
ui/src/
■■■ pages/
■   ■■■ HomePage.tsx          # Entry point — three mode tiles
■   ■■■ AnalyzePage.tsx        # Upload + recommendation display
■   ■■■ ShootModePage.tsx      # Step-by-step on-set instructions
■   ■■■ RecipesPage.tsx        # 13 recipe cards
■   ■■■ MyKitPage.tsx          # Equipment management
■   ■■■ SettingsPage.tsx       # User preferences
■   ■■■ LabPage.tsx            # NGW Lab (restricted)
■■■ components/
■   ■■■ ui/                    # Primitive UI components
■   ■■■ analysis/              # Recommendation display cards
■   ■■■ shoot-mode/            # Step cards, role selector
■   ■■■ lab/                   # Gold set, candidates, signals tabs
■■■ hooks/
    ■■■ useAnalysis.ts         # Upload + poll analysis result
    ■■■ useKit.ts              # My Kit CRUD state
    ■■■ useLabAuth.ts          # Lab email auth state
```

bash — frontend dev commands

```
# Start dev server with HMR
$ cd ui && npm run dev

# Type-check (no emit)
$ cd ui && npm run typecheck

# Lint
$ cd ui && npm run lint

# Production build (outputs to ui/dist/)
$ cd ui && npm run build

# Preview production build locally
$ cd ui && npm run preview
```

Test Suite

The test suite uses pytest with a shared SQLite in-memory database fixture. Tests are organized by domain and must not require a running server or live VLM calls. VLM calls are mocked via pytest-mock.

tests/ — directory layout

```
tests/
■■■ conftest.py              # Shared fixtures: db, client, mock_vlm
■■■ test_engine.py           # Pattern detection + blueprint generation
■■■ test_shoot_match.py      # End-to-end shoot match service
■■■ test_signals.py          # Signal ingestion, hygiene flags
■■■ test_candidates.py       # Candidate CRUD and approval workflow
■■■ test_gold_sets.py        # Gold set CRUD and evaluation
■■■ test_reference_library.py # Reference library ingestion
■■■ test_perception_layer.py  # Face validation, edge-case flags
■■■ test_learning.py         # Failure detection + severity scoring
```

bash — running tests

```
# Full suite (fastest — ~30 s)
$ python3 -m pytest tests/ -q --tb=short

# Single file with verbose output
$ python3 -m pytest tests/test_engine.py -v

# One specific test by name
$ python3 -m pytest tests/test_signals.py::test_hygiene_flags -v

# With coverage report
$ python3 -m pytest tests/ --cov=engine --cov=api --cov-report=term-missing

# Stop on first failure
$ python3 -m pytest tests/ -x
```

conftest.py — key fixtures

```
@pytest.fixture
def db():
    """In-memory SQLite database, created fresh per test."""
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture
def mock_vlm(mocker):
    """Patch VLM gateway so tests never make real API calls."""
    return mocker.patch(
        "engine.vlm_adapter.gateway.analyze_image",
        return_value=MOCK_VLM_RESPONSE
    )
```

Benchmark System

The benchmark system runs the engine against a curated gold set and reports PASS / SOFT_PASS / FAIL per image. Results gate the CI/CD pipeline — any regression in gold set score below 75% fails the build.

bash — running benchmarks

```
# Standard benchmark run (all patterns)
$ python3 scripts/run_benchmarks.py

# Single pattern only
$ python3 scripts/run_benchmarks.py --pattern loop

# Gold sets only (skip live-data benchmarks)
$ python3 scripts/run_benchmarks.py --gold-sets-only

# JSON output for CI parsing
$ python3 scripts/nightly_benchmark.py --output=json > bench.json

# CI wrapper — exits 1 if any gold set score drops
$ bash scripts/ci_benchmark.sh
```

Result	Meaning	Action Required
PASS	Detected pattern matches gold label	None
SOFT_PASS	Correct family, minor variant difference	Review if count increases
FAIL	Wrong pattern or confidence below threshold	Investigate + file candidate
NOT_TESTED	Pattern has no gold set images yet	Add gold sets via Lab
ERROR	Engine raised an exception on this image	Fix before merging

Contribution Workflow

All contributions follow a branch-per-feature workflow with mandatory tests and benchmark validation before merge. No direct commits to main.

1. **Branch**
   Create a feature branch: git checkout -b feat/your-description

2. **Develop**
   Make changes. Keep commits focused. Run tests frequently with pytest tests/ -q

3. **Test**
   All existing tests must pass. New features need new tests. Coverage should not drop.

4. **Benchmark**
   Run python3 scripts/run_benchmarks.py. No new FAIL results allowed.

5. **Validate YAMLs**
   If blueprints changed: python3 scripts/validate_system_yamls.py --strict

6. **Pull Request**
   Open a PR against main. Include benchmark output and test summary in description.

7 Review

At least one curator review required. High-risk changes (solver, VLM) need two.

8 Merge

Squash-merge after approval. CI re-runs benchmarks as a gate before merge.

WHAT REQUIRES EXTRA REVIEW

- Changes to engine/solver.py — affects all pattern confidence scores.
- Changes to engine/blueprints/*.yaml — affects recommendation output.
- Changes to signal inclusion flags — affects learning and metrics eligibility.
- New VLM prompts — requires before/after benchmark comparison in PR.
- Any change that touches session_signals schema — requires migration script.

Environment Setup for New Developers

New developers need three things: a working Python environment, a populated database, and access to the AI Gateway.

bash — complete new developer setup

```
# 1. Clone and enter the repo
$ git clone https://github.com/your-org/ngw-core.git && cd ngw-core

# 2. Python environment
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt

# 3. Frontend
$ cd ui && npm install && cd ..

# 4. Copy and fill in the env file
$ cp .env.example .env.local
# Edit .env.local — fill in AI_GATEWAY_API_KEY and your email

# 5. Initialize and seed the database
$ python3 -c "from db.database import init_db; init_db()"
$ python3 scripts/seed_starter_dataset.py

# 6. Verify everything works
$ python3 scripts/verify_dataset.py
$ python3 -m pytest tests/ -q
$ python3 scripts/run_benchmarks.py

# 7. Start development servers
$ uvicorn main:app --reload &
$ cd ui && npm run dev
```

GETTING LAB ACCESS

- Lab access requires your email in LAB_ALLOWED_EMAILS. Ask your team admin to add it and restart the backend.
- Then in the app: Settings Developer Tools enable, enter your email.
- The Lab tab appears immediately.

DOCUMENT 17

NGW Lab Manual

Signal pipeline, benchmark system v2, reference dataset, VLM archit

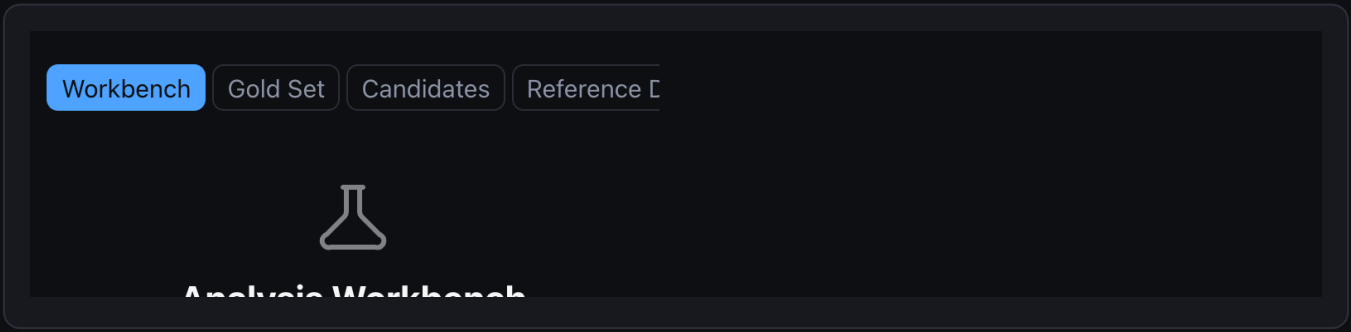
NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

About This Manual

The NGW Lab is the quality-control and continuous-improvement backbone of the No Guesswork engine. Curators use it to record and analyse outcome signals from real sessions, build and maintain the benchmark test-case library, manage gold-set reference images, propose and track rule candidates, and run the full suite of validation scripts. This manual documents every surface, API endpoint, database table, and CLI tool in the Lab — drawn directly from the source.

7-Stage	19	28	v2
VLM analysis pipeline	Canonical blueprints	Reference patterns	Benchmark system



NGW Lab — four-tab layout: Workbench, Gold Sets, Candidates, Signals

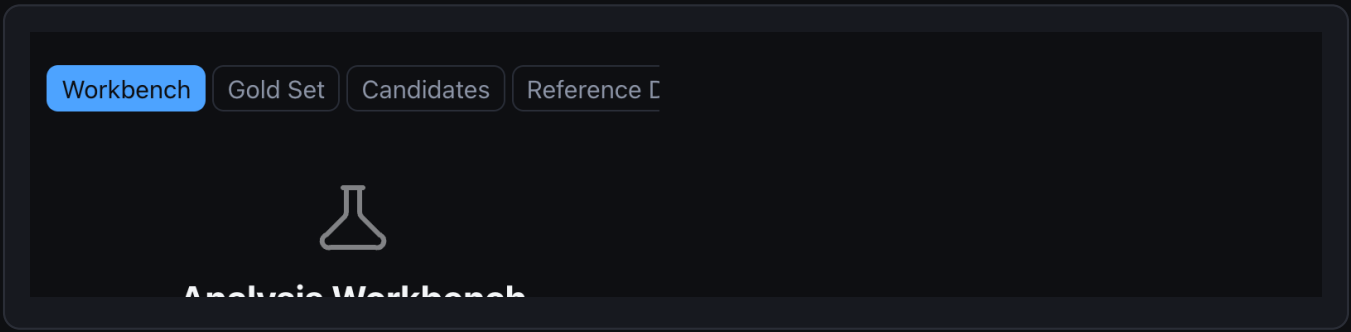
ACCESS CONTROL

- Lab features are gated by NGW_DEV_EMAILS — a comma-separated allowlist in .env.
- All Lab API endpoints require a valid JWT whose email appears in NGW_DEV_EMAILS.
- Set NGW_DEV_MODE=1 to bypass auth locally. NEVER set this in production.
- Enable in UI: Settings Developer Tools toggle on enter your email.

Role	Auth requirement	Key capability
Curator	JWT + NGW_DEV_EMAILS listed	Gold sets, candidates, benchmarks, signals
Second reviewer	JWT + NGW_DEV_EMAILS listed	Approve high-risk rule candidates
CI system	X-CI-Secret header or dev JWT	POST /api/lab/benchmarks/ci-run
Unauthenticated	None	POST /api/lab/signals (session recording)

Section 1 — Signal System

Every user session that produces an outcome is recorded as exactly one signal in the session_signals table. Signals are the primary data source for calibration analysis, pattern performance metrics, and automated candidate generation. The recording call is synchronous — fired at the moment the user submits feedback.



Lab → Signals — hygiene card with live / seeded / internal / analytics-eligible counts

Signal Schema — session_signals

Column	Type	Description
id	TEXT PK	Auto UUID hex
pattern_id	TEXT	Pattern attempted — required
outcome	TEXT	nailed_it | close | failed | unknown
confidence_score	REAL	Engine confidence 0.0–1.0
input_method	TEXT	wizard | reference_photo | manual
subject_type	TEXT	woman | man | child | couple | group
environment	TEXT	studio | indoor | outdoor

mood	TEXT	beauty | cinematic | corporate | editorial | natural
shoot_mode_entered	INT	1 if user opened Shoot Mode for this pattern
steps_completed	INT	Shoot Mode steps finished before feedback
steps_total	INT	Total steps in the assigned sequence
deviation_count	INT	Test-vs-reference evaluation deviations
saved_setup	INT	1 if user saved this setup to My Kit
upgraded	INT	1 if session led to subscription upgrade
revenue_value	REAL	Monetization attribution amount (if upgraded)
signal_source	TEXT	live | seeded | internal | expert_review
include_in_learning	INT	Eligible for candidate auto-generation (default 1)
include_in_metrics	INT	Included in analytics dashboards (default 1)
include_in_conversion	INT	Included in CVR calculations (default 1)
include_in_cohorts	INT	Included in cohort analysis (default 1)
session_id	TEXT	Optional — user session identifier
user_id	TEXT	Optional — user identifier
created_at	REAL	Unix timestamp (unixepoch() default)

Signal Source Taxonomy

Source	Meaning	include_in_* default
live	Real user session — production outcome	All = 1 (analytics eligible)

seeded	Bootstrap data from seed_starter_dataset.py	All = 0 (excluded)
internal	Dev / curator sessions during testing	All = 0 (excluded)
expert_review	Curator-flagged signal with correction	All = 0 (manually promoted)

ANALYTICS ELIGIBILITY RULE

- Only signal_source=live rows are included in metrics, conversion, and cohort analysis.
- Seeded and internal rows occupy the same table but with all include_in_* = 0.
- Expert-reviewed signals can be selectively promoted by setting include_in_learning=1 with a corrected pattern_id.
- The hygiene endpoint shows a real-time count of analytics-eligible vs excluded rows.

Signal API — Complete Reference

POST /api/lab/signals — record a session signal (public — no auth required)

```
{
  "pattern_id":          "rembrandt",
  "outcome":             "nailed_it",   # nailed_it | close | failed | unknown
  "confidence_score":    0.84,
  "input_method":        "reference_photo",
  "subject_type":        "woman",
  "environment":         "studio",
  "mood":                "beauty",
  "shoot_mode_entered":  true,
  "steps_completed":     6,
  "steps_total":         6,
  "deviation_count":     0,
  "saved_setup":         true,
  "upgraded":            false,
  "signal_source":       "live"
}
# Response
{ "success": true, "signal_id": "abc123",
  "outcome": "nailed_it", "signal_source": "live" }
```

GET /api/lab/signals/summary — headline KPIs (auth required)

```
# Query params: ?days=30&source=live
{
  "total_sessions":   842,
  "success_rate":     0.74,
  "top_pattern":      "loop",
  "worst_pattern":    "broad",
  "conversion_rate":  0.12,
  "avg_confidence":   0.79,
  "revenue_total":    1840.00
}
```

GET /api/lab/signals/patterns — per-pattern aggregation

```
# Query params: ?days=30&source=live
# Returns array — one entry per pattern:
{
  "pattern_id":     "loop",
  "success_rate":   0.81,
  "avg_confidence": 0.83,
  "outcomes": { "nailed_it": 48, "close": 22, "failed": 14 },
  "sample_count":   84
}
```

GET /api/lab/signals/calibration — confidence vs outcome mismatch

```
# Query params: ?days=30&source=live
# overconfident = true when gap > 0.20
{
  "pattern_id":       "broad",
  "avg_confidence":   0.78,
  "measured_success": 0.55,
  "gap":              0.23,
  "overconfident":    true
}
```

GET /api/lab/signals/recent — latest N signals

```
# Query params: ?limit=50&pattern_id=loop&source=live
# Default source=live. Each item:
# signal_id, pattern_id, outcome, confidence_score,
# input_method, environment, created_at, session_id
```

GET /api/lab/signals/hygiene — source breakdown (dev auth required)


```
{
  "by_source": {
    "live":          1482,
    "seeded":         500,
    "internal":        23,
    "expert_review":    8
  },
  "analytics_eligible": 1482,
  "flag_overconfident_patterns": ["broad", "clamshell"]
}
```


POST /api/lab/signals/seed — inject 45 bootstrap rows (dev auth)

```
# Query params: ?force=true  (required if seeds already exist)
# Inserts 45 synthetic rows with signal_source=seeded
# All include_in_* flags = 0  (excluded from all analytics)
# Response: { "seeded_count": 45, "skipped": 0 }
```

How To: Read a Calibration Report

1. **Open Lab Signals**
   Navigate to the Signals tab in the Lab navigation.

2. **Select Calibration**
   Tap the Calibration sub-tab — shows confidence vs outcome per pattern.

3. **Find overconfident flag**
   overconfident: true means engine confidence is 20+ pp above measured success.

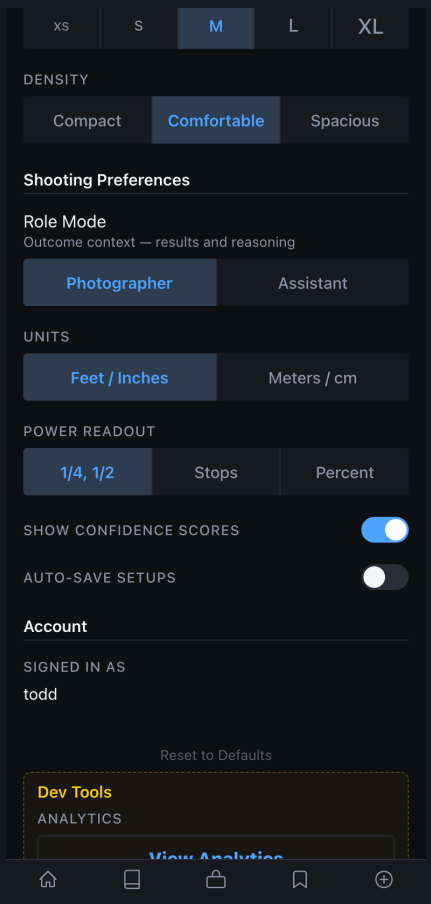
4 **Check sample_count**
Ignore patterns with sample_count < 20. Noise, not signal.

5 **Compare the gap**
gap = avg_confidence – measured_success. Gap > 0.20 = actionable.

6 **Create a rule candidate**
If gap > 0.20 with 20+ samples: document in rule_candidates (Section 3).

Section 2 — Benchmark System v2

The benchmark system evaluates engine accuracy against a curated library of test cases. Each case carries ground-truth expected analysis and expected blueprint geometry. Results are scored across four weighted dimensions and compared against a pinned baseline. CI runs block deployment when any gate condition fails.



Lab Settings — benchmark thresholds and evaluation gate configuration

Scoring Model

Dimension	Weight	What it measures	Target

pattern_accuracy	40%	Correct pattern detected vs. expected	>= 0.80
blueprint_score	30%	Key/fill geometry matches expected positions	>= 0.75
fix_score	20%	Correct fixes proposed for failure-mode cases	>= 0.70
confidence_error	10%	abs(predicted_conf - measured_conf)	<= 0.15

overall_score = 0.40 * pattern_accuracy + 0.30 * blueprint_score + 0.20 * fix_score + 0.10 * (1.0 - confidence_error)

Database Tables

Table	Primary purpose
benchmark_cases	Test case library — image_path, expected_analysis, expected_blueprint, difficulty
benchmark_runs	Run records — timestamps, scores, status, regression_count
benchmark_results	Per-case result per run — predicted_pattern, all four scores, regression_flag
pattern_metrics	Rolled-up per-pattern scores with last_updated timestamp
gold_set_entries	Curator-verified reference images — source material for benchmark cases

Benchmark Case Schema — benchmark_cases

Column	Type	Description

id	TEXT PK	Auto UUID hex
pattern_id	TEXT	Pattern under test
image_path	TEXT	Path to reference photo (required)
difficulty	TEXT	easy | medium | hard (default: medium)
environment_tags	JSON	Array of environment descriptors
expected_analysis	JSON	Ground-truth: pattern_id, environment, subject_type, skin_tone, mood
expected_blueprint	JSON	Ground-truth: key/fill angle_deg, height, modifier family
expected_fixes	JSON	Array of expected fix recommendations (for failure-mode cases)
source_gold_set_id	TEXT	FK to gold_set_entries (optional — set via from-gold-set promotion)
notes	TEXT	Curator notes
created_by	TEXT	Curator email

POST /api/lab/benchmarks/cases — create a test case

```
{
  "pattern_id":  "loop",
  "image_path":  "data/reference_dataset/loop/gs_001/image.jpg",
  "difficulty":  "medium",
  "environment_tags": ["studio", "low_fill"],
  "expected_analysis": {
    "pattern_id":   "loop",
    "environment":  "studio",
    "subject_type": "woman",
    "skin_tone":    "fair",
    "mood":         "beauty"
  },
  "expected_blueprint": {
    "key":  { "angle_deg": 45, "height": 7.5, "modifier": "octa_90cm" },
    "fill": { "side": "camera_left", "type": "reflector" }
  },
  "expected_fixes": [],
  "notes":      "Classic loop. Clean catch-light. No ceiling bounce.",
  "created_by": "curator@org.com"
}
# Response: { "id": "case_uuid", "pattern_id": "loop", "difficulty": "medium" }
```

POST /api/lab/benchmarks/cases/from-gold-set/{id} — promote gold set entry to case

```
# No request body. Fields inferred from gold_set_entries.expected_analysis.
# Sets: source_gold_set_id, expected_analysis, image_path automatically.
# Response: New benchmark_case record with all inferred fields populated.
```

Running Benchmarks

POST /api/lab/benchmarks/run — trigger a manual run

```
{
  "run_type":   "manual",
  "trigger":    "curator@org.com",
  "case_limit":  null,
  "notes":      "Pre-deploy validation after loop threshold change."
}

# Response (summarised):
{
  "run_id":              "run_uuid",
  "status":              "completed",
  "total_cases":         28,
  "passed_cases":        25,
  "overall_score":       0.871,
  "pattern_accuracy":    0.893,
  "avg_blueprint_score": 0.842,
  "confidence_error":    0.118,
  "regression_count":    0,
  "blocked":             false
}
```

POST /api/lab/benchmarks/ci-run — CI/CD integration (X-CI-Secret or dev JWT)

```
# Automatically sets run_type=ci, trigger=ci-system
# Response includes exit_code field for shell integration:
{ "status": "completed", "exit_code": 0,
  "overall_score": 0.871, "message": "All gates passed" }

# exit_code = 1 when ANY of these conditions is true:
#   overall_score < 0.70
#   any single pattern passes < 75% of its cases
#   regression vs baseline > 5 percentage points
```

GET /api/lab/benchmarks/runs/{id}/results — per-case detail (sorted worst-first)

```
# Each result:
{
  "case_id":          "case_uuid",
  "pattern_id":       "broad",
  "difficulty":       "hard",
  "predicted_pattern":"loop",
  "pattern_correct":  false,
  "blueprint_score":  0.62,
  "fix_score":        0.50,
  "confidence_error": 0.24,
  "final_score":      0.577,
  "regression_flag":  true,
  "error_msg":        "Pattern mismatch: expected broad, got loop"
}
```

GET /api/lab/benchmarks/baseline + POST to promote

```
# GET — check current baseline
{ "has_baseline": true, "baseline": {
    "run_id": "run_uuid",
    "overall_score": 0.871,
    "pattern_accuracy": 0.893,
    "set_at": 1742000000 } }

# POST — promote latest completed run to active baseline
# Body: {}  (no parameters)
# Only promote when regression_count = 0 and overall_score >= previous baseline.
```

CI GATE RULES (NEVER BYPASS)

- overall_score < 0.70 exit_code 1, deploy blocked.
- Any pattern: pass rate < 75% exit_code 1, deploy blocked.
- Regression vs baseline > 5pp exit_code 1, deploy blocked.
- Never promote a baseline with regression_count > 0.

How To: Add a Benchmark Case

1. **Choose reference image**
   Clean, unedited photo with clear lighting geometry. Min 1024px wide.

2. **Confirm ground truth**
   Note actual setup: key angle, height, modifier, fill side, ratio.

3. **POST the case**
   POST /api/lab/benchmarks/cases with all expected_* fields populated.

4. **Set difficulty honestly**
   easy = textbook; medium = real-world; hard = ambiguous or edge-case.

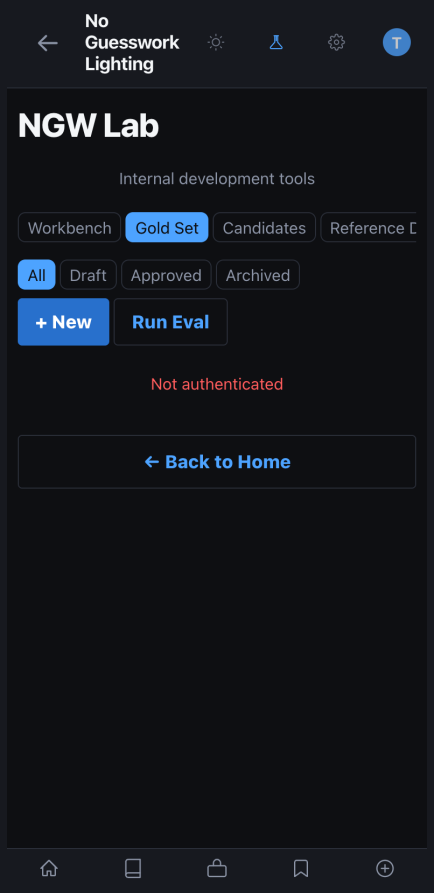
5. **Or promote from gold set**
   POST /api/lab/benchmarks/cases/from-gold-set/{id} auto-infers fields.

6 **Run targeted eval**
POST /api/lab/benchmarks/run with case_limit=1 to verify it scores correctly.

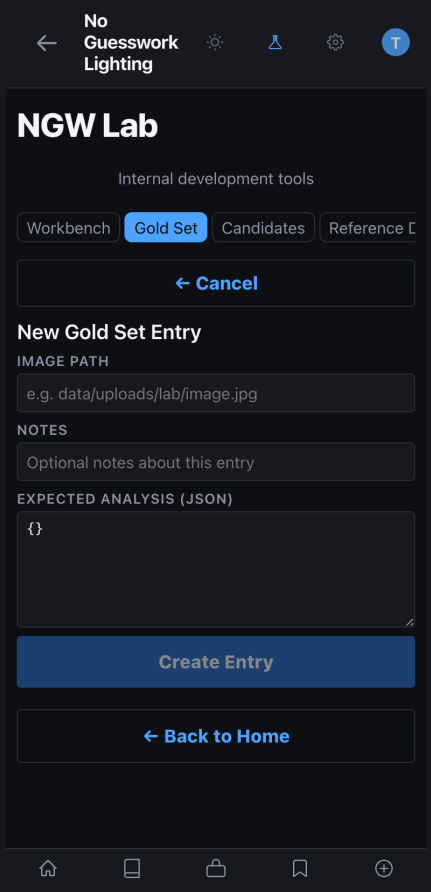
7 **Check pattern_correct**
GET /api/lab/benchmarks/runs/{id}/results — confirm pattern_correct: true.

Section 3 — Gold Sets & Rule Candidates

Gold set entries are curator-verified reference images with ground-truth metadata. They feed benchmark cases via the from-gold-set promotion API and supply evidence for rule candidates. Rule candidates are structured improvement proposals derived from calibration gaps, benchmark failures, or direct curator observation.



Gold Set listing — images with pattern labels, status, and linked benchmark case count



New Gold Set form — image path, pattern, expected_analysis JSON, and curator notes

Gold Set Entry Schema — gold_set_entries

Column	Type	Description
id	TEXT PK	Auto UUID hex
image_path	TEXT	Path to the verified photograph (required)
expected_analysis	JSON	Ground-truth object (see field list below)
notes	TEXT	Curator observation notes
status	TEXT	draft | approved | rejected

created_by	TEXT	Curator email
created_at	REAL	Unix timestamp
updated_at	REAL	Unix timestamp (updated on status change)

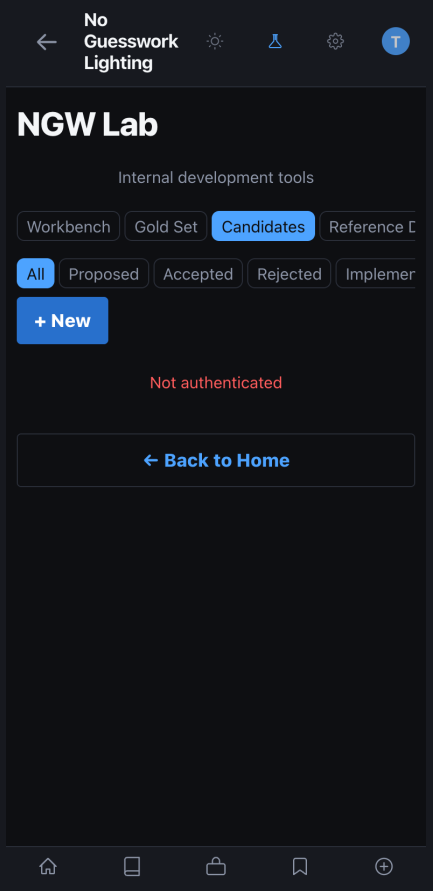
expected_analysis — all supported fields

```
{
  "expected_pattern":  "rembrandt",
  "lighting_family":   "directional",
  "pattern_id":        "rembrandt",
  "key_light":   { "position": "45deg", "modifier": "beauty_dish", "power": 1.0 },
  "fill_light":  { "position": "camera_left", "modifier": "reflector", "power": 0.…
  "rim_light":   { "position": "rear_right", "modifier": "strip_box", "power": 0.5…
  "environment":  "studio",
  "subject_type": "woman",
  "skin_tone":    "fair",
  "mood":         "editorial"
}
```

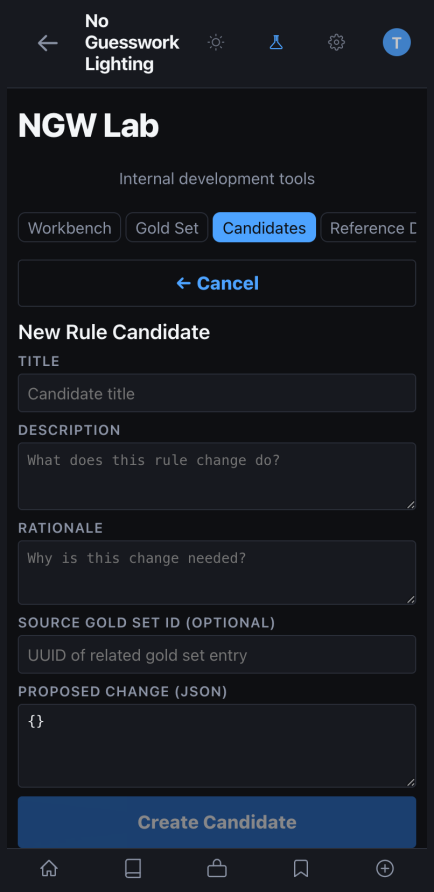
Rule Candidate Schema — rule_candidates

Column	Type	Description
id	TEXT PK	Auto UUID hex
title	TEXT	Concise rule description (required)
description	TEXT	Full rule specification (required)
rationale	TEXT	Evidence: calibration data, benchmark case IDs, gold set IDs
source_gold_set_id	TEXT	Optional FK to gold_set_entries

proposed_change	TEXT	Concrete JSON delta or patch specification
status	TEXT	proposed | under_review | approved | deployed
created_by	TEXT	Contributor email
created_at	REAL	Unix timestamp



Candidates queue — proposed, under_review, approved, and deployed entries with status badges



New Candidate form — title, description, rationale, proposed_change JSON, and linked gold set

Candidate Lifecycle

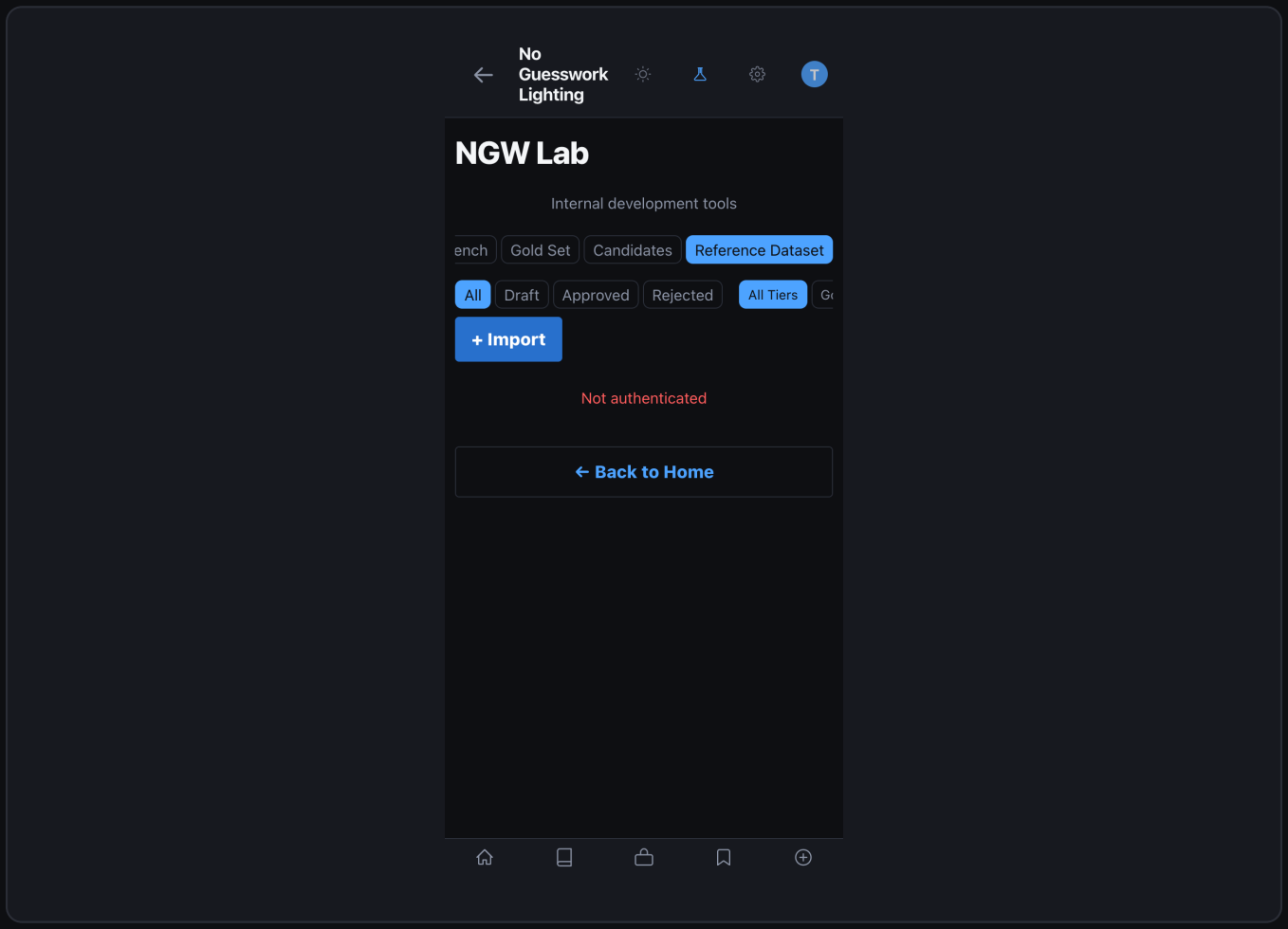
Status	Meaning	Who moves it
proposed	New — awaiting curator review	Creator submits
under_review	Active review in progress	Curator sets
approved	Accepted — ready for engine implementation	Curator approves
deployed	Change live in production engine	Developer deploys

CANDIDATE BEST PRACTICES

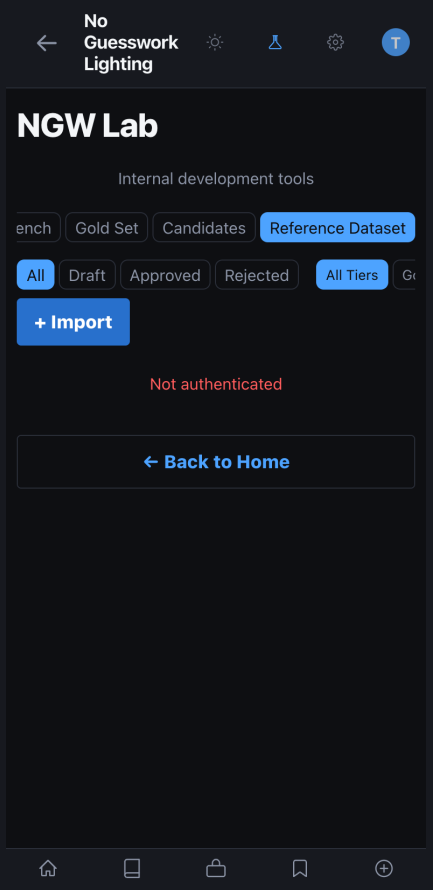
- Every candidate must cite evidence: signal calibration data, case IDs, or gold set IDs.
- proposed_change should be a concrete JSON delta — not vague description.
- Run a full benchmark AFTER implementing an approved candidate before marking deployed.
- If benchmark regresses after implementation: revert candidate status to proposed and add regression note.

Section 4 — Reference Dataset

The reference dataset is the curated library of labeled exemplar images that anchors the reference_read classifier — the highest-priority of the three active pattern-resolution classifiers. The engine loads the dataset at startup and queries it during every analysis. 28 canonical patterns are covered across three quality tiers.



Reference Dataset — grid view with 28-pattern coverage map, tier badges, and approval status



Reference Dataset detail — image with metadata, trust score, and ground_truth structure

28 Canonical Patterns

Family	Patterns
Portrait (studio)	loop, rembrandt, butterfly, clamshell, split, broad, short, paramount
Portrait (natural)	window_portrait, golden_hour, overcast_natural
Fashion / high-key	beauty, high_key_white, glamour, editorial_dramatic
Cinematic / moody	noir, chiaroscuro, low_key_dramatic
Environmental	silhouette, rim_backlit, hair_light_separation

Multi-light	three_point, cross_lighting, kicker_accent
Specialty	catchlight_accent, ambient_fill_dominant, mixed_source

Dataset Entry Metadata Schema

data/reference_dataset/{pattern}/{id}/metadata.json

```
{
  "reference_id":      "loop_gold_001",
  "pattern_id":        "loop",
  "photographer":      "curator@org.com",
  "dataset_tier":      "gold",          # gold | community | synthetic
  "entry_trust_score":  0.90,            # gold default 0.9, community 0.7
  "approval_status":   "approved",       # draft | approved | rejected
  "environment":       "studio",
  "source_type":       "original",
  "light_count":        2,
  "key_direction_deg":  45,
  "shadow_pattern":    "loop",
  "notes":             "Clean catch-light 10 oclock. No ceiling bounce.",
  "ground_truth": {
    "pattern":       "loop",
    "key_angle_deg":  45,
    "key_height":     7.5,
    "fill_ratio":     2.8,
    "modifier_family":"octa"
  },
  "benchmark_metadata": {
    "difficulty":             "medium",
    "category":               "standard",
    "expected_key_direction":  315
  }
}
```

Dataset Tier Reference

Tier	Trust score	Approval	Source
gold	0.90	Dual curator required	Known ground-truth, controlled studio setup
community	0.70	Single curator	User-contributed, curator-verified
synthetic	0.70	None required	Generated by seed_starter_dataset.py

key_direction_deg Convention

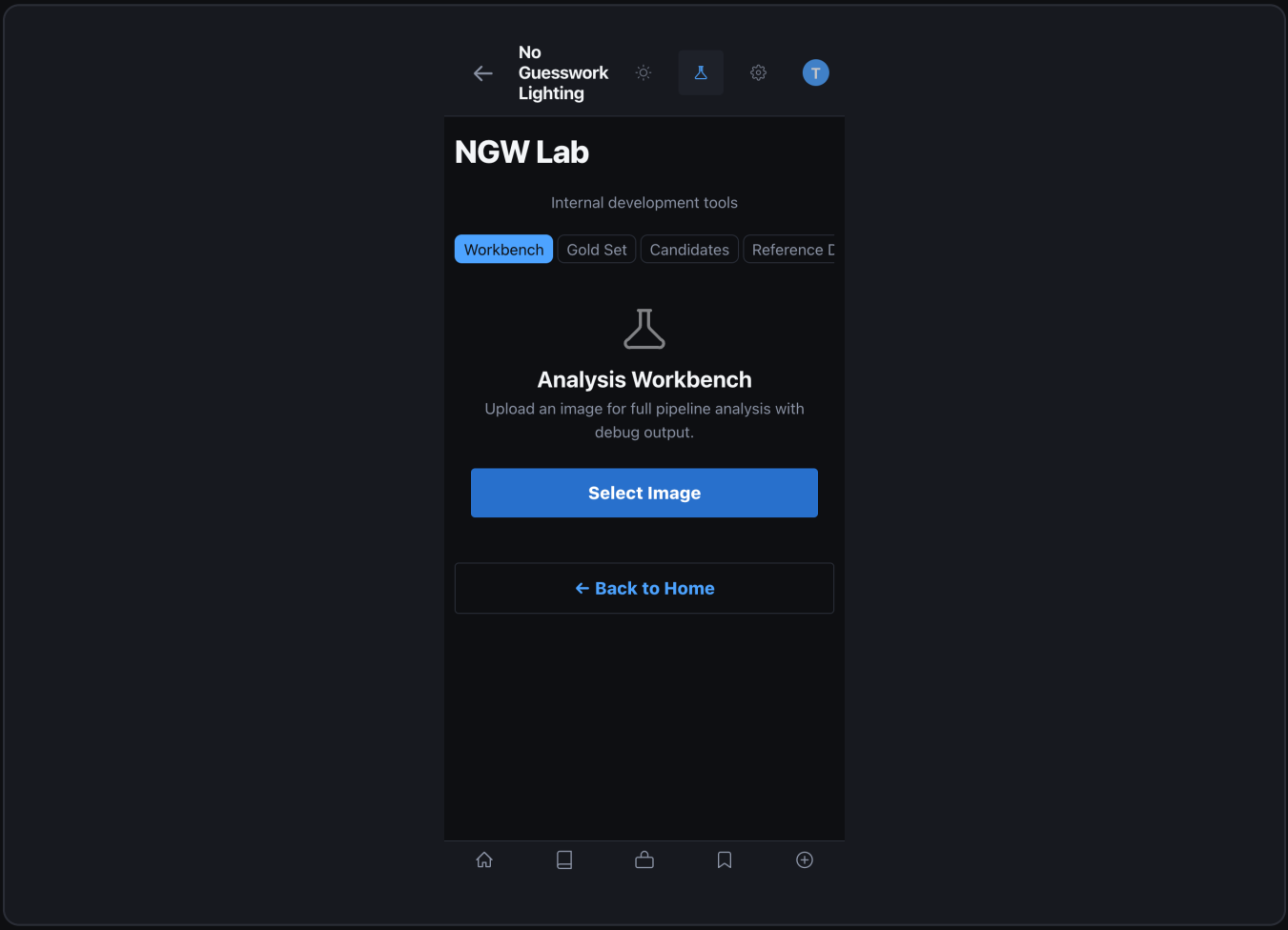
Direction label	key_direction_deg	Typical pattern
upper_left	315 deg	Rembrandt / loop (camera-right side)
upper_right	45 deg	Loop alternate (camera-left side)
left	270 deg	Hard split (camera right)
right	90 deg	Hard split alternate (camera left)
top_center	0 deg	Butterfly / paramount
center / unknown	null	Ambiguous — omit from geo inference

REQUIRED METADATA FIELDS

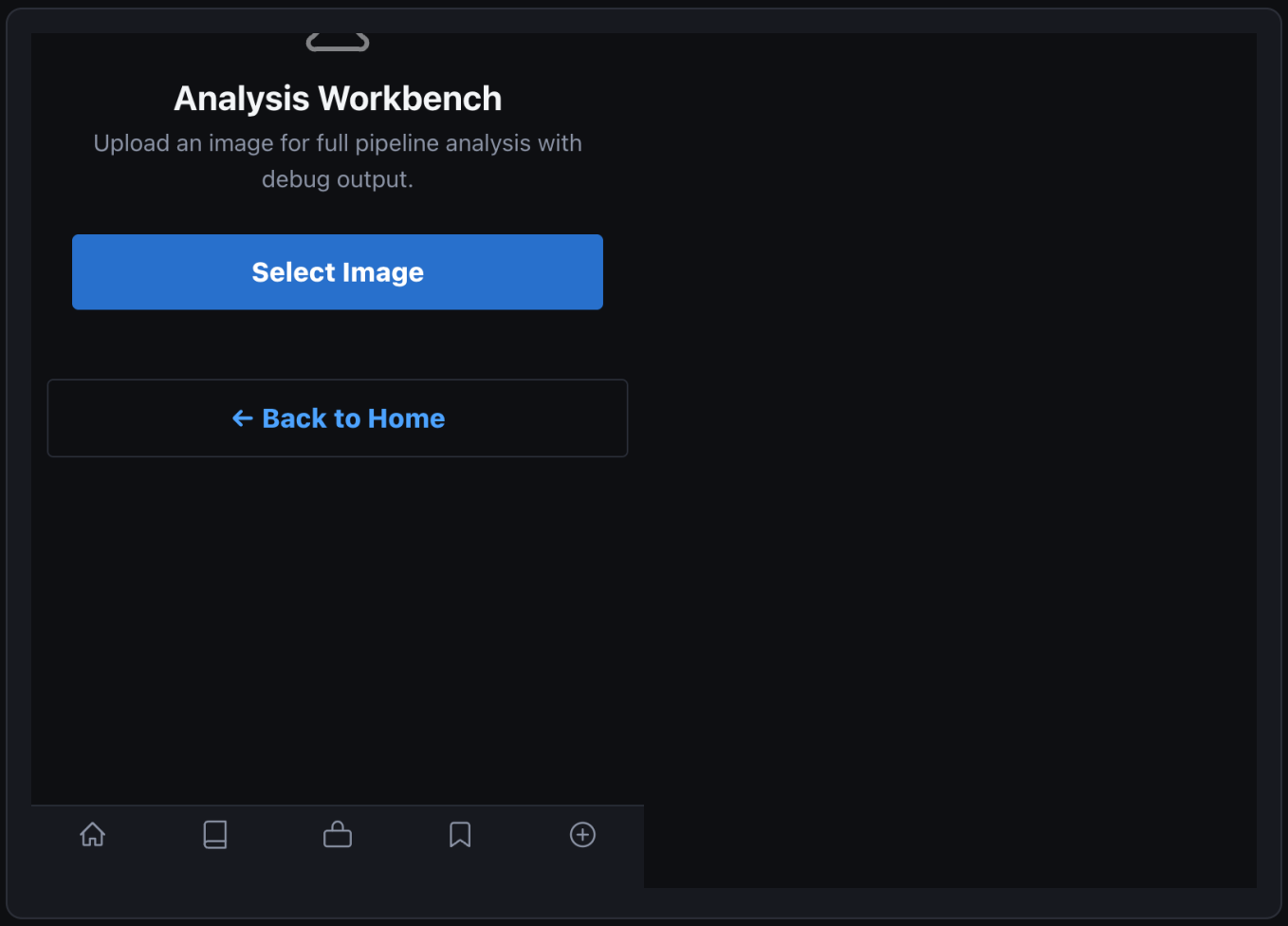
- REQUIRED_META_FIELDS: reference_id, pattern_id, dataset_tier, entry_trust_score.
- verify_dataset.py checks these fields on every entry. Missing = exit code 1.
- VALID_TIERS: gold | community | synthetic.
- VALID_APPROVAL: draft | approved | rejected. Never skip draft before approval.

Section 5 — VLM Analysis Pipeline

Every reference photo analysis runs through a deterministic 7-stage pipeline. The VLM is never asked to name a lighting setup — it extracts observable physical signals only. Pattern determination uses three independent classifiers whose outputs are resolved by the solver chain.



Workbench — debug overlay showing vision pipeline region annotations



Workbench detail — solver chain scores and per-classifier confidence breakdown

7-Stage Pipeline

Stage	Module	What it does
1	engine/vlm.py	VLM call 1 — extract observable signals: shadow, catchlight, geometry, highlights
2	engine/vision_pipeline.py	16+ CV passes — geometric, photometric, and semantic cue extraction

3	engine/cue_inference.py	4-stage inference: geometry source quality environment setup family
4	engine/vlm_reconstruction.py	VLM call 2 — reason about signals, build reconstruction candidates
5	engine/orchestrator.py	Solver chain: consensus consistency contradiction simulator validator
6	engine/orchestrator.py	Pattern resolution: 3 active classifiers ranked candidates
7	engine/reference_read.py	Reference read: cross-validate with dataset authoritative_pattern

Stage 1 — VLM Signal Extraction

The first VLM call uses a system prompt that explicitly restricts the model to observable physical signals only: shadow edges, catchlight shape and position, highlight geometry, colour temperature context. It must never name a setup.

Output	Fields
VLMDescription	subject_type, skin_tones, framing, pose, expression, styling, mood
VLMSignals	geometry, shadows, highlights, catchlights (all physical observables)

Provider selection: VLM_PROVIDER=auto detects openai first, then anthropic. Default models: GPT-4.1 (OpenAI) | claude-sonnet-4-20250514 (Anthropic). Set VLM_PROVIDER=none to disable VLM and fall back to rule-based only.

Stage 2 — Vision Pipeline (16+ Passes)

Pass group	Passes included

Geometry	geometry_pass, pose_solver_pass, inverse_square_solver_pass
Shadow	shadow_pass, shadow_penumbra_pass, occlusion_shadow_pass, multi_shadow_analysis
Light source	highlight_pass, specular_surface_pass, light_direction_field_pass
Catchlight	catchlight_pass (shape, topology, position — all three separate cues)
Environment	background_pass, window_geometry_pass, solar_geometry_pass, environment_light_pass
Modifier	modifier_shape_solver_pass, color_temperature_pass, bounce_geometry_pass
Composite	light_role_support_signals, global_uncertainty_notes

Output: VisualCueReport with 15+ structured cue objects — ShadowEdgeHardness, PrimaryShadowDirection, VerticalLightAngle, CatchlightPosition, CatchlightShape, CatchlightTopology, HighlightGeometry, ReflectionSignatures, BackgroundAnalysis, EnvironmentIndicators, ColorTemperatureContext, LightCountSignals, MultiShadowAnalysis, OcclusionPatterns, CompositeConfidence.

Stage 3 — 4-Stage Cue Inference

Inference stage	Output
GeometryInference	Light direction, height, count, fill ratio, shadow pattern
SourceQualityInference	Modifier family, transition character (soft / hard)
EnvironmentInference	Natural vs studio, background type, special cases
SetupFamilyInference	Primary hypothesis + alternate patterns + ambiguity notes

Stage 4 — VLM Reconstruction (12 Dimensions)

Reconstruction dimension	Description
Dominant source direction	Clock-position estimate from shadow + catchlight geometry
Dominant source height	Vertical elevation inference (low / eye / high)
Source size class	Large-soft / medium / small-hard modifier inference
Source distance class	Near / mid / far placement inference
Modifier family candidates	Octa / umbrella / dish / strip / bare / window (ranked)
Environment	Studio / natural / mixed / outdoor
Light count	Single / two-light / multi-light inference
Light roles	Key / fill / rim / hair / background role assignment
Negative fill likelihood	Probability that fill side uses absorption (black v-flat)
Background lighting likelihood	Probability of a dedicated background light
Bounce likelihood	Probability of bounce fill or ceiling bounce contamination
Ambiguity assessment	Structured uncertainty notes propagated to solver chain

Stage 5 — Solver Chain

Solver	Role
consensus_solver	Aggregate classifier outputs into weighted consensus scores
consistency_engine	Verify cue combinations obey physical lighting constraints
contradiction_engine	Flag and resolve contradictory cue pairs

lighting_simulator	Forward-simulate candidate patterns against the full cue report
hypothesis_validator	Score each hypothesis against all available evidence
solver_trace	Append full reasoning chain to output — never replaces upstream data

Stage 6 — 3-Classifier Pattern Resolution

resolve_pattern_candidates() in engine/orchestrator.py runs three independent classifiers in strict priority order. Higher priority wins on ties.

Priority	Classifier	Method	Evidence used
0 (high)	reference_read	build_lighting_read()	3-layer shadow/highlight/gobo vs dataset
1	catchlight_inference	_infer_pattern_from_catchlights()	Catchlight topology, shape, and count
2	shadow_inference	_infer_shadow_pattern()	Shadow direction + vertical height angle
fallback	rule-based	classify_lighting_pattern()	Mood + modifier + gear (no-image paths only)

AnalysisResult Output Structure

engine/image_analysis_models.py — AnalysisResult


```
{
  "authoritative_pattern": "loop",
  "pattern_candidates":    [{"pattern":"loop","score":0.84},{"pattern":"rembrandt"…
  "confidence_score":      0.84,
  "visual_cue_report":     { /* 15+ VisualCue objects */ },
  "solver_quality": {
    "consistency_score":   0.91,
    "contradiction_count": 0,
    "ambiguity_level":     "low"
  },
  "reference_data": { "matched_entries": [...], "dataset_tier": "gold" },
  "reconstruction": {
    "primary":   { "direction": "upper_left", "height": 7.5, "modifier": "octa" },
    "alternates": [...],
    "uncertainty_notes": []
  }
}
```

Section 6 — Blueprint System

Blueprints are the output specification layer — 19 YAML files in data/systems/canonical/ that map a resolved pattern to exact physical light placement instructions with capture settings, failure modes, and substitution guidance. Loaded at startup, validated by validate_system_yamls.py.

Complete Blueprint YAML Schema

data/systems/canonical/{pattern}.yml — header + environment + camera

```yaml
pattern:      loop
pattern_name: Loop
category:     portrait
difficulty:   medium        # easy | medium | hard
setup_time_minutes: 8
version: "1.0"
status: "active"            # active | experimental | deprecated

environment:
  required: false
  ceiling_height_min_ft: 9.0
  background: seamless
  background_distance_ft: 6.0

subject:
  distance_from_camera_ft: 8.0
  position: center

camera:
  height: 5.5
  lens: 85mm
  angle_to_subject_deg: 0.0
```

data/systems/canonical/{pattern}.yml — capture + lights

```yaml
capture_settings:
  iso:           100-400
  aperture:      f/2.8-f/8
  shutter:       1/125-1/250
  white_balance: 5500K
  notes: ["Meter off subject face"]

lights:              # Array — at least one role:key required
  - role: key
    modifier: octa_90cm
    angle_deg: 45     # horizontal off-axis (0=front, 90=hard side)
    height: 7.5       # feet above floor
    distance_ft: 6.0
    power_ratio: 1.0  # relative to key (key always 1.0)
  - role: fill
    modifier: v_flat_white
    angle_deg: 315
    height: 5.5
    distance_ft: 4.0
    power_ratio: 0.33
```

data/systems/canonical/{pattern}.yml — shadow + failure fields

```yaml
shadow_signature:
  expected_pattern: loop
  nose_shadow:  "Loop below and to the side of nose"
  cheek_shadow: "Triangle shadow fill side"
  contrast: 0.65    # 0=flat, 1=maximum
  softness: 0.70    # 0=hard, 1=very soft

failure_modes:
  - name: flat_lighting
    description: "Fill too bright, loop shadow disappears"
    recovery:    "Move fill back or reduce fill power by 1 stop"

substitutions:
  - if_missing:  octa_90cm
    use:          umbrella_60cm
    tradeoff:     "Slightly harder transition, shadow stays visible"

use_cases: ["Portrait", "Editorial", "Headshots"]
example_photographers: ["Platon", "Annie Leibovitz"]
```

CLOCK-POSITION CONVENTION

- angle_deg = horizontal off-axis from camera forward. 0 = directly in front.
- 45 deg = upper right of subject (camera right side). Catch-light at ~2 oclock.
- 315 deg = upper left of subject (camera left side). Catch-light at ~10 oclock.
- 90 deg = hard side-light. 270 deg = hard side-light (opposite).
- power_ratio is relative: key = 1.0 always. Fill at 0.33 = 1.6 stop under key.

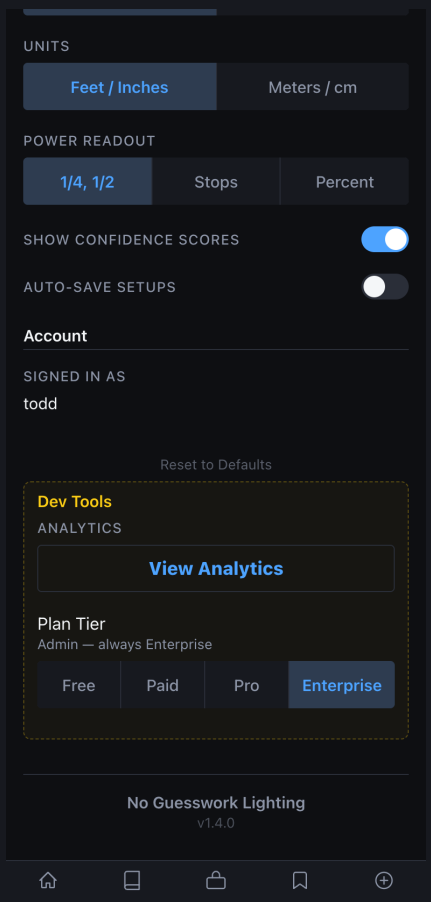
19 Active Blueprint Files

File	Pattern	Difficulty	Light count
loop.yml	Loop	medium	2 (key + fill)
rembrandt.yml	Rembrandt	medium	2 (key + reflector)
butterfly.yml	Butterfly / Paramount	easy	1-2 (frontal key)
clamshell.yml	Clamshell	medium	2 (top key + bottom fill)
split.yml	Split	easy	1 (hard 90 deg)
broad.yml	Broad	easy	2 (key shadow-side visible)
short.yml	Short	medium	2 (key lit-side hidden)
beauty.yml	Beauty	hard	3-4 (dish, fill, kicker)
window_portrait.yml	Window Portrait	easy	1 (natural window)
three_point.yml	Three-Point	medium	3 (key, fill, rim)
editorial_dramatic.yml	Editorial Dramatic	hard	2-3 (hard key + accent)
noir.yml	Noir	hard	1-2 (hard low-key)
high_key_white.yml	High Key White	medium	4+ (key + bg lights)
rim_backlit.yml	Rim / Backlit	medium	2 (rim + fill)
cross_lighting.yml	Cross Lighting	medium	2 (crossed 90 deg)

chiaroscuro.yml	Chiaroscuro	hard	1 (dramatic shadow)
kicker_accent.yml	Kicker Accent	medium	3 (key, fill, kicker)
golden_hour.yml	Golden Hour	easy	1 (natural warm)
ambient_fill_dominant.yml	Ambient Fill	easy	0 (natural ambient)

Section 7 — CLI Tools

All Lab operations can be driven from scripts/. Every script exits 0 on success and 1 on failure, enabling clean CI pipelines.



Dev Tools panel — database status, migration runner, and signal count monitor

verify_dataset.py

Flag	Effect
(none)	Full verification: all 28 patterns, image IO, metadata schema
--quick	Metadata-only — skip image file reads (faster for CI)

--fix	Auto-fix: regenerate thumbnails, rebuild manifest, reset bad fields

bash — verify_dataset.py

```
# Full verification
$ python3 scripts/verify_dataset.py

# Quick metadata check (no image IO)
$ python3 scripts/verify_dataset.py --quick

# Auto-fix minor issues
$ python3 scripts/verify_dataset.py --fix

# Checks: all 28 CANONICAL_PATTERNS present; REQUIRED_META_FIELDS
# present; valid dataset_tier and approval_status values;
# coverage report with missing_high_priority patterns.
# Exit 0 = PASS.  Exit 1 = FAIL (CI-safe).
```

seed_starter_dataset.py

Flag	Effect
(none)	Seed all benchmark entries into data/reference_dataset/
--dry-run	Preview — no disk writes
--force	Overwrite existing entries (use after schema changes)
--filter SUBSTRING	Only process benchmark IDs containing SUBSTRING

bash — seed_starter_dataset.py

```
# Normal seed (first run after init_db)
$ python3 scripts/seed_starter_dataset.py

# Preview without writing
$ python3 scripts/seed_starter_dataset.py --dry-run

# Force-overwrite (after schema change)
$ python3 scripts/seed_starter_dataset.py --force

# Seed only loop pattern entries
$ python3 scripts/seed_starter_dataset.py --filter loop

# Writes: image.jpg + metadata.json per entry,
# 200x200 thumbnails, _version.json, manifest with pattern_coverage.
# Seeded signals: signal_source=seeded, all include_in_*=0.
```

validate_system_yamls.py

bash — validate_system_yamls.py

```
# Validate all 19 canonical blueprint YAMLs
$ python3 scripts/validate_system_yamls.py

# Strict mode — fail on warnings (use before PR merge)
$ python3 scripts/validate_system_yamls.py --strict

# Required fields checked:
#   pattern, pattern_name, category, difficulty, version, status
#   environment.ceiling_height_min_ft
#   lights[] array with at least one role=key entry
#   lights[].angle_deg, height, distance_ft, power_ratio
#   shadow_signature.expected_pattern, contrast, softness
#   failure_modes[] with at least one entry
```

Full Script Reference

Script	Purpose	Key flags
verify_dataset.py	Dataset integrity check	--quick, --fix
seed_starter_dataset.py	Bootstrap synthetic reference data	--dry-run, --force, --filter
validate_system_yamls.py	Lint all 19 blueprint YAML files	--strict
validate_catalog_and_packs.py	Validate gear catalog completeness	--verbose
run_benchmarks.py	Pattern accuracy evaluation (manual)	--pattern=X, --limit=N
nightly_benchmark.py	Full CI benchmark suite (JSON output)	--output=json
ci_benchmark.sh	CI wrapper — exits 1 on regression	--exclude-pattern=X
capture_screenshots.py	Auto-capture UI screenshots for docs	--output-dir=PATH
generate_ngw_docs.py	Build this PDF documentation suite	(none)
generate_lab_manual.py	Build standalone Lab Manual PDF	(none)

Section 8 — Environment Configuration

NGW uses a single .env at the project root. Copy .env.example and fill in values before starting. Never commit .env to version control.

.env Variable Reference

Variable	Required	Description	Default
NGW_JWT_SECRET	Yes	JWT signing secret for Lab API auth	change-me
NGW_DEV_EMAILS	Yes	Comma-separated Lab access allowlist	(none)
NGW_DEV_MODE	No	Bypass JWT auth locally. NEVER in prod.	0
VLM_PROVIDER	No	auto | openai | anthropic | none	auto
VLM_MODEL	No	Override default model name	gpt-4.1
OPENAI_API_KEY	Cond.	Required if VLM_PROVIDER=openai or auto+OpenAI	(none)
ANTHROPIC_API_KEY	Cond.	Required if VLM_PROVIDER=anthropic or auto+Anthropic	(none)
ALLOWED_ORIGINS	No	CORS origins, comma-separated	localhost:5173
LOG_LEVEL	No	DEBUG | INFO | WARNING | ERROR	INFO
HOST	No	Server bind address	0.0.0.0
PORT	No	Server port	8000

VLM_PROVIDER=AUTO BEHAVIOUR

- auto checks OPENAI_API_KEY first, then ANTHROPIC_API_KEY.
- First key found determines which provider is used for the session.
- Set VLM_PROVIDER=none to disable VLM — analysis returns rule-based results only.
- VLM_MODEL override is optional. Defaults: GPT-4.1 (OpenAI) | claude-sonnet-4-20250514 (Anthropic).

Server Startup Sequence

bash — full first-time startup

```
# 1. Copy .env and fill in required values
$ cp .env.example .env

# Minimum required entries in .env:
#   NGW_JWT_SECRET=your-secret-here
#   NGW_DEV_EMAILS=your@email.com
#   OPENAI_API_KEY=sk-...  (or ANTHROPIC_API_KEY)

# 2. Install Python dependencies
$ pip install -r requirements.txt

# 3. Initialise the database
$ python3 -c "from db.database import init_db; init_db()"

# 4. Seed starter data
$ python3 scripts/seed_starter_dataset.py

# 5. Verify dataset integrity
$ python3 scripts/verify_dataset.py

# 6. Validate blueprint YAMLs
$ python3 scripts/validate_system_yamls.py --strict

# 7. Start API server
$ uvicorn api.main:app --reload --port 8000

# 8. Start frontend
$ cd ui && npm install && npm run dev

# 9. Enable Lab in UI
# Settings -> Developer Tools -> toggle on -> enter your NGW_DEV_EMAILS address
```

Section 9 — How-To Workflows

How To: Investigate a Failing Pattern

1. **Check calibration**
   GET /api/lab/signals/calibration?days=30 — find overconfident: true patterns.

2. **Check sample size**
   Ignore patterns with sample_count < 20. Calibration data needs volume.

3. **Run targeted benchmark**
   POST /api/lab/benchmarks/run with notes describing which pattern to watch.

4. **Review worst cases**
   GET /api/lab/benchmarks/runs/{id}/results — sorted worst-first by final_score.

5. **Inspect case history**
   GET /api/lab/benchmarks/cases/{id}/history — see if regression is new or old.

6. **Check blueprint YAML**
   Open data/systems/canonical/{pattern}.yml — verify angle_deg, shadow_signature.

7 **Validate YAML**
python3 scripts/validate_system_yamls.py --strict

8 **Check dataset coverage**
python3 scripts/verify_dataset.py — confirm pattern has approved gold entries.

9 **Create rule candidate**
POST /api/lab/rule_candidates with title, description, rationale, proposed_change.

10 **Re-benchmark after fix**
POST /api/lab/benchmarks/run to confirm improvement. Promote baseline if clean.

How To: Add a Gold Set Entry

1 **Choose a clean original**
Unedited JPEG with visible lighting geometry. Minimum 1024px wide.

2 **Confirm ground truth**
Record actual setup: key angle, height, modifier, fill side, ratio.

3 **Copy to dataset path**
data/reference_dataset/{pattern}/{id}/image.jpg

4 **Write metadata.json**
Fill all REQUIRED_META_FIELDS: reference_id, pattern_id, dataset_tier, entry_trust_score.

5 **Set dataset_tier correctly**
gold = 0.90 trust, dual curator. community = 0.70, single curator.

6 **Start as draft**
approval_status: draft always. Never commit approved without review.

7 **Run verify_dataset.py**
python3 scripts/verify_dataset.py — confirms entry passes schema check.

8 **Get second curator review**
Gold tier requires two curator approvals before approval_status: approved.

9 **Promote to benchmark case**
POST /api/lab/benchmarks/cases/from-gold-set/{id} to auto-infer case fields.

How To: Set Up CI Benchmark Gate

1 **Seed and verify locally**
Run seed + verify_dataset.py + validate_system_yamls.py all passing.

2 **Run first benchmark**
POST /api/lab/benchmarks/run — establish initial scores.

3 **Promote baseline**
POST /api/lab/benchmarks/baseline — pin the run as reference.

4 **Add CI step**
Call POST /api/lab/benchmarks/ci-run with X-CI-Secret header.

5 **Check exit_code**
exit_code: 0 = all gates passed. exit_code: 1 = deploy blocked.

6 **Configure thresholds**
Gate: overall < 0.70 OR any pattern < 75% OR regression > 5pp = fail.

7 **Update baseline post-fix**
After any approved candidate is implemented: re-run, then POST baseline.

Section 10 — Troubleshooting

Symptom	Likely cause	Fix
401 on all /api/lab/* endpoints	Missing or expired JWT	Re-auth; verify NGW_JWT_SECRET matches
Lab tab not visible in UI	Email not in NGW_DEV_EMAILS	Add email to .env, restart server
POST /api/lab/signals returns 422	Missing required pattern_id	Include pattern_id in request body
Signals all show include_in_*=0	signal_source=seeded or internal	Only live source sets include_in_*=1 by default
Benchmark run shows 0 cases evaluated	No benchmark_cases rows in DB	POST cases or use from-gold-set promotion
CI exit_code=1 with no regression_flag	overall_score below 0.70 gate	Review worst-scoring cases; tune blueprint
VLM provider not called	No API key found by auto-detect	Set OPENAI_API_KEY or ANTHROPIC_API_KEY
validate_system_yamls fails	Missing required field in blueprint YAML	Check lights[] array and shadow_signature fields
verify_dataset.py exits 1	Pattern missing entries or bad metadata	Run --fix or manually correct metadata.json
seed_starter_dataset errors	Entries already exist	Run with --force to overwrite
reference_read returns null pattern	No approved entries for that pattern	Add entry with approval_status: approved

Candidate stuck in under_review	No curator has advanced the status	PUT /api/lab/rule_candidates/{id} status: approved

Diagnostic Sequence

1 Health check
GET /health — should return { "status": "ok" }.

2 Lab auth check
GET /api/lab/signals/hygiene — 401 = JWT issue; 200 = Lab auth working.

3 Dataset check
python3 scripts/verify_dataset.py --quick — fast metadata-only pass.

4 Blueprint check
python3 scripts/validate_system_yamls.py — find any YAML schema errors.

5 Signal hygiene
GET /api/lab/signals/hygiene — verify live vs seeded counts are expected.

6 Baseline check
GET /api/lab/benchmarks/baseline — confirm has_baseline: true.

7 **Quick benchmark**
POST /api/lab/benchmarks/run with case_limit=5 — fast pass/fail check.

8 **Inspect worst case**
GET /api/lab/benchmarks/runs/{id}/results — read the error_msg field.

Appendix — Complete API Reference

All /api/lab/* endpoints require Authorization: Bearer with a JWT whose email is listed in NGW_DEV_EMAILS. Exception: POST /api/lab/signals is public — no auth required.

Method + Endpoint	Auth	Description
POST /api/lab/signals	Public	Record session outcome signal
GET /api/lab/signals/summary	Dev JWT	Headline KPIs — days + source params
GET /api/lab/signals/patterns	Dev JWT	Per-pattern aggregated metrics
GET /api/lab/signals/calibration	Dev JWT	Confidence vs outcome gap per pattern
GET /api/lab/signals/recent	Dev JWT	Latest N signals (default source=live)
GET /api/lab/signals/hygiene	Dev JWT	Source breakdown and eligibility flags
POST /api/lab/signals/seed	Dev JWT	Insert 45 synthetic bootstrap rows
POST /api/lab/benchmarks/cases	Dev JWT	Create a benchmark test case
GET /api/lab/benchmarks/cases	Dev JWT	List cases (?pattern_id=X&difficulty;=hard)
PUT /api/lab/benchmarks/cases/{id}	Dev JWT	Update case fields or notes
DELETE /api/lab/benchmarks/cases/{id}	Dev JWT	Remove a benchmark case

GET /api/lab/benchmarks/cases/{id}/history	Dev JWT	Score history for one case across runs
POST /api/lab/benchmarks/cases/from-gold-set/{id}	Dev JWT	Promote gold_set_entries row to case
POST /api/lab/benchmarks/run	Dev JWT	Trigger a manual benchmark run
POST /api/lab/benchmarks/ci-run	CI Secret	CI-triggered run — returns exit_code
GET /api/lab/benchmarks/runs	Dev JWT	List benchmark runs (?limit=N)
GET /api/lab/benchmarks/runs/{id}/results	Dev JWT	Per-case results sorted worst-first
GET /api/lab/benchmarks/pattern-metrics	Dev JWT	Per-pattern metrics with delta vs last run
GET /api/lab/benchmarks/baseline	Dev JWT	Current baseline run info
POST /api/lab/benchmarks/baseline	Dev JWT	Promote latest completed run to baseline

DATA BACKUP

- All lab data is in the SQLite DB at data/ngw.db (dev) or Postgres (prod).
- Reference dataset images are in data/reference_dataset/ — back up alongside DB.
- Before any schema migration: sqlite3 data/ngw.db .dump > backup_$(date +%Y%m%d).sql
- _version.json in reference_dataset/ tracks schema_version — check after updates.

DOCUMENT 18

Analysis Guide

VLM analysis pipeline, shoot-match API, AnalysisResult structure, La

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential
NGW

About This Guide

The NGW analysis system transforms a reference photograph — or a set of natural-language inputs — into a fully structured lighting prescription. Every surface in the system traces back to a single Python call: `analyze_image()` in `engine/orchestrator.py`. This guide documents the seven-stage pipeline, every field in `AnalysisResult`, the shoot-match and lab API endpoints, the results screen card inventory, debug overlay generation, and edge-case handling — drawn directly from the source.

7-Stage	4	24	29
VLM analysis pipeline	Active classifiers	Visual cue signals	Lab API endpoints

AUTH MODEL

- POST /shoot-match — user-facing; requires a valid session JWT only.
- POST /lab/analyze — dev only; JWT email must appear in NGW_DEV_EMAILS.
- GET /lab/reference-dataset/.../debug-overlay — dev only; same auth.
- Set NGW_DEV_MODE=1 locally to bypass auth. Never in production.

Surface	Who uses it	Auth
POST /shoot-match	All users	Session JWT
POST /upload-reference	All users	Session JWT
POST /lab/analyze	Curators / devs	JWT + NGW_DEV_EMAILS

Lab Workbench (UI)	Curators / devs	JWT + NGW_DEV_EMAILS
Debug overlay	Curators / devs	JWT + NGW_DEV_EMAILS

Section 1 — What Is Image Analysis?

Image analysis is the process of reading a reference photograph and determining exactly what lighting setup was used to make it. The NGW engine reads pixel-level evidence — shadow direction and softness, catchlight shape and topology, highlight axis symmetry, background fall-off, specular behavior, and more — and maps that evidence to one of 28 canonical lighting patterns from the reference dataset.

Analysis is triggered in two ways. When a user uploads a reference image, `POST /shoot-match` runs the full pipeline and returns a lighting prescription. When a curator uses the Lab Workbench, `POST /lab/analyze` runs the same pipeline with every internal structure exposed for inspection.

The pipeline is not a single model call. It is a deterministic orchestration of computer-vision passes, multi-stage VLM reconstruction, four independent classifiers, and a solver chain — all producing structured evidence that feeds a single pattern-resolution function. The authoritative pattern is always the output of `resolve_pattern_candidates()`, never a direct model response.

Analysis vs. Shoot-Match

The shoot-match endpoint handles two distinct input modes. When no reference image is provided, it matches natural-language inputs (subject, mood, environment, gear) to the closest pattern using heuristic scoring. When a reference image is present, the full analysis pipeline runs and the natural-language inputs are used only to refine gear recommendations.

Mode	Input	Pipeline depth
Text-only match	subject + mood + environment	Heuristic only — no VLM
Reference analysis	reference image (JPEG/PNG)	Full 7-stage pipeline
Lab analyze	image upload (multipart)	Full pipeline + debug overlay

Section 2 — The 7-Stage Analysis Pipeline

Every reference image passes through seven sequential stages inside `analyze_image()` in `engine/orchestrator.py`. Each stage reads from and writes to the shared `AnalysisResult` object. No stage produces the final pattern — all seven contribute evidence that `resolve_pattern_candidates()` weighs in Stage 6.

Stage	Name	Module	Writes to AnalysisResult
1	Signal Extraction	engine/image_analysis.py	description, cue_report (24 cues)
2	Vision Pipeline	engine/vision_pipeline.py	vision_data (face box, catchlights, regions)
3	Cue Inference	engine/cue_inference.py	cue_inference_result
4	VLM Reconstruction	engine/vlm_reconstruction.py	vlm_reconstruction (12 dimensions)
5	Solver Chain	engine/solver_chain.py	solver_result (contradictions resolved)
6	Pattern Resolution	engine/orchestrator.py	pattern_candidates, authoritative_pattern
7	Reference Read	engine/reference_read.py	reference_analysis (3-layer deep read)

Stage 1 — Signal Extraction

`describe_image()` runs the image through the full cue extraction pipeline and populates `AnalysisResult.cue_report` — a `VisualCueReport` with 24 named fields. Each field is an Optional dataclass that may be None if the signal could not be computed (e.g., face-dependent cues when no face is detected). `cue_report.cues_computed` records how many of the 24 were successfully extracted.

Stage 2 — Vision Pipeline

`_run_extended_pipeline()` executes 16+ independent signal passes against the image using computer vision (MediaPipe, OpenCV). Key outputs include: face bounding box, face detection confidence score, catchlight positions and topology, highlight axis map, shadow direction and hardness, region attribution percentages, and background illumination pattern. Results land in `AnalysisResult.vision_data`.

Stage 3 — Cue Inference

`run_cue_inference_pipeline()` interprets the raw cues from Stage 1 using a rule-based inference engine. It synthesizes shadow direction + height + hardness into a structured lighting hypothesis and feeds `AnalysisResult.cue_inference_result` which becomes the `cue_inference` classifier input in Stage 6.

Stage 4 — VLM Reconstruction

The VLM (Vision-Language Model) is called once and asked to reconstruct the lighting setup across 12 structured dimensions: direction, height, source size, modifier family,

light count, key role, fill role, rim role, background role, bounce likelihood, mixed sources, and pattern name. The response is parsed into `AnalysisResult.vlm_reconstruction`. The `pattern_name` dimension feeds the `lighting_inference` classifier in Stage 6.

Stage 5 — Solver Chain

Six solvers run in sequence to detect and resolve contradictions between the four classifier inputs: consensus_solver consistency_engine contradiction_engine lighting_simulator hypothesis_validator solver_trace. The solver result is stored in `AnalysisResult.solver_result` and influences the confidence adjustments applied in Stage 6.

Stage 6 — Pattern Resolution

`resolve_pattern_candidates()` is the authoritative ranking function. It collects candidate patterns from all four classifiers, applies priority ordering and any contradiction demotions, and returns a `PatternCandidates` object. The `primary` candidate becomes `authoritative_pattern` on `AnalysisResult`.

Stage 7 — Reference Read

`build_reference_photo_analysis()` performs a three-layer deep read of the image: (1) technical read — hardware and modifier identification, (2) aesthetic read — mood, contrast, color temperature, (3) recreation guide — step-by-step instructions for reproducing the setup. This populates `AnalysisResult.reference_analysis` and feeds the reference-image cards in the results screen.

Section 3 — AnalysisResult Structure

`AnalysisResult` is a Python dataclass defined in `engine/orchestrator.py` (lines 649–688). It uses `__slots__` for performance. All 24 fields are set to safe defaults in `__init__` — downstream consumers should always check `ok` before reading inference fields.

Field	Type	Description
ok	bool	False if pipeline raised an unrecoverable error
description	Dict[str, Any]	Raw image description from Stage 1 cue extraction
vision_data	Dict[str, Any]	Extended pipeline output — face box, catchlights, regions
classification	Dict[str, Any]	Image type classification (portrait / product / scene)
cue_report	VisualCueReport	24-field cue report; cues_computed tracks success count
cue_inference_result	Optional[Dict]	Stage 3 cue inference output (shadow pattern hypothesis)
lighting_intel	LightingIntelligence	Light count, modifier, pattern — pre-resolution summary
reference_analysis	ReferenceAnalysis	Three-layer deep read from Stage 7 (tech / aesthetic / guide)
pipeline_results	Optional[Dict]	Raw per-stage outputs; for debug inspection

vlm_description	Any	Raw VLM image description (Stage 1 model call)
vlm_reconstruction	Any	12-dimension structured reconstruction from Stage 4
solver_result	Any	Stage 5 contradiction analysis and resolution trace
debug_data	Dict[str, Any]	Diagnostic data — face_box, overlay path, timing
notes	List[str]	Human-readable warnings from any pipeline stage
authoritative_pattern	str	Final resolved pattern name (e.g. "rembrandt", "loop")
authoritative_pattern_source	str	Classifier that produced it (reference_read / lighting_inference / etc.)
pattern_candidates	PatternCandidates	Ranked candidates from all classifiers — primary + alternates
pattern_confidence	float	Primary candidate confidence score 0.0–1.0
pattern_confidence_label	str	strong (>0.75) / partial (≥0.50) / weak (<0.50)
face_validation	FaceValidation	Face detection quality — detected, confidence, quality tier, yaw, area ratio
signal_reliability	SignalReliability	Signal coverage — available/total, weak signals, missing signals
perception_explanation	PerceptionExplanation	Supporting/contradicting signals, ambiguity flags, reasoning text

edge_case_flags	EdgeCaseFlags	Boolean flags for blown highlights, mixed CT, B&W;, no-face, low-key

PatternCandidates

`PatternCandidates` is the core output of Stage 6. Its `to_dict()` method produces the `patternCandidates` key in every API response.

Field	Type	Description
primary	PatternCandidate	Highest-ranked candidate — defines authoritative_pattern
alternates	List[PatternCandidate]	Other viable candidates in confidence order
needs_review	bool	True when classifiers significantly disagree
contradictions	List[str]	Human-readable contradiction descriptions

Each `PatternCandidate` has: `pattern` (canonical name), `source` (classifier), `confidence` (0–1), `rank` (1-based within classifier output).

Section 4 — Pattern Resolution & Classifiers

`resolve_pattern_candidates()` collects candidates from four independent classifiers and applies a strict priority ordering. Only one function determines the authoritative pattern — downstream code must read `pattern_candidates.authoritative_pattern`, never reach directly into classifier outputs.

Classifier Priority Order

Priority	Classifier	Source module	Basis
0 (highest)	reference_read	engine/reference_read.py	Richest analysis — full three-layer deep read with VLM
1	lighting_inference	engine/lighting_inference.py	Vision-based catchlight topology + shadow geometry
2	cue_inference	engine/cue_inference.py	Shadow direction + height + hardness rule synthesis
3	light_structure	engine/vision_pipeline.py	Nose-shadow geometry direct pattern derivation
—	unknown fallback	orchestrator.py	When no classifier produces a result

Contradiction Demotion

When vision signals (triangle_isolation, shadow_density, lr_asymmetry, highlight_symmetry) strongly contradict the `reference_read` primary candidate, its effective priority is demoted from 0 to 2 and its confidence is halved. This allows

competing candidates that align better with raw pixel evidence to win. The demotion reason is recorded in `PatternCandidates.contradictions`.

Confidence Label Tiers

Label	Confidence range	Meaning
strong	> 0.75	Classifiers agree, signals clear — high trust
partial	0.50 – 0.75	Reasonable evidence — use with normal care
weak	< 0.50	Ambiguous signals — flag for curator review

28 CANONICAL PATTERNS

- split, rembrandt, loop, butterfly, clamshell, triangle, broad, short
- gobo, flat, rim_only, high_key, low_key, flat_fashion, window_portrait
- golden_hour, overcast_natural, ring_light, bare_bulb_editorial, strip_dramatic
- short_fashion_key, soft_editorial_key, editorial_rim_key
- tabletop_soft_product, bottle_backlight, athletic_rim_sculpt
- window_negative_fill, hybrid, unknown

Section 5 — Shoot-Match API

The shoot-match endpoint is the primary user-facing analysis surface. It accepts either natural-language inputs or a base64-encoded reference image. When an image is present the full 7-stage pipeline runs; otherwise heuristic matching is used.

POST /shoot-match

Request body (JSON)

```
{
  "subject":        "headshot",          // woman | man | child | couple | group …
  "mood":           "editorial",
  "environment":    "studio",             // studio | indoor | outdoor
  "ceiling":        "normal",             // normal | low
  "gearMode":       "anyGear",            // anyGear | myGear
  "gear":           ["softbox", "rim"],   // optional gear list
  "skinTone":       "medium",             // optional
  "referenceImage": "<base64-string>",    // optional — triggers full pipeline
  "masterMode":     null                  // optional override
}
```

Response (condensed)


```
{
  "status": "ok",
  "requestId": "abc123",
  "processingMs": 2340,
  "authoritative_pattern": "rembrandt",
  "patternCandidates": {
    "primary_candidate": { "pattern": "rembrandt", "source": "reference_read",
                           "confidence": 0.87, "confidence_label": "strong" },
    "alternate_candidates": [ { "pattern": "loop", "source": "lighting_inference",
                                "confidence": 0.61 } ],
    "needs_review": false,
    "contradictions": []
  },
  "faceValidation": { "face_detected": true, "face_quality": "good",
                      "face_confidence": 0.94, "face_yaw": 12.3,
                      "face_box_area_ratio": 0.18 },
  "signalReliability": { "signals_available": 19, "signals_total": 24,
                         "overall_signal_strength": 0.81,
                         "weak_signals": [], "missing_signals": ["pose_induced_sha…
  "edgeCaseFlags": { "no_face": false, "blown_highlights": false,
                     "mixed_color_temperature": false, "bw_processing": false },
  "lightingIntelligence": { ... },
  "referenceImageAnalysis": { ... },
  "cards": [ ... ]
}
```


POST /upload-reference

Accepts a multipart image upload. Runs the full analysis pipeline and returns the same response shape as `POST /shoot-match` with a reference image. Use this when the reference image is too large for base64 inline encoding (max inline size ≈ 8 MB; upload limit is 10 MB).

Section 6 — Lab Workbench

The Lab Workbench is the curator tool for running full-fidelity analysis with all internal structures exposed. It lives in the Workbench tab of the Lab screen (`ui/src/screens/LabScreen.jsx`) and is backed by `POST /lab/analyze`.

POST /lab/analyze

Request (multipart/form-data)

```
image:   <file upload>      # JPEG or PNG, max 10 MB
debug:   true | false       # query param — generates visual overlay
```

Response fields

```
{
  "status":         "ok",
  "image_path":     "/static/uploads/abc123.jpg",
  "description":    { ... },          // Stage 1 raw cue extraction
  "reference_analysis": { ... },      // Stage 7 three-layer deep read
  "vlm":            { ... },          // Stage 1 VLM description
  "vlm_available":  true,
  "vlm_reconstruction": {             // Stage 4 — 12 dimensions
    "direction":    "45_camera_left",
    "height":       "above_eye",
    "source_size":  "large_soft",
    "modifier_family": "softbox",
    "light_count":  2,
    "key_role":     "main_key",
    "fill_role":    "fill",
    "rim_role":     "none",
    "background_role": "none",
    "bounce_likelihood": "low",
    "mixed_sources": false,
    "pattern_name": "rembrandt"
  },
  "cv":             { ... },          // Stage 2 vision_data
  "classification": { ... },          // image type classification
  "lighting_inference": { ... },      // lighting_intel summary
  "solver":         { ... },          // Stage 5 solver chain result
  "analyzed_by":    "dev@example.com",
  "analyzed_at":    "2025-11-14T09:23:11Z",
  "debug_overlay_url": "/static/debug/abc123_overlay.jpg"  // only when debug=true
}
```

DEBUG OVERLAY FLAG

- Pass ?debug=true to generate a visual overlay image.
- The overlay annotates: face bounding box (cyan), shadow direction arrows (amber),
- catchlight positions (white dots), highlight regions (blue), background regions (green).
- URL is returned in debug_overlay_url — accessible at GET /static/debug/<filename>.
- Overlays are stored in /static/debug/ and not automatically cleaned up.

Workbench tab	Purpose
Workbench	Run POST /lab/analyze — upload image, inspect full response, view debug overlay
Gold Sets	Manage gold-set reference entries for benchmark evaluation
Candidates	Propose and track rule candidates through review lifecycle
Reference Dataset	Browse the 28-pattern reference library; ingest new images
Signals	Inspect session_signals hygiene — live/seeded/internal counts
Learning Ops	Trigger candidate auto-generation from accumulated signals
Benchmarks	Run benchmark suite, view pass/soft-pass/fail by pattern

Section 7 — Results Screen Cards

The results screen (`ui/src/screens/ResultsScreen.jsx`) assembles the analysis output into a sequence of cards. Cards are conditionally rendered based on whether a reference image was provided and the user's subscription tier. Phase labels organize cards into logical groups.

Card component	Always shown	Requires ref image	Description
LookSummaryCard	Yes	No	Pattern name, confidence label, key modifier and light count — free tier
BlueprintCard	Yes	No	Full blueprint YAML details — modifier, positions, distances, ratios
DiagramCard	Yes	No	SVG lighting diagram with clock-position annotations
ShootSetupCard	Yes	No	Plain-English setup instructions
SpaceCheckCard	Yes	No	Minimum room dimensions and ceiling requirements
CameraSubjectCard	Yes	No	Camera position, focal length, and subject framing guidance
HowToTestCard	Yes	No	Test-shot checklist for on-set verification
WhatToLookForCard	Yes	No	Visual confirmation cues when setup is correct
QuickFixesCard	Yes	No	Common failure modes and single-sentence corrections

RecommendedKitsCard	Yes	No	Gear recommendation cards matching the blueprint modifier family
OtherSetupsCard	Yes	No	Alternate patterns with similar aesthetic — jump to different result
SkinToneCard	Yes	No	Skin-tone specific power and modifier adjustment notes
SignalQualityCard	Yes	No	Classifier agreement, confidence tier, signal availability summary
TestShotCard	Yes	No	On-set test shot evaluation entry point — links to Shoot Mode
MySetupsCard	Yes	No	Save-to-kit and previously saved setups for this pattern
FeedbackCard	Yes	No	Outcome recording — nailed_it / close / failed — writes session_signal
ReferenceImageCard	No	Yes	The uploaded reference image with zoom overlay
RefImageReadCard	No	Yes	Technical read — hardware and modifier identification
RefLightingCard	No	Yes	Lighting analysis — direction, height, source, modifier
RefRecreationCard	No	Yes	Step-by-step recreation guide from Stage 7 reference read
RefInterpretationsCard	No	Yes	Alternative interpretations and confidence notes from the deep read

Section 8 — Debug Overlays

When `?debug=true` is passed to `POST /lab/analyze`, the engine generates a visual annotation overlay on top of the uploaded image. The overlay is saved to `/static/debug/_overlay.jpg` and its URL is returned as `debug_overlay_url` in the response.

What Overlays Annotate

Annotation	Color	Source signal
Face bounding box	Cyan	vision_data.face_box from MediaPipe
Shadow direction arrow	Amber	cue_report.primary_shadow_direction
Catchlight dots	White	vision_data.catchlights (per catchlight)
Highlight regions	Blue fill	cue_report.highlight_axis_map
Background region	Green outline	vision_data.region_attribution.background
Light position estimate	Orange dot	Derived from shadow + catchlight intersection
Pattern label	White text	authoritative_pattern + confidence_label

Accessing overlays

```
# Run lab analyze with debug flag
curl -X POST "http://localhost:8000/lab/analyze?debug=true" \
     -H "Authorization: Bearer <dev_jwt>" \
     -F "image=@reference.jpg"

# Response includes:
# "debug_overlay_url": "/static/debug/abc123_overlay.jpg"

# Fetch overlay:
curl http://localhost:8000/static/debug/abc123_overlay.jpg -o overlay.jpg
```

OVERLAY STORAGE

- Overlays are NOT automatically deleted — they accumulate in /static/debug/.
- Periodically clean up: rm -rf /static/debug/*.jpg (dev only).
- In production, overlays are behind dev auth — they are not publicly accessible.
- Overlay filenames are derived from the upload hash, not the original filename.

Section 9 — Visual Cue Report (24 Signals)

`VisualCueReport` in `engine/image_analysis_models.py` holds every signal the pipeline extracts from a single image. Each field is Optional — None means the signal could not be computed. `cue_report.cues_computed` tracks how many of the 24 were successfully extracted. Face-dependent signals return None when no face is detected.

Field	Face-dependent	What it measures
shadow_edge_hardness	Partial	Soft vs. hard shadow falloff — encodes source distance and diffusion
primary_shadow_direction	Yes	Clock position of key light derived from facial shadow geometry
vertical_light_angle	Yes	Height angle — high / eye-level / low — from shadow below brow/chin
catchlight_position	Yes	Clock position of catchlight in the iris
catchlight_shape	Yes	Round, softbox, ring, window, strip, or absent
catchlight_topology	Yes	Single / dual / ring / fill-only — counts and distribution
highlight_axis_map	No	Directional histogram of highlight regions across the frame
highlight_symmetry	No	Left/right highlight balance — encodes off-axis key strength
continuous_source_signals	No	Whether source appears continuous (LED panel) vs. flash

bounce_contributor	Partial	Probability and direction of a fill via bounce or reflector
separation_light	No	Presence and strength of rim / hair / separation light
off_axis_key	Yes	Degree to which key is placed off camera axis
light_structure	Yes	Direct pattern name derived from nose-shadow geometry
highlight_to_shadow_transition	Partial	Gradient width encodes feathering and modifier proximity
contrast_ratio	No	Highlight-to-shadow luminance ratio
subject_background_separation	No	Subject-background luminance and color separation
background_illumination	No	Background light pattern: even / gradient / spot / dark / natural
specular_highlight_behavior	No	Specular shape and distribution on skin/surfaces
reflection_architecture	No	Reflector-bounce geometry analysis
multi_shadow_detection	Partial	Multiple shadows indicating multi-light setup
environmental_shadow_continuity	No	Outdoor vs. studio shadow cues; foliage, window, mixed CT hints
pose_induced_shadow_interference	Yes	Body-pose shadows that could confuse direction detection
tonal_processing_estimation	No	B&W; detection; heavy contrast grading; tonal processing hints

shadow_interruption_pattern	Yes	Gobo / grid / projected pattern detected via shadow shape

Section 10 — Complete API Endpoint Reference

All 29 analysis-related endpoints across four route files. Auth column: Session = valid user JWT; Dev = JWT + email in NGW_DEV_EMAILS; None = unauthenticated.

Shoot-Match Routes

Method	Path	Auth	Purpose
POST	/shoot-match	Session	Full analysis or heuristic match — primary user endpoint
POST	/upload-reference	Session	Multipart image upload full pipeline analysis

Lab Analysis

Method	Path	Auth	Purpose
GET	/lab/status	Dev	Check dev access — returns email + NGW_DEV_MODE flag
POST	/lab/analyze	Dev	Full pipeline analysis + optional debug overlay

Gold Set

Method	Path	Auth	Purpose

GET	/lab/gold-set	Dev	List all gold-set entries
GET	/lab/gold-set/{entry_id}	Dev	Retrieve single entry by ID
POST	/lab/gold-set	Dev	Create new gold-set entry from analysis
PUT	/lab/gold-set/{entry_id}	Dev	Update entry (pattern, notes, flags)
DELETE	/lab/gold-set/{entry_id}	Dev	Remove entry from gold set
POST	/lab/gold-set/evaluate	Dev	Batch evaluate all gold-set entries against current engine

Rule Candidates

Method	Path	Auth	Purpose
GET	/lab/candidates	Dev	List all candidates
GET	/lab/candidates/{id}	Dev	Retrieve single candidate
POST	/lab/candidates	Dev	Create candidate from curator observation
PUT	/lab/candidates/{id}	Dev	Update status (proposed approved rejected)
DELETE	/lab/candidates/{id}	Dev	Remove candidate

Reference Library

Method	Path	Auth	Purpose
POST	/lab/reference-library/ingest	Dev	Ingest new reference image into library
POST	/lab/reference-library/rebuild-index	Dev	Rebuild search index after bulk changes
POST	/lab/reference-library/generate-legacy-sidecars	Dev	Generate sidecar JSON for legacy images
POST	/lab/reference-library/validate	Dev	Validate all library entries for schema compliance
GET	/lab/reference-library	Dev	List all library entries
GET	/lab/reference-library/{id}	Dev	Retrieve single entry metadata
POST	/lab/reference-library	Dev	Create entry (direct, no image processing)
PUT	/lab/reference-library/{id}	Dev	Update entry metadata
DELETE	/lab/reference-library/{id}	Dev	Remove from library
POST	/lab/reference-library/from-reconstruction	Dev	Create entry from VLM reconstruction output

Reference Dataset

Method	Path	Auth	Purpose

POST	/lab/reference-dataset/ingest	Dev	Ingest image into the 28-pattern reference dataset
GET	/lab/reference-dataset	Dev	List all dataset entries
GET	/lab/reference-dataset/version	Dev	Dataset version string
GET	/lab/reference-dataset/manifest	Dev	Full manifest with per-pattern counts
GET	/lab/reference-dataset/{pattern_id}/{ref_id}	Dev	Entry metadata
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/image	Dev	Full-resolution image
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/thumbnail	Dev	Thumbnail image
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/debug-overlay	Dev	Debug overlay image
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/approve	Dev	Approve entry for inclusion in training
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/reject	Dev	Reject entry with reason
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/reprocess	Dev	Re-run analysis on existing entry

Section 11 — Performance & Edge Cases

The full analysis pipeline processes a typical portrait image in 2–4 seconds on a production server (VLM latency dominates). Debug overlay generation adds 200–500 ms. The pipeline is designed to degrade gracefully — if any individual stage fails, `AnalysisResult.ok` remains True and the stage's output field is left at its safe default. Only a total pipeline failure sets `ok = False`.

Face Detection Edge Cases

Five of the 24 visual cues are face-dependent and return None when no face is detected: `primary_shadow_direction`, `vertical_light_angle`, `pose_induced_shadow_interference`, `shadow_interruption_pattern`, and `light_structure`. When face detection fails, the engine falls back to non-face cues (highlight axis map, background illumination, environmental shadows) and sets `edge_case_flags.no_face = True` and `face_validation.face_quality = "none"`.

FaceValidation Fields

Field	Type	Description
face_detected	bool	True when MediaPipe face detection returns a bounding box
face_confidence	float	MediaPipe detection confidence score 0.0–1.0
face_quality	str	"good" (large area, yaw present) | "partial" | "none"
face_yaw	float|None	Face yaw angle in degrees; None if yaw cannot be computed

face_box_area_ratio	float	face_area / image_area — small ratio (<0.01) = "tiny_face" flag

EdgeCaseFlags

Flag	Trigger condition
no_face	face_validation.face_detected is False
blown_highlights	contrast_ratio.ratio > 8.0
mixed_color_temperature	Both warm_* and cool_* hints in environmental_shadow_continuity
outdoor_foliage_shadows	"dappled_foliage" in environmental_shadow_continuity.environment_hints
window_light_gradient	background_illumination.pattern == "gradient" AND natural indicators
extreme_low_key	classification.brightness == "low" AND light_structure.shadow_density > 0.5
bw_processing	tonal_processing_estimation.is_bw is True

SignalReliability

Populated by the perception layer after all stages complete. `signals_available` counts non-None cue fields on `VisualCueReport`. `weak_signals` lists cue names whose confidence attribute is below 0.3. `overall_signal_strength` is computed from `cue_report.overall_confidence()`. When `signals_available < 10`, the `"low_signal_count"` ambiguity flag is set in `perception_explanation.ambiguity_flags`.